



REPÚBLICA BOLIVARIANA DE VENEZUELA
UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
COORDINACIÓN GENERAL DE PREGRADO
PROYECTO DE CARRERA: INGENIERÍA INFORMÁTICA

Evaluar la seguridad de sistemas web a través de pruebas automatizadas de penetración con la finalidad de generar un reporte detallado con los hallazgos encontrados.

Autores:

Br. Marelis Victoria Marcano Bermúdez.

Br. Eladio Rafael Godoy Parada.

Tutora:

MSc. Roca, Dubraska.

Puerto Ordaz, abril de 2025.



REPÚBLICA BOLIVARIANA DE VENEZUELA
UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
COORDINACIÓN GENERAL DE PREGRADO
PROYECTO DE CARRERA: INGENIERÍA INFORMÁTICA

***Evaluar la seguridad de sistemas web a través de pruebas
automatizadas de penetración con la finalidad de generar un reporte
detallado con los hallazgos encontrados.***

*“Trabajo de Grado presentado como requisito parcial para optar al título de
Ingeniero en Informática”*

Autores:

Br. Marelis Victoria Marcano Bermúdez.

C.I: V-22.831.021

Br. Eladio Rafael Godoy Parada.

C.I: V-21.451.328

Tutora:

MSc. Roca, Dubraska.

Puerto Ordaz, abril de 2025.



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA

COORDINACIÓN GENERAL DE PREGRADO

COORDINACIÓN DE ING. EN INFORMÁTICA

ACTA DE EVALUACIÓN DE TRABAJO DE GRADO

Los suscritos, reunidos hoy 16/05/2025 a las 7:00 PM, bajo la plataforma GOOGLE MEET, miembros del Jurado designado por la Comisión de Trabajo de Grado según control CGT/036/17, de fecha 24 de Abril de 2017 cumplidos con todos los aspectos formales para la Presentación Oral y Pública y en atención a lo dispuesto en el Artículo 32 del Reglamento de Trabajo de Grado de Pregrado de la Universidad Nacional Experimental de Guayana (UNEG), para evaluar el Trabajo de Grado titulado: "EVALUAR LA SEGURIDAD DE SISTEMAS WEB A TRAVÉS DE PRUEBAS AUTOMATIZADAS DE PENETRACIÓN CON LA FINALIDAD DE GENERAR UN REPORTE DETALLADO CON LOS HALLAZGOS ENCONTRADOS." presentado por los bachilleres: MARELIS VICTORIA MARCANO BERMÚDEZ C.I. V22831021 y ELADIO RAFAEL GODOY PARADA C.I. V21451328 en cumplimiento con lo establecido en el mismo reglamento, para optar al grado de Ing. en Informática, hacemos constar que habiéndolos evaluado hemos acordado emitir el siguiente veredicto:

Aprobación con mención: Honorífica.

Presidente del Jurado: _____

Prof: HUAMAN HUAMAN ANA YSABEL

C.I.:V22824279

Primer Miembro: _____

Prof: MARIANELLA COLMENARES COLMENARES

C.I.:V12233067

Segundo Miembro: _____

Prof: GLENDIS DEL VALLE BARDAN ROSQUEZ

C.I.:V10387550

Tutor Académico: _____

Prof: DUBRASKA ROCA

C.I.:V11173754



Código de Verificación CACE: A741DF2D4

2025/06/05 11:27



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
COORDINACIÓN GENERAL DE PREGRADO
COORDINACIÓN DE ING. EN INFORMÁTICA

ACTA DE MENCIÓN DE TRABAJO DE GRADO

Los suscritos, reunidos hoy 16/05/2025 a las 7:00 PM, bajo la plataforma GOOGLE MEET, miembros del Jurado designado por la Comisión de Trabajo de Grado según control CGT/036/17, de fecha 24 de Abril de 2017 cumplidos con todos los aspectos formales para la Presentación Oral y Pública y en atención a lo dispuesto en el Artículo 32 del Reglamento de Trabajo de Grado de Pregrado de la Universidad Nacional Experimental de Guayana (UNEG), para evaluar el Trabajo de Grado titulado: "EVALUAR LA SEGURIDAD DE SISTEMAS WEB A TRAVÉS DE PRUEBAS AUTOMATIZADAS DE PENETRACIÓN CON LA FINALIDAD DE GENERAR UN REPORTE DETALLADO CON LOS HALLAZGOS ENCONTRADOS." presentado por los bachilleres: MARELIS VICTORIA MARCANO BERMÚDEZ C.I. V22831021 y ELADIO RAFAEL GODOY PARADA C.I. V21451328, dieron como veredicto: Aprobado con mención: Honorífica, ya que posee un tratamiento novedoso y significativo del tema investigado, en tanto constituye un aporte diferente al área y/o tema abordado, posee una pertinencia contextual del problema abordado (el problema de investigación y sus resultados responden a demandas claramente establecidas de orden social, institucional, disciplinar o de otro orden), articula de manera clara y precisa los aspectos estructurales medulares del trabajo; a saber: problema, objetivos, metodología y resultados, posee una fundamentación teórica suficiente y adecuada al tema, al problema y a aquellos aspectos identificados como relevantes y hace uso claro y preciso de la lengua en el desarrollo de su texto académico, donde se diferencia de manera formal las ideas propias y ajenas con su respectiva identificación de la fuente. Los abajo firmantes dan fé de todo lo anterior.

Presidente del Jurado: _____

Prof: HUAMAN HUAMAN ANA YSABEL

C.I.: V22824279



Primer Miembro: _____

Prof: MARIANELLA COLMENARES COLMENARES.

C.I.: V12233067

Segundo Miembro: _____

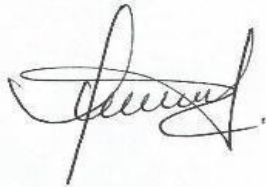
Prof: GLENDIS DEL VALLE BARDAN

C.I.: V10387550

APROBACIÓN DE LA TUTORA ACADÉMICA

En mi carácter de tutora del Trabajo de Grado de Pregrado, titulado **Evaluar la seguridad de sistemas web a través de pruebas automatizadas de penetración con la finalidad de generar un reporte detallado con los hallazgos encontrados**, presentado por los ciudadanos *Marelis Victoria Marcano Bermúdez*, titular de la Cédula de Identidad No. V-22.831.021 y *Eladio Rafael Godoy Parada*, titular de la Cédula de Identidad V-21.451.328, para optar al título de Ingeniero en Informática, considero que dicho trabajo investigativo reúne los requisitos y méritos necesarios y suficientes para ser sometido a la evaluación y presentación oral y pública por parte de jurado examinador designado.

En Puerto Ordaz, a los dieciséis días del mes de abril del año dos mil veinticinco.



MSc. Dubraska Roca
CI: N°. V-11.173.754

DEDICATORIA

A Dios, por Su infinita fidelidad, porque Sus planes siempre son mejores que los míos y, gracias a ello, hoy estoy aquí.

A mi amado esposo, Andrés Centeno, por ser mi apoyo incondicional, mi compañero de vida y el mejor cómplice en cada aventura.

A mis padres, Melquis y Natalia, por sus oraciones y su guía espiritual, pero también por su esfuerzo inquebrantable para que siempre tuviéramos un camino lleno de oportunidades.

A mi hermana, Meralis, por ser la mejor hermana mayor, por allanar el camino antes que yo y compartir sus aprendizajes para que pudiera evitar tropiezos.

A mi hermano, Ernesto, por su apoyo y cariño encubierto, por cada noche en la que esperó por mí y me acompañó de regreso a casa.

Marelis Victoria Marcano Bermúdez

DEDICATORIA

A Dios porque sin su compañía y guía, no podría haber alcanzado este logro tan importante. A mi abuelita Carmen Caballero, porque su presencia incluso cuando esta no fue cien por ciento física, me dio fuerza. Su forma de vivir, su entrega y su temple han sido faros constantes que me han ayudado a moldear el hombre que soy hoy.

A mi madre, Fanny Parada Caballero, que ha sido todo: guía, apoyo, consejera, refugio. Gracias por enseñarme que la responsabilidad, la honestidad y la rectitud no son solo valores, sino formas de caminar por la vida con dignidad. Nunca has soltado mi mano, incluso cuando el camino estaba cuesta arriba.

A mi padre, Godofredo Godoy, por ser un pilar importante en mi vida y por nunca dejar de recordarme la importancia de prepararse, de estudiar, de aspirar siempre a más. Tu confianza en mis capacidades y tu forma de impulsarme a crecer profesionalmente han sido una fuente permanente de motivación.

A mi esposa, Valeska Salazar, quien ha estado a mi lado en cada momento desde que este sueño comenzó a tomar forma. Tu fe en mí, tu paciencia, tu amor, apoyo y comprensión han sido fundamentales para no perder el rumbo.

A una persona muy especial en mi vida Yannet Oliveros por todos los consejos, enseñanzas, y el cariño brindado, que me han ayudado a crecer en muchos aspectos de mi vida.

A Valeria y Aranssa. Ustedes aún son pequeñas, pero ya son mi mayor fuente de inspiración. Todo esto también es para ustedes, para que cuando crezcan, sepan que con esfuerzo, pasión y entrega se puede alcanzar cualquier meta que uno se proponga. Que esto sea un ejemplo de que los sueños, aunque tarden, sí se cumplen.

Y finalmente a mí mismo, porque a pesar de las situaciones y el tiempo, nunca perdí la convicción y me propuse concluir esta bonita etapa de mi vida.

Eladio Rafael Godoy Parada

AGRADECIMIENTOS

A Dios, por guiarnos en cada paso de este camino, darnos fortaleza en los momentos difíciles y sabiduría para culminar con éxito esta etapa. Su fidelidad ha sido el pilar fundamental en todo este proceso.

A mi compañera Kivivi, por su compromiso, entrega y esa chispa única que hizo de este proyecto una experiencia enriquecedora y divertida. Gracias por tu apoyo constante y tu energía inagotable.

A mi compañero Eladito, por su dedicación, constancia y compañerismo genuino. Gracias por nunca rendirte y por ser un apoyo firme en cada desafío que enfrentamos juntos.

A nuestros familiares, por su amor incondicional, sus oraciones, su paciencia y por creer en nosotros incluso cuando nosotros mismos dudamos. Cada palabra de aliento fue clave para llegar hasta aquí.

A nuestra tutora, la profesora Dubraska Roca, por su guía, compromiso y acompañamiento cercano durante todo el desarrollo de este trabajo. Su orientación fue crucial para lograr nuestros objetivos.

A la Universidad Nacional Experimental de Guayana y a todos los profesores que formaron parte de nuestra formación académica. Gracias por sembrar en nosotros las herramientas y los valores que hoy nos acompañan.

ÍNDICE GENERAL

	Pág.
LISTA DE TABLAS	V
LISTA DE FIGURAS	VI
RESUMEN	VII
INTRODUCCIÓN	1
CAPÍTULO I	3
PLANTEAMIENTO DEL PROBLEMA	3
Objetivos de la Investigación	8
Justificación de la investigación	9
Alcance de la investigación	10
CAPÍTULO II	12
MARCO TEÓRICO	12
Antecedentes de la investigación	12
Bases Teóricas de la investigación	14
Bases Legales	22
CAPÍTULO III	29
MARCO METODOLÓGICO	29
Tipo de investigación	29
Diseño de la investigación	30
Técnicas de recolección de datos	30
Población	30
Muestra	30
Procedimiento metodológico de la investigación	32
CAPÍTULO IV	35
RESULTADOS	35
CAPÍTULO V	24
CONCLUSIONES	121
RECOMENDACIONES	123
REFERENCIAS	124

LISTA DE TABLAS

Tabla 1: Cronograma	65
Tabla 2: Riesgos y mitigaciones del proyecto	65
Tabla 3: Posibles desafíos técnicos	79
Tabla 4: Códigos de vulnerabilidad	104
Tabla 5: Requerimientos funcionales	119
Tabla 6: Requerimientos no funcionales	120

LISTA DE FIGURAS

Figura 1: Caso de uso del usuario.....	73
Figura 2: Diagrama Escaneo de un dominio	74
Figura 3: Diagrama de visualizar los resultados.....	75
Figura 4: Diagrama para listar escaneos.....	75
Figura 5: Diagrama para descargar reporte	76
Figura 6: Diagrama de Componentes	77
Figura 7: Modelo entidad-relación (ER) de la base de datos	78
Figura 8: Diseño del Logotipo.....	82
Figura 9: Estructura general de la interfaz de usuario (UI) y diseño inicial.....	84
Figura 10: Dashboard	85
Figura 11: Reportes	86
Figura 12: Acerca de	87
Figura 13: Disclaimer	88
Figura 14: Reporte de resultados para conjunto de pruebas	100
Figura 15: Pantalla de Inicio	107
Figura 16: Pantalla de Dashboard.....	108
Figura 17: Pantalla de Resultados	109
Figura 18: Pantalla de Reportes.....	109
Figura 19: Pantalla de Reportes.....	110
Figura 20: Pantalla de Reportes.....	111
Figura 21: Pantalla Acerca de.....	112
Figura 22: Pantalla Disclaimer	112
Figura 23: Estructura de los repositorios Git.....	114
Figura 24: Estructura de los repositorios Git.....	115
Figura 25: Pruebas funcionales del sistema integrado.....	116
Figura 26: Revisión de compatibilidad con navegadores y resolución de pantalla.....	119

REPÚBLICA BOLIVARIANA DE VENEZUELA
UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
COORDINACIÓN GENERAL DE PREGRADO
PROYECTO DE CARRERA: INGENIERÍA INFORMÁTICA

Evaluar la seguridad de sistemas web a través de pruebas automatizadas de penetración con la finalidad de generar un reporte detallado con los hallazgos encontrados.

Autores: Marelis Marcano y Eladio Godoy

Tutora: MSc. Dubraska Roca

Puerto Ordaz, abril 2025

Resumen

La presente investigación tiene como propósito realizar evaluaciones sistemáticas de la seguridad de sistemas web a través de pruebas automatizadas de penetración. El objetivo principal es identificar vulnerabilidades críticas y generar reportes detallados que permitan fortalecer la ciberseguridad de las organizaciones. Se presentan herramientas especializadas, metodologías de pruebas y se analizan los beneficios de automatizar este proceso. Tipo de investigación documental tecnológica, con un enfoque descriptivo, un diseño no experimental, con una población que la representan los casos prácticos, ya analizados por las empresas especialistas en la temática sobre seguridad de la información en el año 2024, identificada en el planteamiento del problema.

Se creó una arquitectura escalable basada en el modelo cliente-servidor. Dicha arquitectura fue implementada haciendo uso de modernas tecnologías como Django (Backend), Vue.js (Frontend), Docker (Contenedores), tecnologías que permitieron utilizar un Framework robusto que sigue el patrón MVC, facilitando el desarrollo estructurado, con herramientas integradas para autenticación, manejo de sesiones y seguridad (por ejemplo, protección contra XSS y CSRF), creando interfaces interactivas y manejables, empaquetando la aplicación y dependencias en contenedores, asegurando un entorno consistente, para mejorar la portabilidad y escalabilidad del sistema. El desarrollo del sistema se llevó a cabo siguiendo la metodología SCRUM. Con una duración de 15 semanas. Logrando crear un sistema automatizado de las pruebas de penetración: SQL Injection, Cross-Site Scripting (XSS), Weak Hashing Algorithm, Insecure Direct Object References, Security Misconfiguration, Known Vulnerable Components (Retire.js), Insufficient Logging and Monitoring, Server-Side Request Forgery (SSRF), Insecure Deserialization, Authentication Bypass, en los sistemas web.

Descriptores: Evaluación, seguridad, sistema web, pruebas automatizadas de penetración.

INTRODUCCIÓN

La evaluación de la seguridad de sistemas web a través de pruebas automatizadas de penetración es un enfoque fundamental para identificar vulnerabilidades antes de que sean explotadas por agentes malintencionados, fortaleciendo la postura de seguridad de las organizaciones y garantizar la protección frente a posibles ataques cibernéticos. Estas pruebas simulan técnicas utilizadas por atacantes, empleando herramientas automatizadas para explorar debilidades en la configuración, el código y la infraestructura del sistema.

Al realizar estas evaluaciones, se genera un reporte detallado con los hallazgos encontrados, que permite a los administradores implementar medidas correctivas efectivas y reforzar la seguridad general del sistema. Este proceso no solo mejora la resistencia del sistema frente a amenazas, sino que también garantiza la confidencialidad, integridad y disponibilidad de la información crítica que manejan los sitios web.

Entre los beneficios de este proceso se consideran:

- Identificación temprana de riesgos: Permite tomar medidas preventivas antes de sufrir un ataque real.

- Optimización de recursos: Al ser un proceso automatizado, se reduce el tiempo necesario para evaluar grandes sistemas.

- Cumplimiento normativo: Ayuda a las empresas a cumplir con estándares de seguridad como PCI DSS, ISO 27001, entre otros.

- Fortalecimiento de la seguridad: Facilita la creación de un entorno más robusto y resistente frente a amenazas externas.

En ese sentido la estructura de esta investigación está conformada por los siguientes capítulos:

Capítulo I: Planteamiento del problema. Incluye el planteamiento del problema, los objetivos de la investigación, la justificación y el alcance de esta última, permitiendo contextualizar la situación problema a un marco de estudio.

Capítulo II: Marco Teórico. Incluye los antecedentes de investigación y bases teóricas que fundamentan el estudio.

Capítulo III: Marco Metodológico. Comprende toda la metodología de investigación aplicada su diseño, tipo, nivel, población y muestra de estudio, además del procedimiento de la investigación y la selección y definición de metodologías, herramientas y lenguajes usados en el desarrollo.

Capítulo IV: Resultados. En este capítulo se exponen los resultados obtenidos de la investigación, así como las fases de ejecución del desarrollo de la aplicación y su entrega final.

Finalmente se presentan las conclusiones, recomendaciones, referencias y anexos del trabajo de investigación.

CAPÍTULO I

PLANTEAMIENTO DEL PROBLEMA

Una amenaza de ciberseguridad es una indicación de que un hacker o actor malicioso está intentando obtener acceso no autorizado a una red para lanzar un ataque cibernético. Al respecto, en un informe realizado por IBM (2024), con relación al tipo de amenazas de ciberseguridad, se evidenció que el costo promedio global de una filtración de datos en 2023 fue de 4.45 millones de dólares, un aumento del 15 % en 3 años.

De acuerdo a la definición que plantea el Glosario de términos de Ciberseguridad (2021) señala una Amenaza como una:

Circunstancia desfavorable que puede ocurrir y que cuando sucede tiene consecuencias negativas sobre los activos provocando su indisponibilidad, funcionamiento incorrecto o pérdida de valor. Una amenaza puede tener causas naturales, ser accidental o intencionada. Si esta circunstancia desfavorable acontece a la vez que existe una vulnerabilidad o debilidad de los sistemas o aprovechando su existencia, puede derivar en un incidente de seguridad.

Además, informes de Fortinet, Sophos, Trend Micro, Watchguard y Kaspersky (2024) señalan el aumento del malware evasivo, tanto básico como cifrado, las detecciones de malware aumentaron en un 80%; lo que pone en evidencia el volumen creciente de amenazas que afectan al perímetro de las redes. Este incremento se observó especialmente en América y Asia-Pacífico. Otras amenazas de acuerdo al informe son: Resurgimiento de ciberataques living-off-the-land, Ataques a servidores Exchange, Malware como servicio, Ataques de ransomware de alto nivel, ciberataques Zero Day, Infiltración interna y reclutamiento de organizaciones objetivo, Ataques ligados a eventos geopolíticos y sociales, Ampliación de tácticas, técnicas y procedimientos en el espacio 5G, Explotación de blockchain para extorsión, Potenciación de la ingeniería social con IA Generativa, Armas de datos contra modelos de machine learning en la nube, Ataques a la cadena de suministro, Amenazas de gusanos en la nube, Resurgimiento de rootkits del kernel, Amenazas a los sistemas MFT y Sistemas de Internet de las Cosas extendido (XIOT).

En este contexto, la evaluación de la seguridad de sistemas web a través de pruebas automatizadas de penetración es esencial en el mundo digital actual, donde los ataques cibernéticos son cada vez más sofisticados y frecuentes, los puntos claves que destacan su importancia:

1. **Protección de datos sensibles:** Ayuda a identificar y mitigar vulnerabilidades que podrían exponer información confidencial, como datos personales, financieros o estratégicos, garantizando así la privacidad y la confianza de los usuarios.
2. **Prevención de ataques:** Al simular técnicas de ataque reales, las pruebas automatizadas permiten anticiparse a las estrategias que podrían ser empleadas por hackers y reducir los riesgos de brechas de seguridad.
3. **Cumplimiento de normativas:** Estas evaluaciones son fundamentales para asegurar que los sistemas cumplen con estándares de seguridad, como PCI DSS o GDPR, lo que protege a las empresas de sanciones legales y mejora su reputación.
4. **Optimización de recursos:** Al ser automatizadas, estas pruebas son rápidas y eficientes, permitiendo a los equipos de seguridad centrarse en la implementación de soluciones en lugar de en la identificación manual de problemas.
5. **Fortalecimiento de la ciberseguridad:** Este proceso contribuye a crear sistemas más robustos, capaces de resistir amenazas avanzadas, lo que resulta crucial para la continuidad operativa de las organizaciones.

En un mundo cada vez más interconectado, invertir en evaluaciones de seguridad como estas no solo es una necesidad, sino una estrategia inteligente para protegerse frente a los riesgos del ciberespacio.

Un aspecto importante lo representan las vulnerabilidades, que representan una debilidad o fallo en un sistema, red, software, hardware o proceso que puede ser explotado por un atacante para obtener acceso no autorizado, causar interrupciones, robar información, o realizar otras acciones maliciosas. Las vulnerabilidades pueden surgir debido a errores de diseño, implementación, configuración incorrecta o falta de actualizaciones de seguridad.

De acuerdo a la definición en el Glosario de términos de Ciberseguridad (2021) se define como una debilidad o fallo de un sistema que puede ser aprovechado con

finés maliciosos (normalmente mediante un programa que se denomina exploit). Cuando se descubre el desarrollador del software o hardware lo solucionará publicando una actualización de seguridad del producto.

Entre las más frecuentes que se presentan se tienen las siguientes:

Inyección de código (SQL Injection): Consiste en inyectar código SQL malicioso en los formularios de entrada de un sitio web para manipular la base de datos. Esto puede permitir a los atacantes robar información sensible, modificar datos o incluso eliminar registros completos.

Cross-site Scripting (XSS): Permite a los atacantes inyectar scripts maliciosos en páginas web que serán ejecutadas por otros usuarios. De esta manera, pueden robar cookies, redirigir a los usuarios a sitios maliciosos o incluso tomar el control de sus cuentas.

Autenticación y gestión de sesiones: Contraseñas débiles: El uso de contraseñas fáciles de adivinar sigue siendo una de las principales causas de brechas de seguridad.

Sesiones no seguras: Si las sesiones no se gestionan de forma adecuada, los atacantes pueden secuestrarlas y hacerse pasar por usuarios legítimos.

Errores de configuración: Permisos inadecuados: Si los archivos y directorios no tienen los permisos correctos, los atacantes pueden obtener acceso a información sensible o incluso ejecutar código.

Configuraciones por defecto: Muchos sistemas y aplicaciones vienen con configuraciones por defecto que pueden ser fácilmente explotadas.

De allí la importancia del tema, además señala Computing (2024), que la complejidad y sofisticación de las amenazas se multiplican, la guerra híbrida entre Rusia, Ucrania y la invasión de Gaza han sido un punto de inflexión en el ámbito del cibercrimen. Los ciberataques han sido cada vez más sofisticados y han afectado a empresas y gobiernos de todo el mundo. Entre algunas amenazas están: Denegación de servicio (DoS): Ataques diseñados para hacer que un servicio o recurso sea inaccesible para sus usuarios legítimos; Malware: Software malicioso que puede infectar sistemas y robar información y el Phishing: Ataques de ingeniería social que buscan engañar a los usuarios para que revelen información confidencial.

Al respecto, los especialistas señalan un incremento en la complejidad de las operaciones de ciberdelincuencia como Servicio (CaaS) y la gran influencia de la inteligencia artificial (IA). Se prevé que evolucionen las tácticas de amenazas persistentes avanzadas, que la IA generativa sea cada vez más utilizada en actividades maliciosas, y que los atacantes se concentren en sectores claves para exigir rescates mayores. Además, se espera un aumento en los ataques de tipo zero day, la manipulación de datos para extorsión en modelos de aprendizaje automático en la nube, y la amenaza de ataques de gusanos en entornos de nube y Kubernetes. Estas tendencias suponen retos importantes para la ciberseguridad, subrayando la necesidad de colaboración, ciber resiliencia organizacional y atención a las nuevas vulnerabilidades.

De acuerdo a estudio presentado por EY Global Cybersecurity Leadership Insights Study (2024) publicado en https://www.ey.com/es_ve/insights/cybersecurity/panorama-ciberseguridad-latinoamerica-riesgos-enfrentan-empresas?form=MG0AV3 que contempla una investigación que reúne las perspectivas de más de 500 líderes en seguridad informática, plantea que Latinoamérica enfrenta varios desafíos y oportunidades, donde algunos puntos claves a considerar son:

Filtraciones de Datos: el 62% de las empresas latinoamericanas han sufrido alguna filtración de datos en el último año. Esto subraya la necesidad urgente de mejorar las medidas de seguridad.

Inversión en ciberseguridad: Aproximadamente el 50% de las empresas en la región invierten entre 10 y 49 millones de dólares en ciberseguridad. Sin embargo, aún hay un largo camino por recorrer para alcanzar un nivel de protección adecuado.

Tecnologías Emergentes: La adopción de tecnologías como la nube, el Internet de las Cosas (IoT), la inteligencia artificial (IA) y la computación cuántica plantea nuevos riesgos y desafíos para la seguridad cibernética.

Madurez Cibernética: Un reporte del Banco Interamericano de Desarrollo (BID) destaca que hay brechas significativas en la capacidad de seguridad cibernética de los países de la región. Es crucial mejorar la resiliencia cibernética para proteger el crecimiento económico y la sostenibilidad.

Infraestructura de Seguridad: Muchas empresas en la región necesitan actualizar su infraestructura de seguridad para protegerse contra amenazas persistentes (ATP). La falta de parches y actualizaciones puede dejar a los sistemas vulnerables.

Ataques a páginas Web: Los ataques a páginas web son comunes en la región, con herramientas como Muieblackcat utilizadas para detectar y explotar vulnerabilidades en sitios web.

Dispositivos Móviles: Los dispositivos móviles, especialmente los Android, son blancos frecuentes de ataques en América Latina. Es importante implementar soluciones de seguridad específicas para estos dispositivos.

Estos aspectos resaltan la importancia de mejorar la ciberseguridad en la región para proteger la información y los recursos de las organizaciones. Asimismo, en Venezuela la ciberseguridad es un tema de gran relevancia debido a la creciente cantidad de ciberataques y la necesidad de proteger la información y los sistemas informáticos.

Señala el Diario.com (2024), que Venezuela se encuentra entre los países con más ciberataques, ocupando el puesto 80 en una lista global. Esto subraya la urgencia de mejorar las medidas de seguridad cibernética. A pesar de que en 2016, el gobierno implementó el Plan Nacional de Ciberseguridad y Ciberdefensa, además, existe la Ley Especial contra Delitos Informáticos del año 2001, que busca prevenir y sancionar delitos cibernéticos, sin embargo, su efectividad es cuestionable.

Por otra parte, en cuanto a Infraestructura y Capacitación, la Superintendencia de Servicios de Certificación Electrónica (SUSCERTE) responsable de implementar medidas de seguridad informática y formar a personas en temas de seguridad cibernética, aún está en proceso. A pesar de los avances tecnológicos y las nuevas legislaciones, el país enfrenta desafíos significativos en la protección de sus sistemas web. La inversión en software de seguridad y la creación de protocolos de seguridad en las organizaciones son esenciales para mejorar la situación. Los ciberataques no solo afectan a las empresas, sino también a los usuarios individuales, causando pérdidas económicas y socavando la confianza en los sistemas digitales.

Considerando la situación planteada, se presentan las siguientes interrogantes de la investigación:

¿Cuál es el estado del arte de la seguridad de los sistemas web, que vulnerabilidades existen?

En función de la interrogante anterior y con la finalidad de dar respuesta a la misma, surgen las siguientes preguntas:

¿Cuál es la situación actual de la seguridad de los sistemas web?

¿Cuáles son las vulnerabilidades que se presentan en los sistemas web, para identificar el nivel de riesgo en esas plataformas?

¿Cómo será el diseño del plan de pruebas automatizadas con base a la información recopilada para evaluar la seguridad de un sistema web?

¿Cómo será la aplicación que permite las pruebas automatizadas, utilizando metodologías de pentesting para generar un reporte detallado con los hallazgos encontrados?

En base a lo planteado en las preguntas de investigación, se plantearon los objetivos que se indican a continuación:

OBJETIVO GENERAL

Evaluar la seguridad de sistemas web a través de pruebas automatizadas de penetración con la finalidad de generar un reporte detallado con los hallazgos encontrados.

OBJETIVOS ESPECÍFICOS

1. Analizar el estado del arte de la seguridad de los sistemas web.
2. Caracterizar las vulnerabilidades de los sistemas web, para identificar el nivel de riesgo en las plataformas.
3. Diseñar el plan de pruebas automatizadas con base a la información recopilada para evaluar la seguridad de un sistema web.

4. Desarrollar el sistema de pruebas automatizadas con base a la información recopilada para evaluar la seguridad de un sistema web, utilizando la metodología SCRUM.

Justificación de la investigación

Desde el punto de vista teórico, el trabajo de investigación contiene elementos actualizados de la temática, con una revisión documental amplia, de las pruebas automatizadas de penetración (pentesting) que ofrecen una metodología efectiva para evaluar la seguridad de los sistemas web. A través de estas pruebas, se pueden identificar vulnerabilidades potenciales de manera sistemática y eficiente, simulando ataques reales que los ciberdelincuentes llevan a cabo. Esta evaluación no solo ayuda a detectar debilidades en el sistema, sino que también proporciona una base sólida para generar un reporte detallado con los hallazgos encontrados.

Desde el punto de vista práctico, las pruebas automatizadas representan una implementación, en un contexto donde los ataques cibernéticos son cada vez más frecuentes y devastadores, esta investigación se posiciona como una herramienta valiosa para el desarrollo de estrategias de ciberseguridad más robustas. La aplicación de pruebas automatizadas de penetración y la implementación de las recomendaciones propuestas sirven como modelo para otras organizaciones que busquen mejorar su postura de seguridad de manera efectiva y eficiente.

En ese sentido, la seguridad de los sistemas web se ha vuelto una preocupación crítica en el panorama tecnológico actual, donde las amenazas cibernéticas evolucionan constantemente y se vuelven más sofisticadas. Las organizaciones, tanto públicas como privadas, dependen cada vez más de aplicaciones web para realizar sus operaciones, almacenar datos sensibles y proporcionar servicios a los usuarios. Esta dependencia hace que la protección de estos sistemas sea fundamental para garantizar la integridad, confidencialidad y disponibilidad de la información.

Desde el punto de vista metodológico, la realización de esta tesis es crucial porque representa una visión detallada del estado actual de la seguridad de los sistemas web

evaluados. Además, el reporte detallado con los hallazgos encontrados de las pruebas de penetración automatizadas permite a las organizaciones mejorar sus prácticas de seguridad, mitigando los riesgos asociados a las vulnerabilidades detectadas. Este enfoque proactivo no solo contribuye a la protección de los datos y los recursos de la organización, sino que también fortalece la confianza de los usuarios en los sistemas web utilizados.

Finalmente, desde el punto de vista social esta investigación no solo contribuye a la seguridad informática en general, sino que también proporciona un modelo replicable para cualquier organización que requiera implementar sistemas y requieren las pruebas automatizadas para poder validarlos. Los resultados derivados de este estudio son fundamentales e impactan en el desarrollo de las estrategias de ciberseguridad más robustas y adaptables frente a las amenazas emergentes, fomentando un entorno seguro y resiliente.

Alcance de la investigación

El sistema automatizado de pruebas de penetración ofrece varias opciones para evaluar y mejorar la seguridad de los sistemas web. El alcance incluye las siguientes opciones:

1. Escaneo de Vulnerabilidades para identificar posibles puntos débiles, estos representan fallas conocidas, se realiza mediante el uso de herramientas automatizadas que escanean aplicaciones web y servidores, que pueden ser explotados por atacantes.
2. Simulación de Ataques reales para probar la resistencia de los sistemas web frente a diferentes tipos de amenazas, como inyecciones SQL, cross-site scripting (XSS) y otras vulnerabilidades relacionadas con la entrada de datos maliciosos, y otros ataques comunes.
3. Evaluación de configuraciones: para detectar especificaciones incorrectas en servidores, bases de datos o componentes web.

4. Cumplimiento de Normativas estándares y regulaciones de seguridad, como ISO 27001, HIPAA, SOC2 y GDPR.
5. Pruebas de autenticación y autorización: para verificar si los controles de acceso y la autenticación son adecuados y resistentes.
6. Informes sobre las vulnerabilidades encontradas, con la finalidad de generar un reporte detallado con los hallazgos encontrados, y corregir las vulnerabilidades detectadas.

CAPÍTULO II

MARCO TEÓRICO

El presente capítulo tiene como finalidad contextualizar la situación problema dentro de un conjunto de teorías, metodologías y estudios al definir y explicar diferentes términos antecedentes y bases teóricas que enmarcan la temática a investigar. A continuación, se presenta el marco teórico o referencial, señala Hernández y Sampieri (2018) que implica analizar y exponer de una manera organizada las teorías, investigaciones previas y los antecedentes en general que se consideren válidos y adecuados para contextualizar y orientar tu estudio. Al respecto, se incluyen los antecedentes de la investigación, bases teóricas y bases legales del estudio a realizar.

Antecedentes de la investigación

Aguilar, J. (2021) presentó la tesis: Retos y oportunidades en materia de ciberseguridad de América Latina frente al contexto global de ciber amenazas a la seguridad nacional y política exterior. En la UNAM, para obtener el Título Profesional de Dr. En la Facultad de Ciencias Políticas y Sociales de la Universidad Nacional Autónoma de México. Este estudio establece que la región latinoamericana posee fuertes carencias en el desarrollo de una política nacional de ciberseguridad y en la construcción de capacidades para enfrentar los riesgos y amenazas provenientes del ciberespacio en dimensiones que afectan la seguridad nacional y política exterior.

Este trabajo brindó al presente estudio aportes teóricos relacionados con contexto global de ciber amenazas y situación de América latina.

Díaz, E. (2018) presentó la tesis: Análisis de metodologías para pruebas de penetración mediante ethical hacking – Bogotá, Colombia, en la Universidad Nacional Abierta y a Distancia “UNAD” Escuela de Ciencias Básicas, Tecnología e Ingeniería proyecto de seguridad informática, para optar por el Título Profesional de Especialista

en Seguridad Informática. Realizó una evaluación de la seguridad de una plataforma utilizada en el sector educativo, golpeado fuertemente a nivel mundial, trayendo consigo problemas para algunas instituciones que han sido víctimas de distintos ataques cibernéticos, por ejemplo, de ransomware. La plataforma que se auditó es utilizada por 50 colegios privados, sin embargo, nunca había sido puesta a prueba y no se conocía qué vulnerabilidades podría llegar tener y que tan expuesta se encontraba la información de quienes la utilizan.

El tipo de investigación se clasificó como trabajo enfocado hacia la modalidad de Monografía. En cuanto a las técnicas de recolección de datos se manejaron la entrevista y la técnica de focus group. Los resultados demostraron que la plataforma educativa posee una seguridad mejorable o insuficiente ya que se encontraron 7 vulnerabilidades, entre estas: 1 crítica, 3 altas, 2 medias y 1 de criticidad baja.

La vinculación de este trabajo a la presente investigación es porque permite la revisión de los aportes teóricos relacionados con la seguridad de la información en los sistemas.

Castro, C. (2018) presentó el trabajo: Pruebas de penetración e intrusión, para optar al Título de especialista en Seguridad Informática en la Universidad Piloto de Colombia. El trabajo expone algunas metodologías existentes para llevar a cabo pruebas de penetración e intrusión a través de la descripción de sus características, etapas y actividades que se llevan a cabo en cada una de ellas. Esto con el propósito de aproximar al lector al valor que, en la actualidad, la industria ha otorgado a esta actividad en aras de aportar al campo de la seguridad y la transformación digital, siendo desarrolladas y respaldadas por prestigiosas entidades y/o comunidades al punto de ser consideradas como un marco de referencia para llevar a cabo las pruebas de penetración y facilitar la adopción e implementación de su práctica en las organizaciones.

Este antecedente aportó ideas para usar un enfoque definido, permitiendo adaptar las especificaciones en el plan de pruebas que se propone.

Bases Teóricas de la investigación.

Representan el fundamento conceptual que sustenta el trabajo de investigación. Estas proporcionan un marco de referencia para comprender el problema de investigación.

1. Seguridad de la Información.

La seguridad de la información es un concepto que se involucra cada vez más en muchos aspectos de la sociedad hiperconectada, en gran parte como resultado de la adopción casi ubicua de la tecnología de información y comunicación.

Según la ISO/IEC (2016), la seguridad de la información se podría definir como aquellos procesos, buenas prácticas y metodologías que busquen proteger la información y los sistemas de información del acceso, uso, divulgación, interrupción, modificación o destrucción no autorizada. Esto conlleva a proteger los datos y recursos de infraestructura tecnológica de aquellos quiénes intentarían hacer un mal uso de ellos.

Por otro lado, en un sentido general, seguridad significa proteger los activos. Esto puede significar protegerlos de atacantes que invaden las redes, desastres naturales, condiciones ambientales adversas, cortes de energía, robo o vandalismo u otros estados indeseables.

Actualmente se utilizan las computadoras en cualquier contexto, se trabaja con computadoras, se juega con ellas en casa, se utilizan para clases en línea, se compran productos por Internet, se utilizan teléfonos inteligentes para verificar los saldos bancarios, entre otras funcionalidades que se presentan en la vida cotidiana. Aunque la tecnología permite ser más productivos y acceder a una gran cantidad de información con solo un click del ratón, también conlleva una gran cantidad de problemas de seguridad. Si la información sobre los sistemas utilizados se expone a un ciberdelincuente, las consecuencias pueden ser terribles.

2. Confidencialidad, Integridad y Disponibilidad.

Señala Parada (2018) que tres de los conceptos principales en seguridad de la información son precisamente la confidencialidad, integridad y disponibilidad, comúnmente conocida como la tríada de la seguridad de la información. La tríada de la CIA, que ha sido utilizada por más de 20 años, y está centrada en la seguridad de los datos.

La confidencialidad es un concepto similar, pero no igual, a la privacidad. La confidencialidad es un componente necesario de la privacidad y se refiere a nuestra capacidad de proteger nuestros datos de aquellos que no están autorizados para verlos. La confidencialidad es un concepto que puede implementarse en muchos niveles de un proceso.

En cuanto a la integridad se refiere a la capacidad de evitar que los datos se modifiquen de manera no autorizada o indeseable. Esto podría significar el cambio o la eliminación no autorizada de los datos o partes de ellos, o podría significar un cambio o eliminación autorizado, pero no deseable. Para mantener la integridad, no solo se necesita tener los medios para evitar cambios no autorizados en los datos, sino también la capacidad de revertir los cambios autorizados que deben deshacerse.

Existen mecanismos que permiten controlar la integridad en los sistemas de archivos de muchos sistemas operativos modernos como Windows y Linux. Con el fin de prevenir cambios no autorizados, tales sistemas a menudo implementan permisos que restringen las acciones que un usuario no autorizado puede realizar en un archivo determinado.

Además, algunos de estos sistemas y muchas aplicaciones, como las bases de datos, pueden permitir deshacer o revertir cambios que no son deseables. La integridad es particularmente importante cuando se discuten los datos que proporcionan la base para otras decisiones.

Por último, la disponibilidad se refiere a la capacidad de acceder a los datos cuando se necesitan. La pérdida de disponibilidad puede referirse a una amplia variedad de interrupciones en cualquier parte de la cadena de comunicaciones que permiten acceder a los datos. Tales problemas pueden ser el resultado de pérdida de energía, problemas del sistema operativo o de la aplicación, ataques a la red de datos, compromiso de un sistema u otros problemas que impidan a los usuarios acceder a su información. Tales problemas son comúnmente causados por los ya conocidos y avanzados ataques de denegación de servicio (DoS) (Wang et al., 2017).

3. Amenazas, vulnerabilidades y riesgos

Amenazas.

Son los tipos de ataques que tienen el potencial de causar daño a los activos. Las amenazas tienden a ser específicas de ciertos entornos, particularmente en el mundo de la seguridad de la información.

Vulnerabilidades.

Las vulnerabilidades son debilidades que pueden usarse para dañarnos. En esencia, son agujeros que pueden ser explotados por amenazas para causarnos daño. Una capacidad de vulnerabilidad podría ser un sistema operativo específico o una aplicación que estamos ejecutando, una ubicación física donde hemos elegido ubicar nuestro edificio de oficinas, un centro de datos que se puebla sobre la capacidad de su sistema de aire acondicionado, la falta de respaldo generadores u otros factores.

Riesgo

El riesgo es la probabilidad de que algo malo suceda. Para poder tener un riesgo en un entorno particular, se necesita tener una amenaza y una capacidad de vulnerabilidad que la amenaza específica pueda explotar. Por ejemplo, si existe una estructura hecha de madera y la prendemos fuego, se tiene una amenaza (el fuego) y una vulnerabilidad que coincide (la estructura de madera). En este caso, definitivamente existe un riesgo. Del mismo modo, si hay la misma amenaza de incendio, pero la estructura está hecha

de concreto, ya no hay un riesgo creíble, porque la amenaza no tiene una vulnerabilidad que explotar.

Algunas organizaciones, como la Agencia de Seguridad Nacional de los Estados Unidos (NSA, por sus siglas en inglés), agregan un factor adicional a la ecuación de amenaza / vulnerabilidad / riesgo, en forma de impacto. Si se considera que el valor del activo amenazado es un factor, esto puede cambiar si se observa un riesgo presente o no.

Controles

Para poder mitigar el riesgo, se pueden establecer medidas que ayuden a garantizar que se tenga en cuenta un tipo determinado de amenaza. Estas medidas se denominan controles. Los controles se dividen en tres categorías: física, lógica y administrativa.

4. Conceptos Fundamentales de Ciberseguridad.

La ciberseguridad se refiere a métodos de uso, procesos y tecnologías para prevenir, detectar y recuperarse de daños a la confidencialidad, integridad y disponibilidad de la información en el ciberespacio (Leyva, 2015). Representan el conjunto de prácticas y tecnologías diseñadas para proteger sistemas informáticos, redes y datos frente a accesos no autorizados, ataques o daños. Dentro de este campo, la seguridad de sistemas web adquiere relevancia dada la creciente dependencia de las organizaciones en plataformas digitales.

5. Pruebas de penetración.

Las pruebas de penetración, conocidas como **pentesting**, son evaluaciones diseñadas para identificar vulnerabilidades en sistemas informáticos, aplicaciones web, redes, y otros elementos de infraestructura tecnológica. Estas pruebas ayudan a las

organizaciones a protegerse contra amenazas reales simulando los métodos que podría usar un atacante para comprometer sus sistemas.

Tipos de Pruebas de Penetración

- ☐ **Pruebas internas:** Evalúan la seguridad dentro de la red corporativa simulando ataques desde un usuario malintencionado interno.
- ☒ **Pruebas externas:** Se centran en identificar vulnerabilidades en sistemas expuestos al público, como servidores web o aplicaciones.
- ☒ **Pruebas de aplicaciones móviles y web:** Investigan fallas específicas en software y aplicaciones desarrolladas.
- ☒ **Pruebas de redes inalámbricas:** Analizan la seguridad de las conexiones Wi-Fi y redes inalámbricas.
- ☐ **Pruebas de API:** Buscan vulnerabilidades en interfaces de programación, fundamentales para las integraciones modernas.

Proceso General

El pentesting suele seguir estas etapas:

- ☒ **Recopilación de información:** Identificar objetivos, configuraciones y tecnologías utilizadas.
- ☒ **Escaneo de vulnerabilidades:** Usar herramientas automatizadas para buscar posibles fallas.
- ☒ **Explotación de vulnerabilidades:** Intentar comprometer el sistema usando esas fallas.
- ☐ **Informes:** Proporcionar un resumen detallado con recomendaciones para corregir vulnerabilidades.

6. Pruebas Automatizadas de Penetración.

Las pruebas automatizadas de penetración son una evolución del pentesting tradicional. Utilizan herramientas especializadas para detectar vulnerabilidades de forma eficiente y rápida. Estas herramientas replican comportamientos maliciosos,

evaluando múltiples escenarios de ataque sin necesidad de intervención humana constante. Ejemplos de herramientas populares son Metasploit, Nessus y Burp Suite.

7. Importancia de la Evaluación de Seguridad Web.

Los sistemas web son uno de los objetivos principales de los cibercriminales, ya que pueden contener datos sensibles y son accesibles desde cualquier lugar del mundo. Evaluar su seguridad con pruebas automatizadas permite:

- Identificar vulnerabilidades rápidamente.
- Prevenir brechas de seguridad.
- Cumplir con normativas internacionales como PCI DSS o GDPR.
- Reducir los costos asociados a posibles incidentes de seguridad.

a. Proceso de Evaluación Automatizada.

El proceso incluye las siguientes etapas:

- Reconocimiento: Recolectar información sobre el sistema objetivo.
- Escaneo: Detectar vulnerabilidades y configuraciones inseguras.
- Explotación simulada: Probar las debilidades identificadas de manera ética.
- Reporte: Documentar los hallazgos y recomendar soluciones.

b. Marco Normativo y Estándares Relacionados.

Diversas normativas y estándares internacionales respaldan la realización de pruebas de penetración, como:

ISO 27001 - Gestión de seguridad de la información.

La norma ISO 27001 es un estándar internacional que establece los requisitos para implementar, mantener y mejorar un Sistema de Gestión de Seguridad de la Información (SGSI). Su objetivo principal es garantizar la confidencialidad, integridad y disponibilidad de la información dentro de una organización. Este estándar internacional establece un marco para la gestión de la seguridad de la información en las organizaciones, incluyendo la protección de sistemas web y datos sensibles. Es una norma internacional de gran relevancia. Se centra en establecer, implementar, mantener y mejorar un Sistema de Gestión de Seguridad de la Información (SGSI), garantizando la protección de datos sensibles frente a riesgos cibernéticos.

Entre los principales aspectos de ISO 2700 se consideran lo siguiente:

☒ **Estructura del SGSI:**

Define un marco sistemático para identificar, gestionar y reducir riesgos relacionados con la seguridad de la información.

☒ **Evaluación de riesgos:**

Exige la identificación de activos, amenazas y vulnerabilidades, así como la implementación de controles para mitigar riesgos.

☐ **Controles de seguridad:**

La norma incluye un conjunto de medidas recomendadas como protección de datos, gestión de accesos, seguridad física y monitoreo.

☐ **Cumplimiento y certificación:**

Las organizaciones que adoptan ISO 27001 pueden certificarse, demostrando su compromiso con la seguridad de la información y mejorando su confianza ante clientes y socios.

☐ **Ciclo PHVA (Planificar-Hacer-Verificar-Actuar):**

Promueve un proceso de mejora continua en la gestión de seguridad. Esta norma es ampliamente adoptada en sectores donde la protección de datos es fundamental, como el financiero, tecnológico y de salud.

Características de ISO 27001.

1. Evaluación de riesgos: Identificar y gestionar los riesgos relacionados con la seguridad de la información.

2. Políticas de seguridad: Definir y aplicar políticas claras para proteger los datos.
3. Controles de seguridad: Implementar medidas técnicas, organizativas y físicas para mitigar riesgos.
4. Mejora continua: Promover un enfoque de mejora constante en la gestión de la seguridad.

OWASP (Open Web Application Security Project)

Guías para mejorar la seguridad en aplicaciones web.

OWASP es una organización sin fines de lucro dedicada a mejorar la seguridad de las aplicaciones web. Ofrece recursos, herramientas y guías ampliamente reconocidas en el ámbito de la ciberseguridad.

Recursos de OWASP:

1. **OWASP Top Ten:** Una lista de los riesgos de seguridad más críticos en aplicaciones web. Es una referencia esencial para desarrolladores y equipos de seguridad.
2. **Web Security Testing Guide (WSTG):** Una guía completa para realizar pruebas de seguridad en aplicaciones web y servicios.
3. **OWASP Cheat Sheets:** Documentos breves que ofrecen consejos prácticos sobre temas específicos de seguridad.
4. **OWASP ASVS (Application Security Verification Standard):** Un estándar para verificar la seguridad de aplicaciones.

BASES LEGALES

En cuanto a las Bases Legales, existen diversas leyes y reglamentos que regulan la seguridad de los sistemas web, diseñados para proteger la información y garantizar un entorno digital seguro.

1. **Ley Sobre Mensajes De Datos Y Firmas Electrónicas (2000).** Tiene como objetivo principal otorgar validez jurídica a los mensajes de datos y firmas electrónicas, promoviendo su uso en transacciones electrónicas y garantizando su seguridad y autenticidad. Esta ley regula aspectos clave como:
 - a. **Reconocimiento legal:** Establece que los mensajes de datos y las firmas electrónicas tienen el mismo valor jurídico que los documentos físicos y las firmas manuscritas.
 - b. **Proveedores de servicios de certificación:** Regula a las entidades encargadas de emitir certificados electrónicos, asegurando que cumplan con estándares de seguridad y confiabilidad.
 - c. **Definiciones clave:** Incluye conceptos como mensajes de datos, firmas electrónicas, certificados electrónicos y sistemas de información, proporcionando un marco claro para su aplicación.
 - d. **Adaptabilidad tecnológica:** La ley está diseñada para ser flexible y adaptarse a los avances tecnológicos futuros, garantizando su relevancia a largo plazo.
2. **Ley Orgánica De Telecomunicaciones (2000).** En su artículo 2 plantea los objetivos generales de esta Ley, donde se considera: Incorporar y garantizar el cumplimiento de las obligaciones de Servicio Universal, calidad y metas de cobertura mínima uniforme, y aquellas obligaciones relativas a seguridad y defensa, en materia de telecomunicaciones.
3. **Ley Especial Contra Los Delitos Informáticos (2001).** Esta ley establece medidas para prevenir y sancionar delitos relacionados con el uso indebido de tecnologías de información, incluyendo accesos no autorizados y ataques a sistemas web.
4. **Ley Orgánica De Ciencia, Tecnología e Innovación (2005)**

Con relación a las Tecnologías de información la ley plantea en su Artículo 18: La autoridad nacional con competencia en materia de ciencia, tecnología, innovación y sus aplicaciones, ejercerá la dirección en el área de tecnologías de información. En tal sentido, deberá:

- Establecer políticas sobre la generación de contenidos en la red, respetando la diversidad, así como el carácter multiétnico y pluricultural de nuestra sociedad.
- Resguardar la inviolabilidad del carácter confidencial de los datos electrónicos obtenidos en el ejercicio de las funciones de los órganos y entes públicos.
- Democratizar el acceso a las tecnologías de información.

5. **Reglamento General de Protección de Datos** (GDPR - Unión Europea): Este reglamento protege la privacidad y los datos personales de los usuarios, exigiendo a las empresas implementar medidas de seguridad robustas para evitar filtraciones.

6. **Ley de Servicios de la Sociedad de la Información y del Comercio Electrónico** (LSSI - España): Regula aspectos relacionados con la seguridad en el comercio electrónico y la protección de datos en línea.

Ingeniería del software

La ingeniería comprende la aplicación de conocimientos, herramientas y técnicas con el propósito de resolver problemas de la sociedad, para Pressman (2010) la ingeniería del software comprende una tecnología con cuatro (4) capas, estas son:

- **Capa de procesos:** permite el desarrollo racional y oportuno del software de cómputo.
- **Capa de métodos:** proporciona la experiencia técnica para elaborar el software
- **Capa de herramientas:** proporciona un proceso automatizado o semiautomatizado para el proceso y los métodos.
- **Capa de calidad:** Permite evaluar la calidad del software, garantizando un funcionamiento correcto del mismo.

Así mismo el proceso general para el desarrollo del software comprende las siguientes actividades:

Comunicación: Donde se busca colaborar con el cliente para establecer una serie de objetivos, características y funciones del software.

Planeación: donde se define una estructura compuesta por una serie de tareas técnicas por realizar, los riesgos probables y los recursos que requieren.

Modelado: Donde se crean modelos y diagramas que permitan entender mejor el software a desarrollar.

Construcción: Donde se genera el código a través de editores de texto y/o compiladores.

Despliegue: Donde se entrega el software que finalmente el usuario utiliza.

Desarrollo web

Barba J (2014) define al desarrollo web como “la programación necesaria para la construcción del sitio web”. El desarrollo web está conformado por dos capas, la capa de cliente que tiene comprende la interfaz gráfica con la que el usuario puede interactuar y que es capaz de realizar peticiones para obtener información, acceder a hipervínculos, modificar un recurso, descargar un documento entre otros. Por otro lado, tenemos la capa del servidor que es la encargada de procesar las peticiones realizadas por el cliente, se trata de un hardware ya sea una computadora o un “cluster” con la capacidad de procesar peticiones realizando acciones como, conexión con servicios, operaciones sobre una base de datos y ejecución de actividades que requieren un alto nivel de procesamiento y/o recursos.

Por otro lado, Maini, M. (2015), indica que antes de desarrollar una web es necesario contar un plan y preparación donde se identifique la audiencia objetivo, se preparan una serie de líneas guías a seguir y se diseñe una estructura base para la página que sea bien pensada y fácil de navegar. Toda esta preparación indicada por el autor permite tener una interfaz de usuario a medida para las personas que utilicen el sistema con el objetivo de promover la utilización del resultado del desarrollo web.

Arquitectura Cliente Servidor

El modelo cliente-servidor es el modelo predominante actualmente para aplicaciones web, móviles, sistemas distribuidos y redes de computadoras. Permite la construcción de sistemas complejos de manera flexible, portable y altamente escalable.

La arquitectura cliente-servidor es el modelo de sistemas distribuidos más conocido y con más estudios hasta la fecha, autores más recientes como Bricher, M (2017) definen esta arquitectura como:

“Un modelo de sistemas distribuidos en donde múltiples clientes realizan múltiples peticiones a múltiples servidores, que procesan la información y devuelven los resultados a quien corresponda los procesos de cliente interactúan con los procesos de servidor individuales en equipos anfitriones (host) potencialmente separados, con el fin de acceder a los recursos compartidos que administran”.

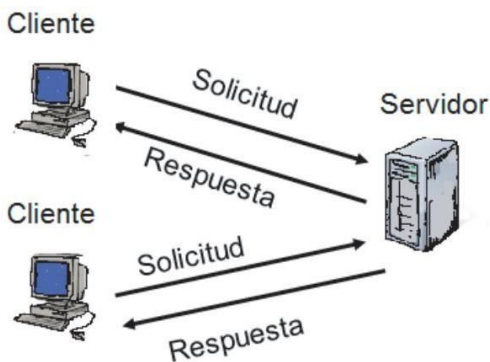


Figura 1: Arquitectura cliente-servidor
Fuente: Fuentes, F (2015).

Lenguajes de Desarrollo

Los lenguajes de desarrollo o lenguajes de programación, de acuerdo a Espinoza, L (2019), son una serie de comandos que escritos secuencialmente y respetando la sintaxis resuelven una situación problemática en cuanto a información. Estos ayudan al programador a llevar modelos de negocio expresados en lenguaje humano a

instrucciones que el computador ejecuta, lo cual permite resolver necesidades del ser humano y generar un valor para una empresa o desarrollador.

Al traducir estos problemas a un código entendible por máquinas, los lenguajes de programación permiten automatizar procesos, analizar datos, crear software, apps y sistemas informáticos que resuelven necesidades de distintos usuarios y organizaciones. De esta manera, generan un importante valor tanto para las empresas tecnológicas que comercializan software, como para sus clientes que obtienen soluciones digitales a sus requerimientos.

Muchos negocios y emprendimientos tecnológicos dependen directamente de la creación de software a través de lenguajes de programación para funcionar y generar ganancias. Desde aplicaciones web y móviles, pasando por software de análisis de datos y de automatización industrial, hasta videojuegos y sistemas operativos, todos requieren del poder expresivo de los lenguajes de programación para modelar soluciones que resuelvan problemáticas específicas y tengan un valor en el mercado.

Existen muchos tipos de lenguajes de desarrollo cada uno con sus ventajas, desventajas, propósito y ambiente de ejecución, entre los más conocidos encontramos a Javascript, Python, PHP, Java, Rust, Ruby y otros.

Al tratarse de una aplicación web, se decidió utilizar lenguajes destinados al funcionamiento dentro de los navegadores.

Las herramientas seleccionadas para el proyecto de trabajo de grado fueron las siguientes:

Django (Backend).

- Framework robusto que sigue el patrón MVC, facilitando el desarrollo estructurado.
- Incluye herramientas integradas para autenticación, manejo de sesiones y seguridad (por ejemplo, protección contra XSS y CSRF).
- Comunidad activa y documentación extensa que respalda su uso.
- Ideal para construir APIs REST necesarias para integrar el frontend.

Vue.js (Frontend).

De acuerdo a la documentación de Vue, este se define como:

“Un marco de trabajo o *“framework”* de JavaScript para crear interfaces de usuario. Se basa en HTML, CSS y JavaScript estándar y proporciona un modelo de programación declarativo y basado en componentes que le ayuda a desarrollar interfaces de usuario de manera eficiente, ya sean simples o complejas”.

Vue.js se ha consolidado como uno de los frameworks front-end más populares debido a varias características que lo hacen ideal para el desarrollo de aplicaciones web modernas. Su curva de aprendizaje es suave y progresiva, permitiendo incorporar de manera sencilla en proyectos existentes, pero también escalar para crear aplicaciones complejas.

La gran comunidad, documentación extensa y ecosistema de paquetes de Vue aseguran la disponibilidad de soporte profesional para resolver cualquier problema de implementación o mantenimiento, permitiendo aspectos importantes en el proceso del desarrollo:

- Framework progresivo, ideal para crear interfaces interactivas y manejables.
- Fácil de integrar con otros proyectos existentes.
- Ecosistema rico, con herramientas como Vue Router para navegación y Vue para manejo del estado.
- Permite una experiencia de usuario fluida y altamente personalizable.

En ese sentido, una de sus mayores fortalezas es el alto rendimiento, tanto en la representación inicial como en las actualizaciones posteriores de la interfaz. Además, la arquitectura liviana de Vue, enfocada en la capa de vista permite al desarrollador enfocarse en entregar productos de alta calidad de forma rápida y efectiva. La flexibilidad de Vue facilita su integración con otros frameworks y librerías que gestionan los datos y la lógica de negocio, este cuenta con un ecosistema muy completo para enrutado, gestión de estado, pruebas unitarias y más, cubriendo todas las necesidades en el desarrollo web moderno.

Docker (Contenedores).

- Permite empaquetar aplicaciones y dependencias en contenedores, asegurando un entorno consistente.
- Facilita la configuración y despliegue en diferentes entornos (desarrollo, pruebas y producción).
- Elimina problemas de "funciona en mi máquina".
- Mejora la portabilidad y escalabilidad del sistema.

CAPÍTULO III

MARCO METODOLÓGICO

En relación con el marco metodológico a definir, para Evaluar la seguridad de sistemas web a través de pruebas automatizadas de penetración con la finalidad de generar un reporte detallado con los hallazgos encontrados, señala Arias (2012) la metodología de un proyecto es la sección que incluye la descripción del tipo o tipos de investigación utilizados, así como las técnicas y los instrumentos que serán utilizados para llevar a cabo la acción de investigar. En ese sentido se describe cada aspecto que la conforma.

TIPO DE INVESTIGACIÓN

En el presente trabajo el tipo de investigación es documental tecnológico. En cuanto al tipo Documental, de acuerdo con Bernal (2010) consiste en un análisis de la información escrita sobre un determinado tema, con el propósito de establecer relaciones, diferencias, etapas, posturas o estado actual del conocimiento respecto al tema objeto de estudio. Al respecto, esta propuesta es documental porque se revisarán y analizarán las principales fuentes documentales como: documentos escritos (libros, periódicos, informes, revistas, tratados, conferencias escritas y digitales), sobre vulnerabilidades actuales, tipos de pruebas y metodologías a utilizar.

Además, es una investigación tecnológica, de acuerdo con lo que plantea Nieto (2018) es un método de investigación que se basa en la recolección, análisis e interpretación de información existente en diversos documentos, como libros, artículos académicos, informes, tesis, y otros tipos de publicaciones. Este tipo de investigación se caracteriza por el uso de fuentes escritas y puede incluir tanto materiales impresos como digitales.

El presente trabajo aplica la investigación tecnológica debido a que se desea desarrollar una solución a la problemática identificada a través de la automatización e implementación de las pruebas de penetración.

DISEÑO DE LA INVESTIGACIÓN

Hernández Sampieri et al. (1991) explica que la investigación no experimental es aquella que se realiza sin manipular deliberadamente variables, es decir, el investigador no hace variar intencionalmente las variables independientes. En la investigación no experimental se observa fenómenos tal y como se dan en su contexto natural, para después analizarlos. En esta propuesta se aplicará un diseño no experimental debido a que se desea proveer una solución a una situación ya establecida así, mediante el análisis de los procesos actuales, se desarrollará una solución para hacer recomendaciones sobre las vulnerabilidades identificadas.

POBLACIÓN Y MUESTRA

Hernández & Sampieri definen población como el conjunto de todos los casos que concuerdan con una serie de especificaciones. En esta propuesta la población a considerar lo representan la población (casos prácticos) ya analizados por las empresas especialistas en la temática sobre seguridad de la información en el año 2024, ya identificada en el planteamiento del problema.

En ese sentido, con relación a la muestra Palella y Martins (2008) explican que representa un subconjunto de la población, que es accesible y limitado, sobre el cuál se realizarán los estudios y se obtendrán conclusiones aplicables a la población. En esta investigación se realizará un muestreo intencional sobre casos prácticos, en el cual los elementos seleccionados son elegidos por el criterio del investigador, con el objetivo de obtener la información.

TÉCNICAS E INSTRUMENTOS DE RECOLECCIÓN DE DATOS

Utilizando la definición de Arias (2012), se puede ver las técnicas de recolección de datos como las diferentes maneras o formas de obtener información y un instrumento

siendo el medio material donde los datos puedan ser recuperados, procesados, analizados e interpretados posteriormente (p.67-68).

OBSERVACIÓN

Según Bernal (2010) la observación, como técnica de investigación científica, es un proceso que permite conocer, el objeto de estudio para luego describir y analizar situaciones sobre la realidad estudiada. En el caso de la presente investigación fue utilizada una observación simple o no participante, es decir, donde el investigador no se involucra en el área de estudio, sino que observa de forma neutral.

INSTRUMENTO DE RECOLECCIÓN DE DATOS

Arias (2012) explica que un instrumento de recolección de datos es “cualquier recurso, dispositivo o formato (en papel o digital), que se utiliza para obtener, registrar o almacenar información.” (p .68). Por tanto, a continuación, se detallan los instrumentos utilizados durante el desarrollo de este trabajo.

Cuaderno de notas

Un cuaderno de notas según Camacho & Finol (2008) es un documento similar al diario. En el que se registra la información de los hechos, eventos o acontecimientos en propio terreno; ayudarían a analizar la situación al momento de recoger el material. Es decir, un cuaderno de notas es un documento donde se registra toda información observada que sea relevante para el investigador a la hora de tomar decisiones y/o realizar conclusiones. Este instrumento se utilizará para registrar información durante la revisión documental realizada de las vulnerabilidades, las pruebas y las sesiones de retroalimentación.

PROCEDIMIENTO METODOLÓGICO DE LA INVESTIGACIÓN

1. Analizar el estado del arte de la seguridad de los sistemas web.
 - a. Realizar la recopilación y síntesis de la información más actual y relevante con relación a la seguridad de los sistemas web, para contextualizar teóricamente el trabajo de investigación.
 - b. Identificar las nuevas tendencias en cuanto a nuevas amenazas, tecnologías de defensa y metodologías de pruebas de penetración.

2. Caracterizar las vulnerabilidades de los sistemas web, para identificar el nivel de riesgo en las plataformas.
 - a. Revisar los estándares, que establece la guía OWASP Top 10.
 - b. Identificar las vulnerabilidades más comunes, bajo el estándar CWE (Common Weakness Enumeration).
 - c. Categorizar las vulnerabilidades más comunes, identificadas.
 - d. Relacionar una lista priorizada de vulnerabilidades en función de su impacto y frecuencia (ejemplo: XSS, SQL Injection, CSRF).
 - e. Analizar cómo estas vulnerabilidades afectan los principios de la triada CIA (Confidencialidad, Integridad y Disponibilidad).
 - f. Proporcionar ejemplos prácticos de cada vulnerabilidad para ilustrar su funcionamiento.

3. Diseñar un plan de pruebas con base a la información recopilada para evaluar la seguridad de un sistema web.
 - a. Definir el Alcance de las Pruebas: Especificar qué componentes del sistema web serán evaluados, identificar las restricciones y limitaciones (evitar pruebas destructivas en entornos de producción, o pruebas que comprometan la disponibilidad, no se va a realizar un ataque de buffer overflow, por ejemplo, esto podría terminar en corrupción de datos también).

- b. Establecer el flujo de trabajo del pentesting: reconocimiento, análisis de vulnerabilidades, explotación y generación de reportes.
 - c. Definir herramientas y técnicas automatizadas que se utilizarán.
 - d. Planificar la Secuencia de Pruebas, definir criterios de éxito y fallo.
 - e. Elaborar un reporte detallado con los hallazgos encontrados.
4. Desarrollar el sistema de pruebas automatizadas con base a la información recopilada para evaluar la seguridad de un sistema web.
- a. Aplicar la metodología SCRUM.

METODOLOGÍA SCRUM

En cuanto a la metodología de desarrollo señala Schwaber, K. & Sutherland, J. (2020) que Scrum es un marco de trabajo ágil utilizado para el desarrollo de productos y la gestión de proyectos. Se utilizará porque permite a los equipos trabajar en forma colaborativa de manera más eficiente y efectiva, se realizarán entregas regulares y parciales del trabajo final, de manera prioritaria y en función del beneficio que aportan dichas entregas.

Es ampliamente utilizado en la industria del software, pero también se aplica en muchos otros campos debido a su flexibilidad y enfoque en la mejora continua. Por este motivo, es una metodología especialmente indicada para proyectos complejos, con requisitos cambiantes y en los que la innovación y la flexibilidad son protagonistas. La metodología Scrum pasa por diferentes roles, eventos y artefactos que hacen posible que se lleve a cabo con éxito.

Roles:

- a) Scrum Master: Facilita el proceso Scrum, ayuda a eliminar impedimentos y asegura que el equipo siga las prácticas de Scrum.
- b) Product Owner: Representa los intereses del cliente y es responsable de definir y priorizar los elementos del backlog del producto.

c) Equipo de Desarrollo: Un grupo multifuncional que trabaja en la entrega del producto.

Eventos:

a) Sprint: Un ciclo de trabajo fijo, generalmente de una a cuatro semanas, durante el cual se crea un incremento utilizable del producto.

b) Sprint Planning: Reunión para planificar el trabajo que se realizará en el próximo sprint.

c) Daily Scrum: Reunión diaria de 15 minutos para coordinar actividades y ajustar el plan de trabajo.

d) Sprint Review: Revisión al final del sprint para presentar el incremento del producto a los interesados y obtener retroalimentación.

e) Sprint Retrospective: Reunión al final del sprint para reflexionar sobre el proceso y buscar mejoras.

Artefactos:

a) Product Backlog: Una lista priorizada de todo lo que se necesita en el producto.

b) Sprint Backlog: Una lista de tareas seleccionadas del product backlog para completarse en el sprint actual.

c) Incremento: La suma de todos los elementos del product backlog completados durante un sprint y todos los sprints anteriores.

CAPÍTULO IV

RESULTADOS

Este capítulo presenta todo el proceso que se realizó para la Evaluación de la seguridad de sistemas web a través de pruebas automatizadas de penetración con la finalidad de generar un reporte detallado con los hallazgos encontrados, lo que representa el aporte en este proceso de investigación.

Para tal efecto, se analiza el estado del arte de la seguridad de los sistemas web, caracterizando las vulnerabilidades de los sistemas web, para identificar el nivel de riesgo en las plataformas, además se diseña el plan de pruebas automatizadas con base a la información recopilada para evaluar la seguridad del sistema, en función de ello se desarrolla el sistema de pruebas automatizadas con base a la información recopilada para evaluar la seguridad de un sistema web, utilizando la metodología SCRUM, para finalmente aplicar el sistema de pruebas automatizadas, utilizando metodologías de pentesting para generar un reporte detallado con los hallazgos encontrados.

1. Analizar el estado del arte de la seguridad de los sistemas web.

En cuanto al estado del arte de la seguridad de los sistemas web, representa un panorama dinámico y en constante evolución, impulsado por la sofisticación creciente de las amenazas y la complejidad de las arquitecturas web modernas, a continuación, se presentan aspectos relevantes del mismo:

Realizar la recopilación y síntesis de la información más actual y relevante con relación a la seguridad de los sistemas web, para contextualizar teóricamente el trabajo de investigación.

En cuanto a la seguridad de los sistemas web abarca la protección de aplicaciones, servidores y datos frente a accesos no autorizados, ataques cibernéticos y vulnerabilidades. Esto incluye medidas como la seguridad en la nube, la protección de redes y la seguridad de aplicaciones.

Algunos escáneres de vulnerabilidades en las aplicaciones web, no detectan de manera satisfactoria las variantes de las vulnerabilidades más comunes, estas son más abundantes que en cualquier otra área de la informática.

Otro aspecto, está relacionado con las vulnerabilidades asociadas al manejo de sesión, que se encuentran presentes en aplicaciones web de gran variedad de dominios.

Además, se presentan problemas de fugas de información en sitios web y metodologías para detección de vulnerabilidades, mostrando que la mayor vulnerabilidad de las aplicaciones web son las de tipo inyección SQL. Por consiguiente, hay que promover la formación de perfiles profesionales en esta área para minimizar la falta de conocimiento sobre ciberseguridad.

Por otra parte, existe un tipo de vulnerabilidad llamada control de acceso roto, que significa que los atacantes pueden eludir las restricciones de seguridad y acceder a funcionalidades o datos no autorizados. Esto puede tener consecuencias devastadoras, desde la filtración de información sensible hasta la toma de control completa de la aplicación.

Aunque no existe una lista oficial de las "top 10" vulnerabilidades específicas de control de accesos roto para 2021, el informe OWASP Top 10:2021 proporciona una visión general de las categorías de vulnerabilidades web más críticas.

El control de accesos roto (Broken Access Control) se posicionó como la vulnerabilidad número 1 en este informe. Esto significa que una gran variedad de implementaciones incorrectas de control de acceso contribuyó a esta categoría principal. Entre los tipos comunes de vulnerabilidades de Control de Accesos Roto, se mencionan:

Insecure Direct Object References (IDOR): Ocurre cuando una aplicación expone una referencia directa a un objeto interno (como un archivo, directorio o registro de base de datos) a los usuarios. Los atacantes pueden manipular estas referencias para acceder a recursos que no deberían poder ver o modificar. Por ejemplo, cambiar un ID de usuario en la URL para acceder a la información de otro usuario.

Bypass de Controles de Acceso Basados en Roles: Cuando la aplicación no verifica adecuadamente el rol o los permisos de un usuario antes de permitir el acceso a ciertas funcionalidades. Un usuario con un rol normal podría intentar acceder a funciones administrativas simplemente modificando la URL o manipulando los parámetros de la solicitud.

Elevación de Privilegios (Vertical): Un usuario con privilegios inferiores puede obtener acceso a funcionalidades o datos que normalmente están reservados para usuarios con privilegios superiores (como administradores). Esto puede ocurrir por fallas en la lógica de autorización o por la falta de comprobaciones adecuadas.

Omisión de Controles de Acceso (Horizontal): Un usuario puede acceder a recursos o realizar acciones en nombre de otro usuario con el mismo nivel de privilegios. Por ejemplo, acceder a la cuenta bancaria de otro usuario.

Vulnerabilidades en la Gestión de Sesiones: Una gestión de sesiones insegura puede llevar a un control de accesos roto. Esto incluye problemas como IDs de sesión predecibles, falta de invalidación de sesiones después del cierre de sesión, o la capacidad de secuestrar sesiones de otros usuarios.

Falta de Control de Acceso a Funcionalidades: Algunas funcionalidades críticas no están protegidas por ningún mecanismo de control de acceso, lo que permite a cualquier usuario (autenticado o no) acceder a ellas.

Vulnerabilidades en los Metadatos de los Recursos: Los metadatos asociados a los recursos (por ejemplo, permisos de archivo) no se gestionan correctamente, lo que lleva a un acceso no autorizado.

Vulnerabilidades en APIs: Las interfaces de programación de aplicaciones (APIs) a menudo exponen funcionalidades sensibles. Si estas APIs no están protegidas con mecanismos de autenticación y autorización adecuados, pueden ser explotadas para eludir los controles de acceso.

Confianza en Controles del Lado del Cliente: Dependar únicamente de los controles de acceso implementados en el lado del cliente (por ejemplo, ocultar botones o deshabilitar elementos de la interfaz de usuario) es peligroso, ya que estos pueden ser fácilmente explotados por un atacante.

Falta de Auditoría y Registro de Acceso: La ausencia de registros detallados de los intentos de acceso y las decisiones de autorización dificulta la detección y respuesta a los intentos de explotación de control de acceso roto.

Finalmente, se están realizando grandes inversiones tecnológicas en el desarrollo de nuevas herramientas y enfoques para proteger la información en un entorno digital en constante cambio.

Identificar las nuevas tendencias en cuanto a nuevas amenazas, tecnologías de defensa y metodologías de pruebas de penetración.

Entre las tendencias actuales se consideran los siguientes aspectos:

- a. Ciberseguridad y privacidad: La protección de datos personales es una prioridad, especialmente con regulaciones como el GDPR en Europa.
- b. Ataques sofisticados: Los ataques como ransomware, phishing y malware siguen evolucionando, lo que requiere estrategias de defensa más avanzadas.
- c. Inteligencia Artificial (IA): Se utiliza IA para detectar y prevenir amenazas en tiempo real, mejorando la capacidad de respuesta ante incidentes.
- d. En ese sentido es de gran importancia la ciberseguridad, dado que los ciberataques pueden tener consecuencias graves, como pérdidas económicas, daño a la reputación y sanciones legales. Por ejemplo, el costo promedio de una filtración de datos en 2024 fue de 4,45 millones de dólares.

Ahora bien, con relación a las Nuevas amenazas, tecnologías, de defensa y metodologías de pruebas de penetración, se consideran las siguientes:

Nuevas amenazas

1. Ataques basados en inteligencia artificial (IA): Los atacantes están utilizando IA para crear malware más sofisticado, automatizar ataques de phishing y generar deepfakes convincentes para ingeniería social.
2. Ransomware como servicio (RaaS): Este modelo permite a ciberdelincuentes sin experiencia lanzar ataques de ransomware mediante plataformas de pago.

3. Ataques a la cadena de suministro: Los atacantes explotan vulnerabilidades en proveedores de software o hardware para comprometer sistemas más grandes.

Tecnologías de defensa

1. **Arquitecturas de confianza cero (Zero Trust):** Este enfoque asume que ninguna entidad, interna o externa, es confiable por defecto, implementando autenticación continua y políticas de privilegios mínimos.
2. **Automatización y SOAR:** Herramientas de orquestación y automatización de seguridad que detectan y responden a amenazas en tiempo real.
3. **Seguridad en redes 5G:** Protocolos de cifrado avanzados y autenticación fuerte para proteger la creciente conectividad de dispositivos.

Metodologías de pruebas de penetración

1. OWASP Testing Guide: Una metodología ampliamente utilizada para evaluar la seguridad de aplicaciones web.
2. PTES (Penetration Testing Execution Standard): Un estándar que abarca desde la recopilación de inteligencia hasta la post-explotación y generación de informes.
3. MITRE ATT&CK Framework: Un marco que detalla tácticas y técnicas utilizadas por atacantes, ayudando a simular escenarios reales.

2. Caracterizar las vulnerabilidades de los sistemas web, para identificar el nivel de riesgo en las plataformas.

- a. Revisar los estándares, que establece la guía OWASP Top 10.

La guía OWASP Top 10 (2021) es un recurso ampliamente reconocido en el ámbito de la seguridad de aplicaciones web. Proporciona una lista de las diez principales vulnerabilidades de seguridad que afectan a las aplicaciones web, basada en datos recopilados de pruebas de penetración y análisis de riesgos. Su objetivo es ayudar a los desarrolladores y organizaciones a identificar y mitigar estas amenazas para crear aplicaciones más seguras.

El Top 10:2025 está previsto para la primera mitad del 2025, por lo que en este trabajo de investigación se toma como referencia el Top 10:2021.

En ese sentido, las empresas deben adoptar las consideraciones de este documento e iniciar el proceso de garantizar que sus aplicaciones web minimicen estos riesgos. El uso del Top 10 de OWASP es quizás el primer paso más eficaz para cambiar la cultura de desarrollo de software dentro de su organización a una que produce código más seguro.

Los 10 principales riesgos de seguridad de las aplicaciones web, lo conforman tres nuevas categorías, cuatro categorías con cambios en el nombre y el alcance, y cierta consolidación en el Top 10 para 2021.

1. **A01:2021-Broken Access Control** sube desde la quinta posición; El 94% de las aplicaciones se probaron para detectar algún tipo de control de acceso roto. Las 34 Enumeraciones de Debilidades Comunes (CWE) asignadas al Control de Acceso Roto tuvieron más ocurrencias en las aplicaciones que cualquier otra categoría.
2. **A02:2021-Cryptographic Failures** sube una posición a la #2, anteriormente conocida como Exposición de Datos Confidenciales, que era un síntoma general en lugar de una causa raíz. El enfoque renovado aquí se centra en las fallas relacionadas con la criptografía, que a menudo conducen a la exposición de datos confidenciales o al compromiso del sistema.
3. **A03:2021- The injection** se desliza hasta la tercera posición. El 94% de las aplicaciones se probaron para algún tipo de inyección, y los 33 CWE mapeados en esta categoría tienen la segunda mayor cantidad de ocurrencias en aplicaciones. El Cross-site Scripting ahora forma parte de esta categoría en esta edición.
4. **A04:2021- Insecure design** es una nueva categoría para 2021, con un enfoque en los riesgos relacionados con fallas de diseño. Si realmente queremos "movernos a la izquierda" como industria, se requiere un mayor uso del modelado de amenazas, patrones y principios de diseño seguros y arquitecturas de referencia.

5. **A05:2021-Security Misconfiguration** sube desde el #6 de la edición anterior; El 90% de las aplicaciones se probaron para detectar algún tipo de error de configuración. Con más cambios en software altamente configurable, no es sorprendente ver que esta categoría asciende. La antigua categoría de Entidades Externas XML (XXE) ahora forma parte de esta categoría.
6. **A06:2021-Vulnerable and Obdated Components** se tituló anteriormente Using Components with Known Vulnerabilities y es el #2 en la encuesta de la comunidad Top 10, pero también tenía suficientes datos para llegar al Top 10 a través del análisis de datos. Esta categoría sube desde el #9 en 2017 y es un problema conocido que nos cuesta probar y evaluar el riesgo. Es la única categoría que no tiene ninguna vulnerabilidad y exposición común (CVE) asignada a los CWE incluidos, por lo que se tienen en cuenta en sus puntuaciones un exploit y un peso de impacto predeterminados de 5,0.
7. **A07:2021-Identification and Authentication Failures** era anteriormente Broken Authentication y se está deslizando hacia abajo desde la segunda posición, y ahora incluye CWE que están más relacionados con fallas de identificación. Esta categoría sigue siendo una parte integral del Top 10, pero la mayor disponibilidad de marcos estandarizados parece estar ayudando.
8. **A08:2021- Software and data integrity failures** es una nueva categoría para 2021, que se centra en hacer suposiciones relacionadas con actualizaciones de software, datos críticos y canalizaciones de CI/CD sin verificar la integridad. Uno de los impactos más ponderados de los datos de Common Vulnerability and Exposures/Common Vulnerability Scoring System (CVE/CVSS) asignados a las 10 CWE de esta categoría. La deserialización insegura de 2017 ahora forma parte de esta categoría más grande.
9. **A09:2021-Security Logging and Monitoring Failures** era anteriormente Insuficiente Logging & Monitoring y se agrega de la encuesta de la industria (#3), subiendo desde el #10 anteriormente. Esta categoría se amplía para incluir más tipos de errores, es difícil de probar y no está bien representada en los datos

de CVE/CVSS. Sin embargo, los fallos de esta categoría pueden afectar directamente a la visibilidad, las alertas de incidentes y el análisis forense.

10. **A10:2021-Server-Side Request Forgery** se agrega de la encuesta de la comunidad Top 10 (#1). Los datos muestran una tasa de incidencia relativamente baja con una cobertura de pruebas superior a la media, junto con puntuaciones superiores a la media para el potencial de explotación e impacto. Esta categoría representa el escenario en el que los miembros de la comunidad de seguridad nos dicen que esto es importante, aunque no se ilustra en los datos en este momento.

b. Identificar las vulnerabilidades más comunes, bajo el estándar CWE (Common Weakness Enumeration).

El estándar CWE (Common Weakness Enumeration) identifica y clasifica debilidades comunes en software y hardware que pueden convertirse en vulnerabilidades explotables.

En ese sentido de acuerdo al informe del CWE Top 25 (2024),

1. Neutralización incorrecta de la entrada durante la generación de páginas web («Cross-site scripting») – Id: 79: El producto no neutraliza o neutraliza incorrectamente la entrada controlable por el usuario antes de que se coloque en la salida que se utiliza como página web que se sirve a otros usuarios.
2. Escritura fuera de límites – Id:787: El producto escribe datos más allá del final, o antes del principio, del buffer previsto. A menudo se usa para describir las consecuencias de escribir en la memoria fuera de los límites de un búfer, o en la memoria que de otro modo no es válida.
3. Neutralización incorrecta de elementos especiales utilizados en un comando SQL ('Inyección SQL') – Id:89: El producto construye la totalidad o parte de un comando SQL utilizando entradas influenciadas externamente de un componente ascendente, pero no neutraliza o neutraliza incorrectamente elementos especiales que podrían modificar el comando SQL previsto cuando

se envía a un componente descendente. Sin una eliminación o comillas suficientes de la sintaxis SQL en las entradas controlables por el usuario, la consulta SQL generada puede hacer que esas entradas se interpreten como SQL en lugar de datos de usuario ordinarios.

4. Falsificación de Solicitud entre Sitios (CSRF) – Id:352: La aplicación web no verifica, o no puede, verificar suficientemente si una solicitud fue proporcionada intencionalmente por el usuario que envió la solicitud, que podría haberse originado en un actor no autorizado.
5. Limitación incorrecta de un nombre de ruta a un directorio restringido ('recorrido de ruta') – Id:22: El producto utiliza una entrada externa para construir un nombre de ruta de acceso que está destinado a identificar un archivo o directorio que se encuentra debajo de un directorio principal restringido, pero el producto no neutraliza correctamente los elementos especiales dentro del nombre de ruta que pueden hacer que el nombre de ruta se resuelva en una ubicación que está fuera del directorio restringido.
6. Lectura fuera de límites – Id:125: El producto lee los datos más allá del final, o antes del principio, del buffer previsto.
7. Neutralización incorrecta de elementos especiales utilizados en un comando del sistema operativo ('Inyección de comandos del sistema operativo') – Id:78: El producto construye la totalidad o parte de un comando del sistema operativo utilizando la entrada influenciada externamente de un componente ascendente, pero no neutraliza o neutraliza incorrectamente los elementos especiales que podrían modificar el comando del sistema operativo previsto cuando se envía a un componente descendente.
8. Usar después de gratis – Id:416: El producto utiliza o hace referencia a la memoria después de que se haya liberado. En algún momento posterior, la

memoria puede asignarse de nuevo y guardarse en otro puntero, mientras que el puntero original hace referencia a una ubicación en algún lugar dentro de la nueva asignación. Las operaciones que utilizan el puntero original ya no son válidas porque la memoria "pertenece" al código que opera en el nuevo puntero.

9. Falta autorización – Id:862: El producto no realiza una comprobación de autorización cuando un actor intenta acceder a un recurso o realizar una acción.
10. Carga sin restricciones de archivos con tipo peligroso – Id:434: El producto permite la carga o transferencia de tipos de archivos peligrosos que se procesan automáticamente dentro de su entorno.
11. Control inadecuado de la generación de código («inyección de código») – Id:94: El producto construye la totalidad o parte de un segmento de código utilizando la entrada influenciada externamente de un componente ascendente, pero no neutraliza o neutraliza incorrectamente elementos especiales que podrían modificar la sintaxis o el comportamiento del segmento de código previsto.
12. Validación de entrada incorrecta – Id:20: El producto recibe entradas o datos, pero no valida que la entrada tiene el atributo propiedades que son necesarias para procesar los datos de forma segura y correctamente.
13. Neutralización Inadecuada de Elementos Especiales Utilizados en un Comando ('Inyección de Comando') – Id:77: El producto construye la totalidad o parte de un comando utilizando la entrada influenciada externamente de un componente ascendente, pero no neutraliza o neutraliza incorrectamente los elementos especiales que podrían modificar el comando previsto cuando se envía a un componente descendente.

14. Autenticación incorrecta – Id:287: Cuando un actor afirma tener una identidad determinada, el producto no prueba o prueba de manera insuficiente que la afirmación es correcta.
15. Gestión inadecuada de privilegios - Id:269: El producto no asigna, modifica, rastrea ni comprueba correctamente los privilegios de un actor, lo que crea una esfera de control no deseada para ese actor.
16. Deserialización de datos que no son de confianza - Id:502: El producto deserializa los datos que no son de confianza sin garantizar suficientemente que los datos resultantes sean válidos.
17. Exposición de información confidencial a un actor no autorizado - Id:200: El producto expone información confidencial a un actor que no está explícitamente autorizado a tener acceso a esa información.
18. Autorización incorrecta - Id:863: El producto realiza una comprobación de autorización cuando un actor intenta acceder a un recurso o realizar una acción, pero no realiza correctamente la comprobación.
19. Falsificación de solicitudes del lado del servidor (SSRF) - Id:918: El servidor web recibe una URL o una solicitud similar de un componente ascendente y recupera el contenido de esta URL, pero no garantiza suficientemente que la solicitud se envíe al destino esperado.
20. Restricción incorrecta de operaciones dentro de los límites de un buffer de memoria - Id:119: El producto realiza operaciones en un buffer de memoria, pero lee o escribe en una ubicación de memoria fuera del límite previsto del buffer. Esto puede dar lugar a operaciones de lectura o escritura en ubicaciones de memoria inesperadas que podrían estar vinculadas a otras variables, estructuras de datos o datos de programas internos.

21. Desreferencia de puntero NULL - Id:476: El producto desreferencia un puntero que espera que sea válido, pero es NULL.
22. Uso de credenciales codificadas de forma rígida - Id:798: El producto contiene credenciales codificadas de forma rígida, como una contraseña o una clave criptográfica.
23. Desbordamiento de enteros o envolvente - Id:190: El producto realiza un cálculo que puede producir un desbordamiento de enteros o wraparound cuando la lógica supone que el valor resultante siempre será mayor que el valor original. Esto ocurre cuando un valor entero es incrementado a un valor que es demasiado grande para almacenarlo en el archivo de representación asociado. Cuando esto ocurre, el valor puede convertirse en un número muy pequeño o negativo.
24. Consumo incontrolado de recursos - Id:400: El producto no controla adecuadamente la asignación y el mantenimiento de un recurso limitado.
25. Falta autenticación para la función crítica - Id:306: El producto no realiza ninguna autenticación para la funcionalidad que requiere una identidad de usuario demostrable o que consume una cantidad significativa de recursos.

El CWE Top 25 también sirve como guía estratégica para las organizaciones que buscan tomar decisiones informadas en inversiones en software, seguridad y gestión de riesgos, ayuda a informar sobre:

- Reducción de vulnerabilidades: la información sobre las causas raíz comunes genera comentarios valiosos sobre el SDLC y la planificación arquitectónica de los proveedores, lo que ayuda a eliminar clases enteras de defectos (por ejemplo, seguridad de la memoria, inyección)

- Ahorro de costos: menos vulnerabilidades en el desarrollo de productos significan menos problemas para administrar después de la implementación, lo que en última instancia ahorra dinero y recursos.
- Análisis de tendencias: la información sobre las tendencias de los datos permite a las organizaciones centrarse mejor en los esfuerzos de seguridad.
- Información sobre la explotabilidad: ciertas debilidades, como la inyección de comandos, atraen la atención de los adversarios, lo que permite la priorización de riesgos.
- Confianza del cliente: la transparencia en la forma en que las organizaciones abordan estas debilidades muestra compromiso con la seguridad del producto.

c. Categorizar las vulnerabilidades más comunes, identificadas.

Basándose en estándares como OWASP y CWE la categorización de las vulnerabilidades más comunes identificadas, son las siguientes:

1. Vulnerabilidades de validación de entrada

- Inyección SQL (SQL Injection): Manipulación de consultas a la base de datos.
- Cross-Site Scripting (XSS): Inserción de scripts maliciosos en páginas web.
- Validación insuficiente de entrada (CWE-20): Entradas no verificadas que comprometen el sistema.

2. Problemas de control de acceso

- Control de acceso roto (Broken Access Control): Permite a usuarios no autorizados realizar acciones restringidas.
- Mala configuración de permisos: Acceso innecesario a archivos o sistemas.

3. Vulnerabilidades de gestión de autenticación

- Gestión deficiente de contraseñas: contraseñas débiles o sin protección adecuada.
- Session Fixation: Secuestro de sesiones activas.

- Autenticación rota (CWE-287): Métodos de autenticación mal implementados.

4. Fallas en la criptografía

- Uso de algoritmos inseguros: Algoritmos obsoletos como MD5 o SHA-1.
- Exposición de datos sensibles: Transmisión o almacenamiento sin cifrado adecuado.

5. Configuraciones inseguras

- Mala configuración del servidor: Configuraciones predeterminadas o innecesarias.
- Exposición de información sensible en errores: Mensajes de error detallados.

6. Vulnerabilidades en la lógica de negocio

- Diseño inseguro (Insecure Design): Falta de medidas de seguridad en la arquitectura.
- Manipulación de parámetros: Cambios no controlados en parámetros de la URL o formularios.

7. Exposición accidental o intencionada

- Filtración de información sensible: Por ejemplo, claves API en repositorios públicos.
- Error humano o configuraciones incorrectas.

d. Relacionar una lista priorizada de vulnerabilidades en función de su impacto y frecuencia.

En este caso podemos mencionar las vulnerabilidades cubiertas por la herramienta en desarrollo en las pruebas automatizadas cubriendo en su mayoría las vulnerabilidades del OWASP Top 10 - SQL Injection, Cross-Site Scripting (XSS), Weak Hashing Algorithm, Insecure Direct Object References, Security Misconfiguration, Known Vulnerable Components (Retire.js), Insufficient

Logging and Monitoring, Server-Side Request Forgery (SSRF), Insecure Deserialization, Authentication Bypass.

e. Analizar cómo estas vulnerabilidades afectan los principios de la tríada CIA (Confidencialidad, Integridad y Disponibilidad).

SQL Injection

- Confidencialidad: Un atacante puede extraer datos sensibles de la base de datos (credenciales, información de usuarios, registros financieros).
- Integridad: Permite la manipulación de datos, como modificar precios de productos o cambiar roles de usuario.
- Disponibilidad: Puede usarse para eliminar bases de datos enteras o bloquear acceso mediante consultas pesadas (Denial of Service).

Cross-Site Scripting (XSS)

- Confidencialidad: Un atacante puede robar cookies de sesión, lo que compromete cuentas de usuarios.
- Integridad: Puede alterar la apariencia o contenido de una página inyectando scripts maliciosos.
- Disponibilidad: En ataques avanzados, XSS puede usarse para propagar malware o realizar defacements que afecten la disponibilidad de un sitio.

Weak Hashing Algorithm

- Confidencialidad: Un atacante que obtiene contraseñas cifradas con algoritmos débiles (MD5, SHA-1) puede descifrarlas rápidamente.
- Integridad: Alguien con acceso a los hashes podría modificarlos, permitiendo autenticaciones fraudulentas.
- Disponibilidad: No afecta directamente la disponibilidad, pero una brecha de contraseñas puede derivar en un ataque de mayor impacto.

Insecure Direct Object References

- Confidencialidad: Permite a atacantes acceder a datos de otros usuarios (por ejemplo, viendo facturas o documentos ajenos).
- Integridad: Un atacante puede modificar información de otra persona (cambiar direcciones, alterar permisos).
- Disponibilidad: Si se abusa masivamente, podría causar corrupción de datos o afectar la funcionalidad del sistema.

Security Misconfiguration

- Confidencialidad: Configuraciones inseguras pueden exponer datos sensibles (ejemplo: directorios abiertos en servidores web).
- Integridad: Un atacante puede explotar configuraciones débiles para manipular o modificar datos.
- Disponibilidad: Puede dejar un sistema vulnerable a ataques como DoS o escalación de privilegios.

Known Vulnerable Components (Retire.js)

- Confidencialidad: Componentes desactualizados pueden permitir la explotación de vulnerabilidades conocidas para robar información.
- Integridad: Un atacante podría inyectar código malicioso en componentes vulnerables, comprometiendo la integridad del sistema.
- Disponibilidad: Un exploit en un componente crítico puede provocar fallos en el servicio o ejecución de código remoto.

Insufficient Logging and Monitoring

- Confidencialidad: Sin registros adecuados, no se detectan accesos no autorizados a datos sensibles.
- Integridad: Sin monitoreo, modificaciones no autorizadas en los datos pueden pasar desapercibidas.

- Disponibilidad: Si un atacante lanza un ataque DoS o explota una vulnerabilidad sin ser detectado, puede afectar la disponibilidad del sistema.

Server-Side Request Forgery (SSRF)

- Confidencialidad: Puede usarse para filtrar información interna del servidor o acceder a servicios internos protegidos.
- Integridad: Un atacante puede modificar peticiones enviadas al servidor, alterando datos sensibles.
- Disponibilidad: En algunos casos, se puede usar para lanzar ataques DoS o consumir recursos de la infraestructura objetivo.

Insecure Deserialization

- Confidencialidad: Puede permitir la ejecución remota de código y exponer información sensible del sistema.
- Integridad: Permite manipular datos serializados para alterar información crítica (como roles de usuario o saldo en cuentas bancarias).
- Disponibilidad: Puede explotarse para ejecutar código malicioso que causen una denegación de servicio.

Authentication Bypass

- Confidencialidad: Permite a atacantes acceder a cuentas y datos sin autenticación legítima.
- Integridad: Un atacante con acceso no autorizado podría modificar datos dentro del sistema.
- Disponibilidad: Si se usa para comprometer una cuenta con permisos administrativos, se podría eliminar o deshabilitar servicios enteros.

- a. **Proporcionar ejemplos prácticos de cada vulnerabilidad para ilustrar su funcionamiento.**

SQL Injection

Para la explotación de un campo de inicio de sesión, un ejemplo de código vulnerable sería el siguiente (PHP + MySQL):

```
$username = $_POST['username'];  
$password = $_POST['password'];  
$query = "SELECT * FROM users WHERE username='$username' AND password='$password'";  
$result = mysqli_query($conn, $query);
```

Si el atacante ingresa en el campo de usuario ' OR '1'='1, la consulta se convierte en:

```
SELECT * FROM users WHERE username='' OR '1'='1' AND password='';
```

Esto devuelve todos los usuarios, permitiendo autenticación sin credenciales válidas.

Cross-Site Scripting (XSS)

Para la inyección de un script en un formulario de comentarios, un ejemplo de código vulnerable sería el siguiente (HTML + JavaScript):

```
<form action="submit_comment.php" method="post">  
  <input type="text" name="comment">  
  <input type="submit" value="Enviar">  
</form>
```

Si los datos no están sanitizados, y un atacante escribe el siguiente código como comentario, el script se ejecutará en el navegador de otros usuarios al cargar la página:

```
<script>alert('XSS Attack!');</script>
```

Weak Hashing Algorithm

Un ejemplo práctico de código vulnerable sería el uso de MD5 para almacenar contraseñas:

```
$hashed_password = md5($password);
```

Este algoritmo es fácil de romper con sólo una búsqueda de descryptación en el navegador, también hay herramientas accesibles como hashcat, o bases de datos de hashes precalculados.

Insecure Direct Object References

Para esta vulnerabilidad, un ejemplo práctico sería poder acceder a registros ajenos en una URL, es decir, si un usuario autenticado accede a su factura en el siguiente URL:

```
https://empresa.com/factura?id=123
```

Un atacante podría cambiar el número de el id en la URL:

```
https://empresa.com/factura?id=122
```

Si no hay controles de acceso, el atacante podría ver la factura de otro usuario.

Security Misconfiguration

Una mala configuración de seguridad puede ser el directorio `.git` expuesto. Imaginemos que un desarrollador olvida restringir el acceso al directorio `.git` en un servidor web, el atacante puede acceder a:

```
https://empresa.com/.git/config
```

desde ahí, podría descargar el repositorio completo y analizarlo en busca de credenciales o vulnerabilidades.

Known Vulnerable Components (Retire.js)

Un perfecto ejemplo sería el de una web usando una versión antigua de jQuery (1.7.0) con vulnerabilidades conocidas y divulgadas en la página oficial de CVE:

```
<script src="https://ajax.googleapis.com/ajax/libs/jquery/1.7.0/jquery.min.js"></script>
```

En este caso, el atacante puede usar un ataque CVE conocido contra esa versión para realizar XSS o inyección de código.

Insufficient Logging and Monitoring

En la práctica, esta vulnerabilidad podría ser explotada cuando un atacante prueba credenciales en un sistema varias veces, pero como no hay monitoreo adecuado, no se generan alertas, como consecuencia, encuentra una credencial válida y compromete la cuenta sin que el equipo de seguridad se entere.

```
admin / admin123  
admin / qwerty  
admin / password
```

Server-Side Request Forgery (SSRF)

Por ejemplo, la explotación de una API interna. Un escenario para esta vulnerabilidad sería un servicio permite obtener datos de una URL ingresada por el usuario:

```
$url = $_GET['url'];  
$response = file_get_contents($url);
```

Para la explotación, un atacante usa la URL para hacer solicitudes internas no accesibles desde el exterior:

```
https://empresa.com/api?url=http://localhost/admin
```

Si la API interna no tiene autenticación, el atacante puede extraer información sensible.

Insecure Deserialization

Para la manipulación del objeto serializado, un ejemplo de código vulnerable sería el siguiente (PHP):

```
class User {  
    public $role = "user";  
}  
$user = new User();  
echo serialize($user);
```

Si un atacante intercepta la cookie serializada:

```
O:4:"User":1:{s:4:"role";s:4:"user";}
```

Puede modificarla para elevar privilegios, en este caso le daría privilegios de administrador:

```
O:4:"User":1:{s:4:"role";s:5:"admin";}
```

Authentication Bypass

Una vulnerabilidad común es el uso de las credenciales por defecto, por ejemplo, instalar un servidor MYSQL y no cambiar las credenciales:

```
Usuario: root  
Contraseña: root
```

El atacante simplemente intenta introducir las credenciales por defecto y obtiene acceso al sistema.

3. Diseñar el plan de pruebas automatizadas con base a la información recopilada para evaluar la seguridad de un sistema web.

El diseño del plan de pruebas automatizadas de seguridad es fundamental para evaluar la robustez de un sistema web frente a posibles vulnerabilidades. Con base en la información recopilada en los capítulos anteriores, se establecerá un conjunto de estrategias y metodologías que permitan la ejecución sistemática de pruebas de penetración automatizadas. Este proceso está alineado con los estándares internacionales de seguridad OWASP y CWE descritos anteriormente, con el objetivo de garantizar la integridad, confidencialidad y disponibilidad del sistema evaluado.

a. Definir los fundamentos del plan de pruebas automatizadas.

El plan de pruebas se diseñará tomando en cuenta:

1) Metodología de Pruebas de Seguridad: Se adoptará la metodología OWASP Web Security Testing Guide (<https://owasp.org/www-project-web-security-testing-guide/>), la cual se adapta de manera ideal a la solución

propuesta, además de establecer las mejores prácticas en evaluación de seguridad de aplicaciones web.

2) Tipos de Pruebas Automatizadas: Se definen los tipos de pruebas a realizar, estos incluyen:

- SQL Injection
- Cross-Site Scripting (XSS)
- Weak Hashing Algorithm
- Insecure Direct Object References
- Security Misconfiguration
- Known Vulnerable Components (Retire.js)
- Insufficient Logging and Monitoring
- Server-Side Request Forgery (SSRF)
- Insecure Deserialization
- Authentication Bypass

b. Establecer el flujo de trabajo y la estrategia del plan de pruebas automatizadas.

Los objetivos principales del plan de pruebas automatizadas son:

- 1) Identificar vulnerabilidades en el sistema web mediante la ejecución de pruebas automatizadas.
- 2) Evaluar la efectividad de los controles de seguridad implementados en la aplicación.
- 3) Proporcionar un informe detallado con hallazgos de las vulnerabilidades detectadas clasificadas según su severidad.

En este sentido, el diseño del plan de pruebas seguirá un enfoque estructurado que consta de las siguientes fases:

1) Planificación

En esta fase, se definen los alcances y limitaciones de las pruebas. Dado que se trata de pruebas de caja negra, el análisis se realizará únicamente sobre un dominio específico sin acceso a información interna del sistema, código fuente

o credenciales de autenticación. Este enfoque simula el comportamiento de un atacante externo sin conocimientos previos del sistema, lo que permite evaluar su seguridad desde una perspectiva realista.

Asimismo, se establecerán los permisos necesarios para la ejecución de las pruebas, garantizando que el escaneo automatizado se realice de manera controlada y sin afectar la operatividad del sistema objetivo. Se seleccionarán herramientas adecuadas para la automatización de las pruebas y se definirán las métricas de evaluación para la interpretación de los resultados.

2) Ejecución de Pruebas Automatizadas

Las pruebas se ejecutarán siguiendo el siguiente flujo de trabajo:

- Reconocimiento y mapeo de la URL: Esta fase inicial permite mapear la estructura del sitio web, identificando todas las URL accesibles y los puntos de entrada que podrían ser vulnerables. Se basa en la exploración automática de enlaces y rutas dentro del dominio evaluado.
- Evaluación de vulnerabilidades: Una vez obtenida la estructura del sitio, se procede con el escaneo activo de vulnerabilidades. Se analiza cada punto de entrada identificado en busca de fallos de seguridad, probando diferentes técnicas de ataque automatizadas.
- Generación de alertas: Al finalizar el escaneo activo, se generan alertas con los hallazgos detectados, permitiendo la evaluación de riesgos en función de la severidad de cada vulnerabilidad.
- Visualización de resultados: Finalmente, los hallazgos obtenidos se muestran para validar su impacto, documentando las vulnerabilidades encontradas.

3) Reporte de Resultados

Tras la ejecución de las pruebas, se generará un informe detallado con:

- Resumen de las pruebas realizadas.
- Vulnerabilidades detectadas clasificadas según su severidad.
- Evidencia de cada hallazgo.

- c. Seleccionar las herramientas y técnicas automatizadas que se utilizarán.

Para la ejecución de las pruebas automatizadas, se emplearán herramientas especializadas, particularmente Zed Attack Proxy (ZAP - <https://www.zaproxy.org/>), con las siguientes funcionalidades:

- 1) **Spider Scan API:** Para el análisis de la estructura y mapeo del sitio web.
- 2) **Active Scan API:** Para identificar vulnerabilidades específicas, estas son:
 - 40018 - SQL Injection
 - 40012 - Cross-Site Scripting (XSS)
 - 40026 - Weak Hashing Algorithm
 - 40024 - Insecure Direct Object References
 - 40016 - Security Misconfiguration
 - 40027 - Known Vulnerable Components (Retire.js)
 - 40014 - Insufficient Logging and Monitoring
 - 40020 - Server-Side Request Forgery (SSRF)
 - 40022 - Insecure Deserialization
 - 40017 - Authentication Bypass
- 3) **Core Alerts API:** Para obtener las alertas generadas a partir del Active Scan realizado.

4. Desarrollar el sistema de pruebas automatizadas con base a la información recopilada para evaluar la seguridad de un sistema web, utilizando la metodología SCRUM.

El desarrollo del sistema de pruebas automatizadas sigue un enfoque ágil basado en SCRUM. Esta metodología permite gestionar el proyecto de forma iterativa e incremental, facilitando la adaptación a cambios y asegurando la entrega de una solución funcional en cada fase. La elección de los siguientes sprints responde a la necesidad de estructurar el desarrollo en etapas bien definidas:

1. Requerimientos Iniciales
2. Análisis
3. Diseño
4. Desarrollo
5. Integración y Pruebas
6. Optimización y Finalización

Sprint 1: Requerimientos Iniciales

El primer sprint del desarrollo del sistema de pruebas automatizadas se centra en la definición detallada de los requisitos funcionales y no funcionales, los actores involucrados y las restricciones que influyen en la implementación del sistema. La identificación temprana de estos aspectos permite establecer una base sólida para el desarrollo, garantizando que el producto final cumpla con las expectativas y necesidades del usuario.

Este sprint también aborda la elaboración del cronograma de desarrollo basado en la metodología SCRUM, asegurando una planificación estructurada y eficiente del proyecto.

Backlog del Sprint

- a. Definición y documentación de requisitos funcionales y no funcionales.
- b. Análisis de restricciones técnicas, legales y operativas.
- c. Identificación de requisitos adicionales como seguridad y escalabilidad.

- d. Elaboración del cronograma con estimaciones de tiempo.
- e. Identificación de riesgos y mitigaciones del proyecto.

Resultados

a. Definición y documentación de requisitos funcionales y no funcionales.

Alineándose con la información recopilada anteriormente, y el diseño del plan de pruebas realizado, se propone a continuación los requisitos funcionales y no funcionales para el sistema de pruebas automatizadas.

Requisitos Funcionales: Definen las características y funcionalidades esenciales que debe tener el sistema de pruebas automatizadas para cumplir con su propósito.

1) Evaluación de Seguridad

- El sistema debe realizar análisis automatizados de vulnerabilidades en sistemas web.
- Debe identificar las siguientes vulnerabilidades, algunas de las cuales están contempladas en el OWASP Top 10:
 - SQL Injection
 - Cross-Site Scripting (XSS)
 - Weak Hashing Algorithm
 - Insecure Direct Object References
 - Security Misconfiguration
 - Known Vulnerable Components
 - Insufficient Logging and Monitoring
 - Server-Side Request Forgery (SSRF)
 - Insecure Deserialization
 - Authentication Bypass
- Debe categorizar las vulnerabilidades detectadas según su nivel de criticidad: Bajo, Medio, Alto e Informativo.

2) Gestión de Resultados

- Debe generar un resumen general de las vulnerabilidades detectadas con su caracterización.
- Debe proporcionar una descripción técnica de cada vulnerabilidad y su nivel de criticidad.
- Debe permitir la visualización del detalle del resultado del escaneo.
- Debe permitir la descarga de los reportes detallados en formatos estándar (PDF).

3) Compatibilidad Técnica

- El sistema debe integrarse con Zed Attack Proxy (ZAP) mediante su API para ejecutar los escaneos de manera automatizada.

4) Interfaz de Usuario

- Debe permitir la configuración de objetivo (URL dominio) para las pruebas.
- Debe incluir una página de inicio informativa.
- Debe proporcionar las vistas de detalle y reportes de los escaneos realizados.
- Debe integrar las páginas de Acerca de, conteniendo la versión y desarrolladores del sistema.
- Debe incluir un disclaimer, donde se especifique la limitación de responsabilidad de los desarrolladores, y, además, la aceptación de términos por parte del usuario.

5) Automatización y Flujo de Pruebas

- El sistema debe ejecutar las pruebas de manera automática después de ser configurado.

- La automatización se realizará en las siguientes fases:
- Reconocimiento y mapeo de la estructura del sitio web.
- Ejecución del escaneo activo para detectar vulnerabilidades.
- Generación de alertas y reporte de resultados.

Requisitos No Funcionales: Establecen cómo debe operar el sistema en términos de rendimiento, seguridad y experiencia del usuario.

1) Rendimiento y Escalabilidad

- Debe manejar múltiples pruebas en paralelo para aumentar la eficiencia y el rendimiento.
- Debe permitir la escalabilidad para soportar sitios web de mayor tamaño y tráfico elevado.

2) Compatibilidad Técnica

- Debe ser compatible con ZAP API para la ejecución de escaneos.
- Debe funcionar en navegadores modernos como Chrome, Firefox y Edge.
- Debe ser multiplataforma (compatible con distintos sistemas operativos y entornos).

3) Usabilidad y Experiencia de Usuario

- La interfaz debe ser intuitiva, permitiendo que usuarios con conocimientos básicos puedan ejecutar pruebas.
- Debe ofrecer una visualización clara de los resultados y filtros por dominio.

b. Análisis de restricciones técnicas, legales y operativas.

1) Restricciones técnicas

- Integración con Zed Attack Proxy (ZAP): El sistema debe comunicarse correctamente con ZAP a través de su API.
- Infraestructura: El sistema debe funcionar en un entorno local.

2) Restricciones legales

- Autorización para pruebas: Las pruebas sólo pueden realizarse en sistemas autorizados con permiso explícito del propietario.
- Manejo de datos sensibles: No se debe almacenar información confidencial sin cifrado ni mecanismos de protección adecuados.
- Cumplimiento con normativas: Se debe respetar normativas como el GDPR y ISO 27001, especialmente en lo que respecta a la recopilación y almacenamiento de datos de las pruebas.

3) Restricciones operativas

- Alcance de las pruebas: Solo se ejecutarán pruebas automatizadas sin intervención manual.
- Disponibilidad: El sistema debe estar disponible sin interrumpir las operaciones normales del sitio web evaluado.

c. **Identificación de requisitos adicionales de escalabilidad.**

- 1) Modularidad: El sistema debe permitir la adición de nuevas pruebas de seguridad en el futuro.
- 2) Almacenamiento eficiente: Implementar bases de datos optimizadas para la gestión de reportes.

d. **Elaboración del cronograma con estimaciones de tiempo.**

Se propone el siguiente cronograma con duración de 15 semanas, desde la aprobación de la propuesta de trabajo de investigación, hasta la entrega de esta.

Tabla 1: Cronograma

15 semanas: 6 de enero de 2025 – 20 de abril de 2025

Semanas	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Requerimientos Iniciales															
Análisis															
Diseño															
Desarrollo															
Integración y Pruebas															
Optimización y Finalización															

e. Identificación de riesgos y mitigaciones del proyecto.

Se procede a identificar los riesgos del proyecto, y para cada uno, su nivel de impacto y mitigación propuesta.

Tabla 2: Riesgos y Mitigaciones del proyecto

Riesgo	Impacto	Mitigación
Incompatibilidad con Zed Attack Proxy (ZAP)	Alto	Realizar pruebas de integración tempranas.
Falta de permisos para pruebas en sitios web	Alto	Garantizar acuerdos previos con los propietarios del sitio. Realizar pruebas en sitios webs diseñados para la evaluación de vulnerabilidades.
Retrasos en el desarrollo	Medio	Definir prioridades y realizar reuniones de seguimiento constantes.
Problemas de rendimiento al ejecutar múltiples pruebas en paralelo	Medio	Optimizar la gestión de recursos, utilizar procesamiento asíncrono y realizar pruebas de carga para garantizar estabilidad.

Sprint 2: Análisis

El segundo sprint se centra en el análisis de los requerimientos iniciales definidos en el sprint de requerimientos iniciales, y del plan de pruebas diseñado en el objetivo anterior, considerando la arquitectura cliente-servidor del sistema de pruebas automatizadas. El objetivo principal es transformar esta información en una comprensión clara y detallada de la estructura del sistema, los flujos de datos, las interacciones específicas, la integración con ZAP y los posibles desafíos técnicos. Este sprint garantiza que el desarrollo se base en una estructura sólida, optimizando el rendimiento y la escalabilidad del sistema.

Backlog del Sprint

- a. Definir la arquitectura del sistema.
- b. Análisis de la integración de ZAP y su integración en la arquitectura.
- c. Seleccionar las tecnologías adecuadas para el desarrollo del frontend y backend.
- d. Configuración del entorno de desarrollo y gestión del control de versiones con Git.
- e. Modelado de la arquitectura estática y dinámica del sistema mediante diagramas.
- f. Modelo entidad-relación (ER) de la base de datos.
- g. Identificación y análisis de posibles desafíos técnicos.

Resultados

a. Definir la arquitectura del sistema

Basados en un análisis tanto de los requisitos funcionales y no funcionales que debe incluir el sistema, así como del diseño del plan de pruebas automatizadas realizado en el objetivo anterior, se ha definido una arquitectura cliente-servidor para el sistema de pruebas automatizadas,

donde la interacción entre los componentes se gestiona principalmente a través de una API REST para las comunicaciones transaccionales y WebSockets para la comunicación durante los escaneos.

La elección de una arquitectura cliente-servidor permite una clara separación de responsabilidades entre la interfaz de usuario (frontend) y la lógica de negocio y procesamiento (backend). La API REST facilita la comunicación síncrona para solicitudes y respuestas, mientras que WebSockets habilitan una comunicación bidireccional y de baja latencia, crucial para proporcionar al usuario información actualizada sobre el progreso y los hallazgos de los escaneos sin necesidad de realizar peticiones repetidas.

Esta arquitectura promueve la modularidad, la escalabilidad y la mantenibilidad del sistema. La solución se estructura en tres capas principales:

- 1) Capa de Presentación (Frontend):** Se comunicará con el backend a través de una API REST, enviando configuraciones de escaneo y recuperando resultados en tiempo real mediante WebSockets.
- 2) Capa de Lógica de Negocio (Backend):** Gestionará las solicitudes provenientes del frontend y orquesta las pruebas automatizadas de seguridad.
- 3) Capa de Persistencia (Base de Datos y Almacenamiento de Resultados):** La información de vulnerabilidades identificadas se estructura en un modelo de datos optimizado para consultas eficientes.

Adicionalmente:

Integración con Zed Attack Proxy: El backend se conecta a ZAP mediante su API para realizar pruebas automatizadas, utilizando los módulos de Spider Scan API para explorar y mapear el sitio web objetivo, Active Scan API para ejecutar pruebas de seguridad en busca de vulnerabilidades específicas y Core Alerts API para recuperar y procesar los resultados de escaneo.

Docker: El sistema se ejecutará dentro de un entorno de contenedores “dockerizado”, facilitando la portabilidad y la integración continua.

b. Análisis de la integración de Zed Attack Proxy y su integración en la arquitectura.

Se ha analizado la integración de ZAP como el motor principal para la ejecución de las pruebas de seguridad automatizadas, es el componente crítico del sistema. La integración se realizará a través de su API REST. El backend del sistema actuará como cliente de la API de ZAP, enviando comandos para iniciar y gestionar los escaneos, y para recuperar los resultados de las vulnerabilidades detectadas.

La API de ZAP proporciona un conjunto robusto de funcionalidades que se alinean directamente con los tipos de pruebas de vulnerabilidades definidos en el plan, a continuación, se muestra el mecanismo de integración a utilizar y los desafíos técnicos del mismo:

1) Mecanismo de Integración: ZAP se comunica con el backend a través de su API REST. Para cada prueba de seguridad se siguen los siguientes pasos:

- El backend envía una solicitud a la Spider Scan API para rastrear la estructura del sitio web.
- Una vez completado el escaneo, se inicia el Active Scan API, que prueba la seguridad de los componentes web.
- ZAP ejecuta pruebas basadas en un conjunto de reglas predefinidas, detectando las siguientes vulnerabilidades:
 - 40018 - SQL Injection
 - 40012 - Cross-Site Scripting (XSS)
 - 40026 - Weak Hashing Algorithm
 - 40024 - Insecure Direct Object References
 - 40016 - Security Misconfiguration
 - 40027 - Known Vulnerable Components (Retire.js)
 - 40014 - Insufficient Logging and Monitoring
 - 40020 - Server-Side Request Forgery (SSRF)
 - 40022 - Insecure Deserialization
 - 40017 - Authentication Bypass
- Los resultados se recuperan mediante Core Alerts API, permitiendo su almacenamiento en la base de datos y visualización en el frontend.
- Adicionalmente, se identifican todos los endpoints de la API de ZAP que serán utilizados:
 - AscanActionAddScan Policy: Para aplicar políticas de escaneo específicas.
 - AscanActionEnable Scanners: Para habilitar los escáneres de vulnerabilidades deseados.
 - SpiderAction Scan: Para iniciar el rastreo de la estructura del sitio web.
 - Spider View Status: Para obtener el estado del rastreo.
 - SakanAction Scan: Para iniciar el escaneo activo de vulnerabilidades.

- AscanViewStatus: Para obtener el estado del escaneo activo.
- CoreView Alerts: Para recuperar los resultados de las vulnerabilidades encontradas.
- Ascan Action Stop: Para detener el escaneo.

2) Desafíos Técnicos en la Integración

- Consumo de Recursos: ZAP es intensivo en CPU, memoria y red. Se mitiga ejecutándose en contenedores Docker optimizados.
- Tiempo de Ejecución de los Escaneos: Se usa Celery para gestionar tareas en segundo plano, evitando bloqueos en el sistema.
- Interpretación de Resultados: Se normalizan las alertas obtenidas en un formato estructurado para facilitar su análisis.

c. Seleccionar las tecnologías adecuadas para el desarrollo del frontend y backend.

1) Frontend

- Lenguajes: JavaScript, TypeScript, HTML y CSS serán utilizados para el comportamiento, la estructura y el estilo de la interfaz.
- Framework: VueJS + Nuxt, proporcionan una arquitectura modular y optimiza la carga del frontend.
- Framework PrimeVue (WebComponents): Permite desarrollar una interfaz profesional con componentes preconfigurados.

VueJS y Nuxt.js proporcionan una arquitectura moderna y reactiva para la construcción de interfaces de usuario complejas y eficientes. TypeScript añade tipado estático a JavaScript, mejorando la mantenibilidad y reduciendo los errores. Primer Vue ofrece una rica colección de componentes preconstruidos que acelerarán el desarrollo de la interfaz de usuario.

2) Backend

- Lenguaje de programación principal: Python
- Framework: Django, es un framework robusto con herramientas integradas para seguridad y escalabilidad.
- Para MessageBroker Redis - (NoSQL) y para las Task Queue Celery, ambas permiten la ejecución de tareas en segundo plano.
- Base de Datos: Postgres (SQL), como sistema de gestión de bases de datos relacional para el almacenamiento de los resultados de los escaneos y la configuración del sistema.
- WebSockets: Facilita la actualización de los resultados del escaneo.
- Docker se emplea para la contenerización de la aplicación, facilitando su despliegue y gestión.

La selección de Django (Python) en el backend se basa en su robustez, su amplio ecosistema de librerías y su idoneidad para el desarrollo. PostgreSQL ofrece fiabilidad y escalabilidad para el almacenamiento de datos estructurados. Redis y Celery son herramientas probadas para el manejo eficiente de tareas asíncronas, crucial para la ejecución de escaneos que pueden ser de larga duración, sin bloquear la interfaz de usuario, Redis actuará como message broker para la comunicación asíncrona y como caché, mientras que Celery se utilizará como cola de tareas distribuida para la gestión de los procesos de escaneo en segundo plano. Docker asegura la portabilidad y la consistencia del entorno de desarrollo y producción.

d. Configuración del entorno de desarrollo y gestión del control de versiones con Git.

En orden de garantizar un desarrollo eficiente y organizado del sistema, se estableció la configuración del entorno de desarrollo tanto para el backend como para el frontend. Se instalaron todas las herramientas necesarias, incluidos los entornos de desarrollo integrados (IDE), bibliotecas y dependencias.

Además, para la colaboración en el código fuente y la gestión del ciclo de vida del software, se utilizó Git, un sistema de control de versiones distribuido que permite rastrear cambios y coordinar el trabajo en equipo. La estructura del proyecto en Git sigue un modelo de ramificación definido para mejorar la organización y la integración continua:

1) Branching:

- Main Branch: Contiene el código listo para producción.
- Feature Branches: Cada funcionalidad se desarrolla en una rama específica basada en la rama development.
- Convención de nombres: feature/<nombre>, por ejemplo: feature/autenticación.

2) Commits:

- Los mensajes deben ser breves y descriptivos.

3) Pull Requests:

- Todas las ramas deben ser fusionadas mediante Pull Requests.

4) Revisiones de Código:

- Verificación del cumplimiento de estándares de desarrollo.
- Identificación y mitigación de vulnerabilidades antes de la fusión.

5) Resolución de Conflictos:

- Resolver conflictos en la rama antes de fusionar.
- Documentar si los cambios sí afectan funcionalidades críticas.

e. Modelado de la arquitectura estática y dinámica del sistema mediante diagramas.

La utilización de diagramas para visualizar tanto la estructura estática como el comportamiento dinámico del sistema de pruebas automatizadas permitirá abordar la complejidad inherente y asegurar que la implementación se alinee con los requisitos definidos.

De acuerdo con la información recopilada previamente, se ha seleccionado un conjunto de diagramas que ofrecen perspectivas complementarias de la arquitectura del sistema:

- 1) **Diagrama de Casos de Uso:** Este diagrama se centra en la funcionalidad desde la perspectiva del usuario (actor), y su interacción con el sistema de pruebas automatizadas.

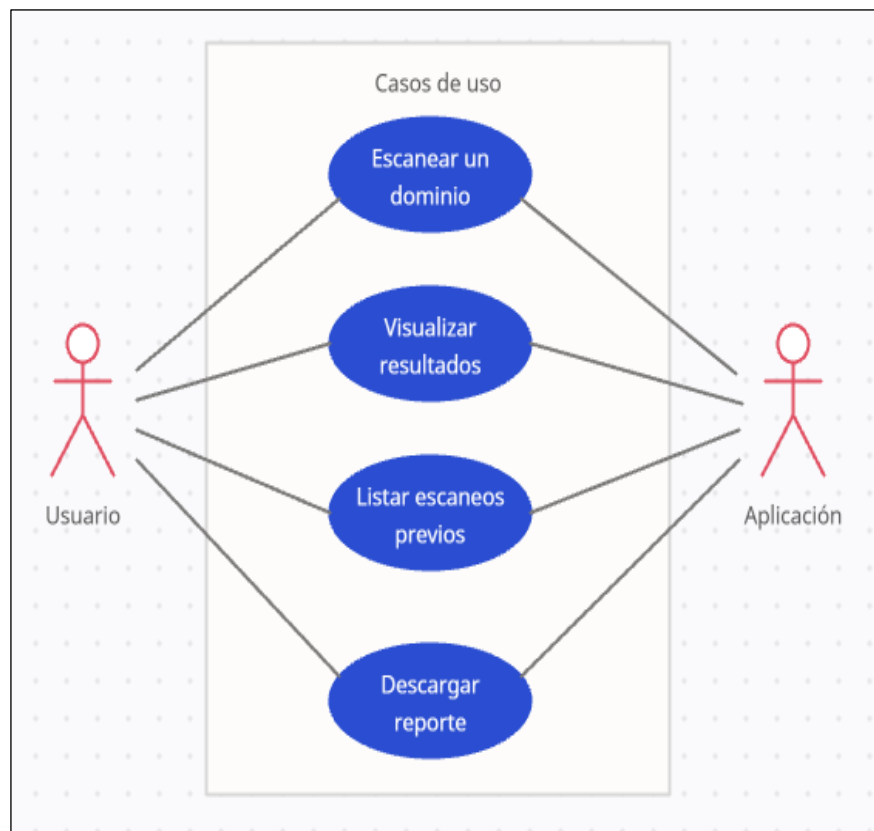


Figura 1: Caso de uso del usuario

2) **Diagrama de Flujo:** Estos diagramas representan la secuencia de actividades o pasos necesarios para llevar a cabo un proceso específico dentro del sistema, en este sentido, se procederán a mostrar los flujos de procesos de cada caso de uso mostrado en el diagrama anterior: escaneo de un dominio y visualizar resultados, listar escaneos previos y descarga de reportes.

□ **Escaneo de un dominio**

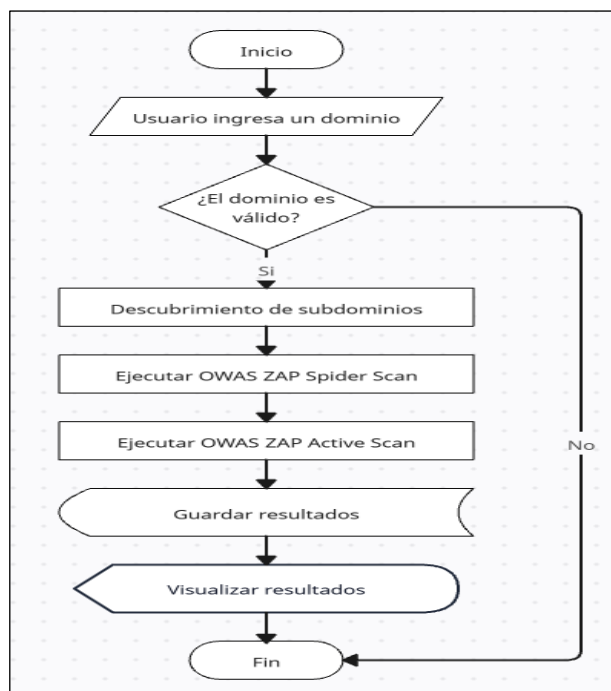


Figura 2: Diagrama Escaneo de un dominio

□ Visualizar resultados

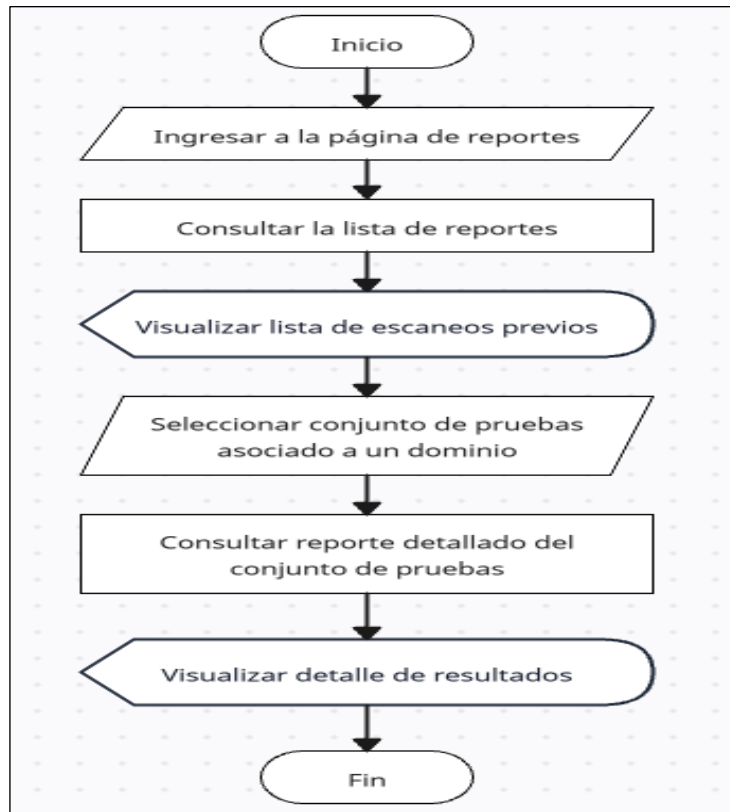


Figura 3: Diagrama de visualizar los resultados

□ Listar escaneos

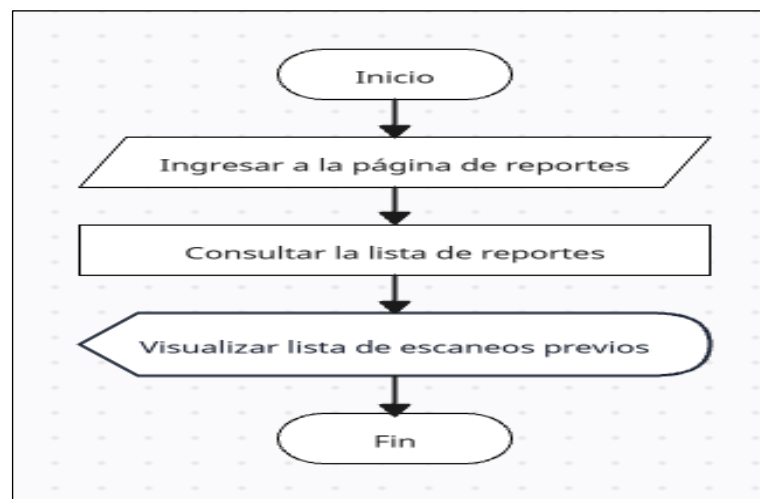


Figura 4: Diagrama para listar escaneos

□ Descargar reporte

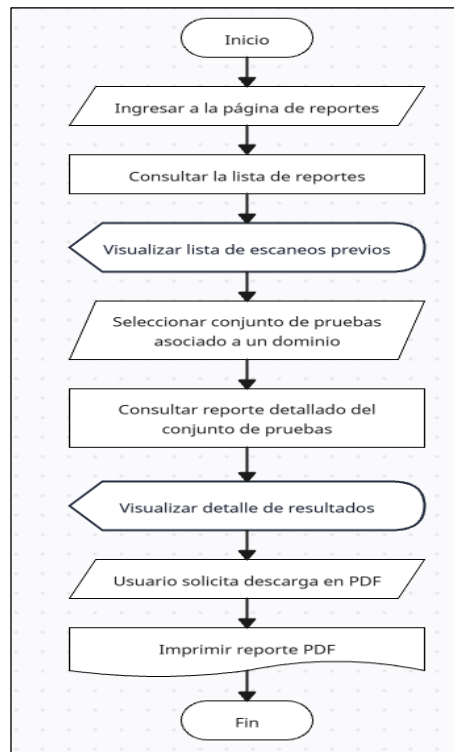


Figura 5: Diagrama para descargar reporte

3) Diagrama de Componentes: Este diagrama ilustra los principales componentes del sistema, sus interfaces, cómo interactúan y las relaciones de dependencia entre ellos. De esta manera se facilita la comprensión de la arquitectura en su totalidad.

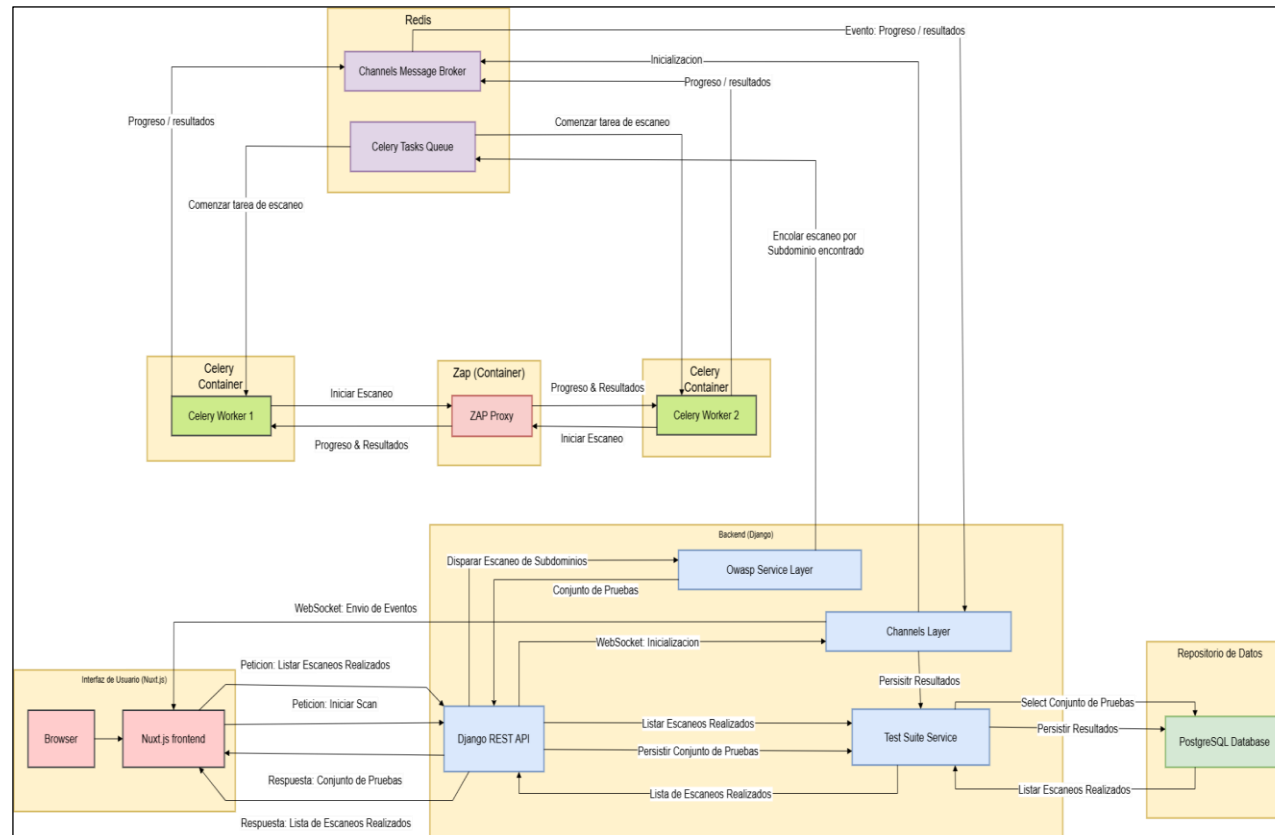


Figura 6: Diagrama de Componentes

f. Modelo entidad-relación (ER) de la base de datos.

Esta será la manera de representar la estructura lógica de los datos que persistirá el sistema, el modelo define las entidades, sus atributos y las relaciones entre ellas.

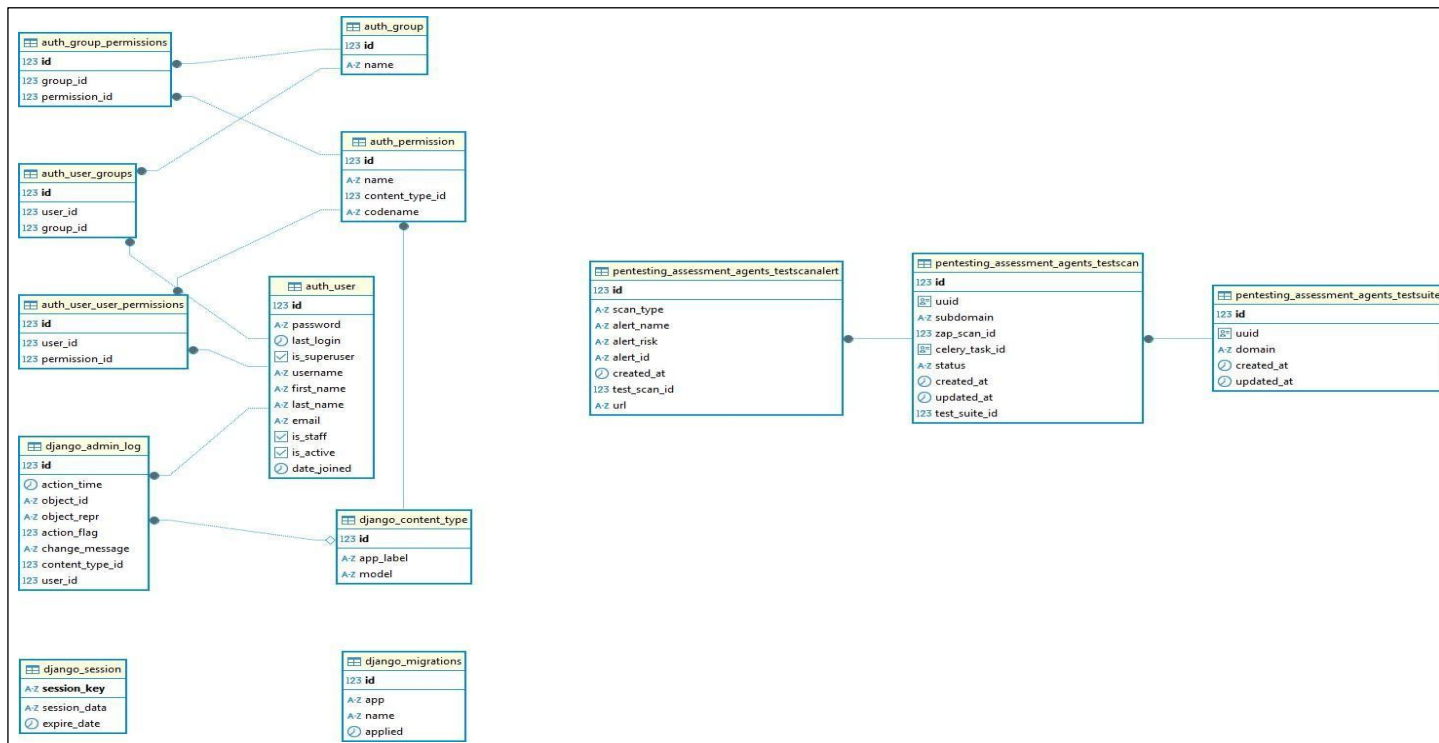


Figura 7: Modelo entidad-relación (ER) de la base de datos

g. Identificación y análisis de posibles desafíos técnicos

Se procede a identificar y analizar los posibles desafíos técnicos del proyecto, y para cada uno, su nivel de impacto y mitigación propuesta.

Tabla 3: Posibles desafíos técnicos

Desafío Técnico	Impacto / Mitigación
Rendimiento del escaneo de ZAP	Impacto: Un escaneo puede tardar varios minutos, afectando la experiencia del usuario. Mitigación: Se ejecutarán escaneos en segundo plano.
Escalabilidad del backend	Impacto: Si se ejecutan varios escaneos simultáneamente, la carga en el servidor puede ser alta. Mitigación: Uso de Redis y Celery para manejar múltiples tareas concurrentes.
Formato y almacenamiento de los resultados	Impacto: Si los datos no están bien estructurados, el análisis de vulnerabilidades será complicado. Mitigación: Se normalizaron los resultados en una estructura de datos optimizada, traducéndose en información comprensible y útil para el usuario final, incluyendo la clasificación de la severidad de las vulnerabilidades encontradas.

Sprint 3: Diseño

Este sprint se enfoca en la transformación de los requerimientos iniciales y el análisis previo en representaciones visuales y estructuradas que servirán como base para el desarrollo del sistema.

Se define la organización gráfica del sistema por medio de la elaboración de prototipos utilizando herramientas de diseño, de esta manera logramos anticipar posibles riesgos, mejorar la experiencia del usuario y facilitar una implementación eficiente y mantenible.

Backlog del Sprint

- a. Diseño de la interfaz de usuario (UI) y experiencia de usuario (UX).
- b. Diseño funcional de la arquitectura del backend.
- c. Definición de los aspectos de seguridad en el diseño.

Resultados

- a. **Diseño de la interfaz de usuario (UI) y experiencia de usuario (UX).**

1) Elementos que debe incluir el diseño:

- Formularios controlados para validación de URL de entrada.
- Indicadores de progreso para escaneos en ejecución.
- Panel de resultados con uso de etiquetas de color por nivel de severidad.
- Botones de acción destacados.
- Iconografía clara para mejorar la comprensión visual de las funciones.

2) Accesibilidad y Responsividad

- El diseño se construyó siguiendo principios de accesibilidad básica, como contraste de colores adecuado, uso de tipografías legibles y navegación coherente.

- También se contempló la adaptabilidad a distintos tamaños de pantalla (diseño responsive), asegurando que el sistema pueda visualizarse correctamente en distintos dispositivos.

3) Estilos para la interfaz de usuario: Para el diseño visual del sistema se adoptó una estética moderna, limpia y funcional, con énfasis en la usabilidad y la accesibilidad. Se combinará el uso del framework PrimeVue para la implementación de componentes de interfaz de usuario con Tailwind CSS como sistema de estilos utilitario, lo que permite un desarrollo ágil y coherente con principios de diseño responsive.

- **Tipografías:** Se utiliza la familia de fuentes predeterminada de Tailwind CSS, la cual incluye una combinación de tipografías sans-serif ampliamente compatibles y visualmente neutras, de esta manera se garantiza una correcta visualización en diferentes sistemas operativos y dispositivos, manteniendo la legibilidad en todo momento. Estas son:

- ☐ ui-sans-serif,
- ☐ system-ui,
- ☐ sans-serif,
- ☐ "Apple Color Emoji",
- ☐ "Segoe UI Emoji",
- ☐ "Segoe UI Symbol",
- ☐ "Noto Color Emoji"

- **Paleta de colores:** Se definió una paleta de colores principal basada en tonos verdes, transmitiendo seguridad, estabilidad y enfoque técnico. Esta paleta, utilizada tanto para elementos principales como para acentos e indicadores, se muestra a continuación:

		50	ecfdf5
- 50: "#ecf5"		100	d1fae5
- 100: "#d1fae5"		200	a7f3d0
- 200: "#a7f3d0"		300	6ee7b7
- 300: "#6ee7b7"		400	10b981
- 400: "#10b981"		600	059669
- 600: "#059669"		700	047857
- 700: "#047857"		800	065f46
- 800: "#065f46"		950	022c22
- 950: "#022c22"			

- 4) **Diseño del Logotipo:** El logotipo del sistema de pruebas automatizadas fue diseñado para reflejar visualmente los conceptos clave del proyecto: seguridad, análisis técnico y automatización. Su construcción se basa en elementos simples pero representativos, que permiten una rápida asociación con el propósito del software.



Figura 8: Diseño del Logotipo

Elementos del logotipo:

- Escudo verde como fondo principal, simbolizando protección, defensa y confianza, valores fundamentales en cualquier solución orientada a la ciberseguridad. El color verde (#10b981), perteneciente a la paleta de Tailwind CSS utilizada en toda la interfaz, refuerza esta sensación de estabilidad, profesionalismo y compromiso con la seguridad.
- Engranajes en color naranja que representan la automatización de procesos técnicos complejos, en este caso, la ejecución de pruebas de seguridad sobre aplicaciones web. El color naranja (#f59e0b o similar) fue elegido color de contraste para destacar los componentes funcionales dentro del escudo. Además, este tono complementa al verde generando un balance visual atractivo y llamativo sin comprometer la legibilidad.
- La lupa colocada sobre los engranajes simboliza el análisis detallado y la inspección profunda del sistema, una alusión directa al proceso de escaneo de vulnerabilidades. Esta fusión entre elementos mecánicos (automatización) y de observación (auditoría) sintetiza perfectamente la esencia del proyecto.
- La combinación de colores asegura una excelente visibilidad tanto en pantallas claras como oscuras. El verde ofrece un fondo sólido que transmite estabilidad, mientras que el naranja proporciona el dinamismo necesario para atraer la atención hacia el componente central del logo. El uso de negro para los contornos y la lupa aporta definición y profesionalismo al diseño.

5) **Estructura general de la interfaz de usuario (UI) y diseño inicial:** La estructura de navegación del sistema se organizó en cinco secciones principales:

- **Inicio:** Página de presentación del sistema, donde se introducen sus funcionalidades, objetivos y características principales.

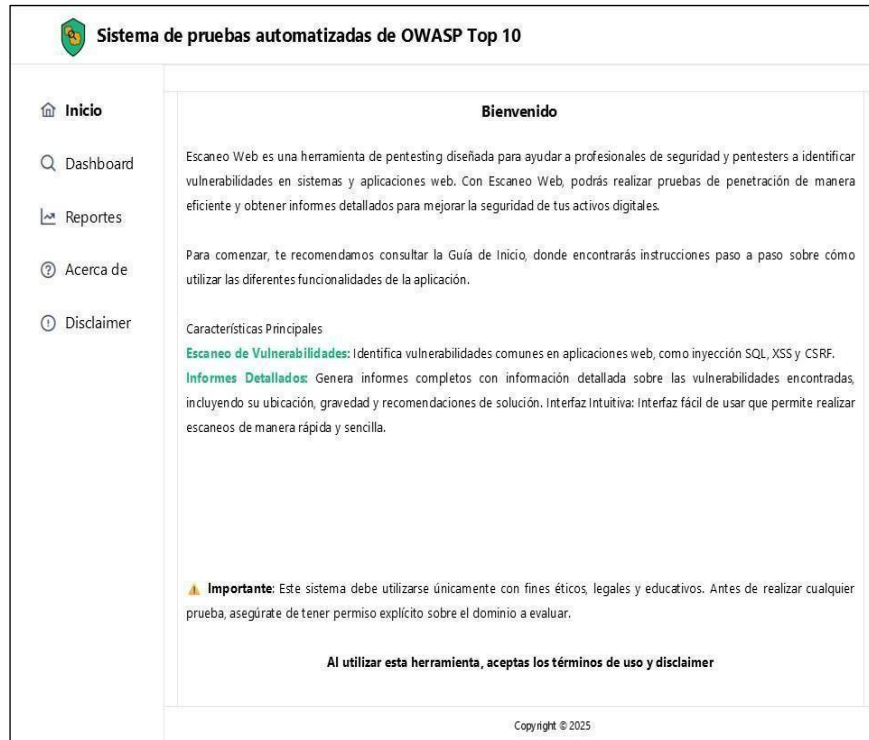


Figura 9: Estructura general de la interfaz de usuario (UI) y diseño inicial

- **Dashboard:** Pantalla principal desde la cual el usuario puede ingresar un dominio para iniciar un nuevo escaneo, y visualizar los resultados al final de este.

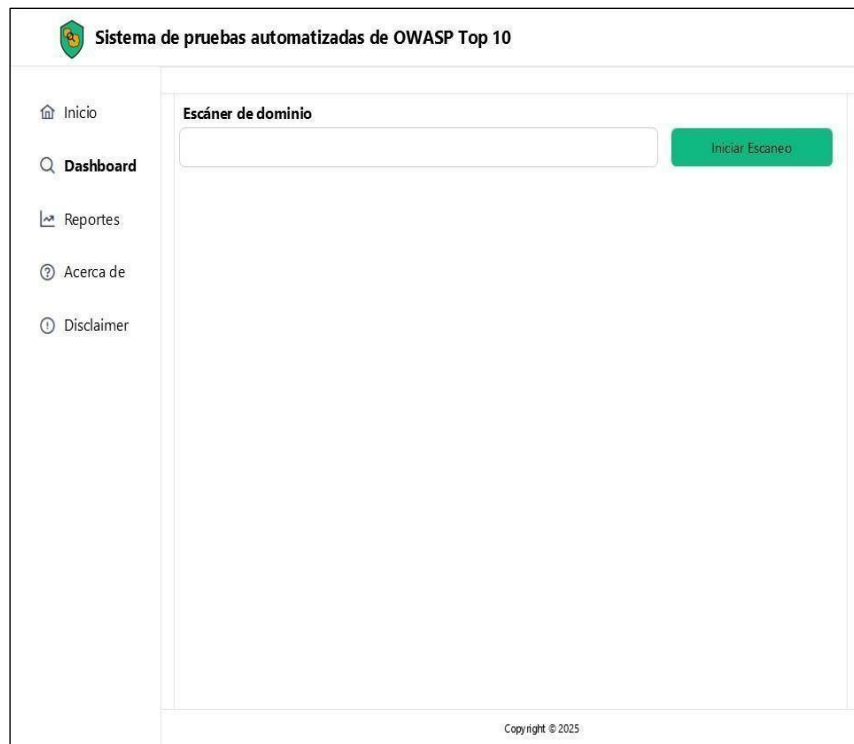


Figura 10: Dashboard

- **Reportes:** Sección que permite acceder al histórico de los escaneos previamente realizados, filtrarlos por dominio, ver el resumen de los hallazgos y descargar los archivos en formato PDF.

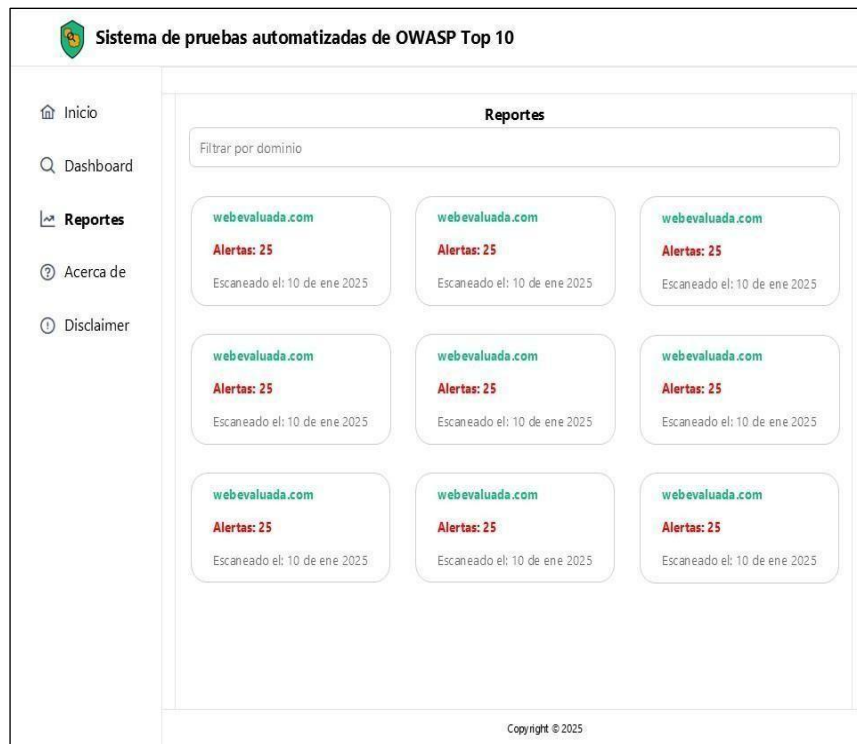


Figura 11: Reportes.

- **Acerca de:** Incluye información técnica sobre el sistema, su propósito académico, tecnologías utilizadas y autores responsables del desarrollo.

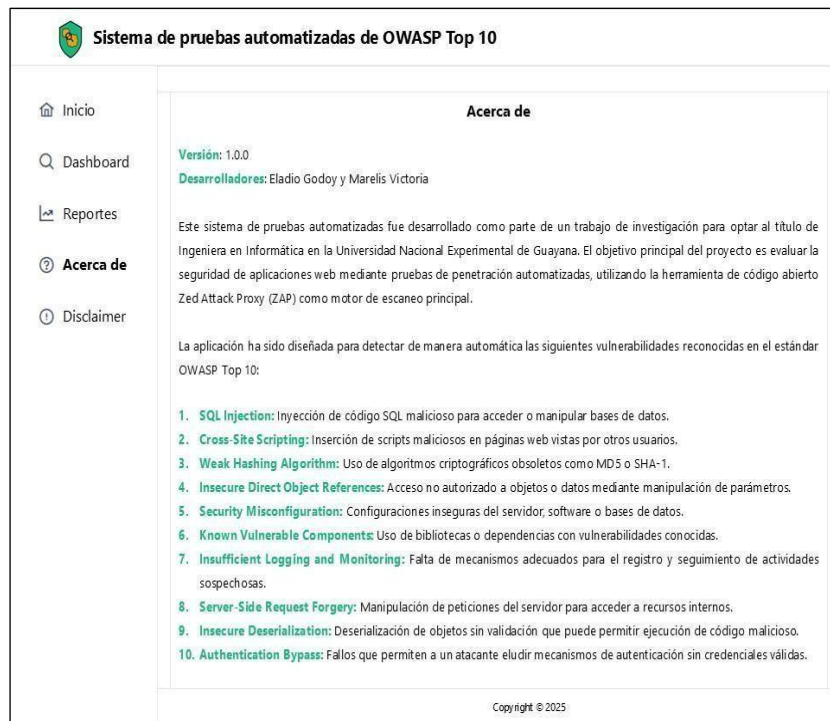


Figura 12: Acerca de

- **Disclaimer:** Pantalla obligatoria para aceptación de términos de uso, condiciones legales y limitaciones del sistema, con el fin de prevenir un uso inadecuado o no autorizado de la herramienta.



Figura 13: Disclaimer.

b. Diseño funcional de la arquitectura del backend.

El diseño funcional de la arquitectura del backend del sistema de pruebas automatizadas debe ser robusto, modular y orientado a tareas asíncronas de alto rendimiento, capaces de gestionar múltiples escaneos de seguridad de forma paralela sin afectar la experiencia del usuario. Para lograrlo, se definió una arquitectura desacoplada basada en servicios que se comunican entre sí mediante colas de tareas, estructuras de datos persistentes y APIs bien definidas.

1) Estructura y componentes del backend: El diseño de la lógica del sistema se dividió en los siguientes módulos funcionales:

- **Controlador de escaneo:** Recibe la configuración ingresada por el usuario a través del frontend con el dominio objetivo ya validado para iniciar el escaneo sobre un dominio, ofrece la posibilidad de listar escaneos previos y los detalles de un escaneo.
- **Componente de gestión de tareas asíncronas:** Utiliza Celery en combinación con Redis como message broker, para ejecutar de forma desacoplada y no bloqueante las pruebas de seguridad automatizadas.
- **Capa de servicio que orquesta la integración con Zed Attack Proxy:** Encapsula la lógica para conectarse con la API de ZAP, lanzar escaneos y recolectar hallazgos.
- **Capa de persistencia y gestión de resultados:** Estructura y guarda la información de las vulnerabilidades detectadas en una base de datos relacional (PostgreSQL), siguiendo el modelo de datos definido en el análisis.
- **Capa de generación de reportes:** Toma los resultados almacenados y genera documentos en formato PDF con el resumen y detalles del escaneo de un dominio, listos para ser descargados por el usuario.

2) Interacción y Comunicación: El diseño para la interacción entre el backend y el frontend queda definido de la siguiente manera:

- La comunicación entre el backend y el frontend se realiza mediante una API RESTful, desarrollada con Django REST Framework, permitiendo que cada acción esté expuesta mediante endpoints bien

estructurados. Para la actualización en tiempo real del estado de los escaneos, se implementaron WebSockets, facilitando una experiencia dinámica sin necesidad de recargar la página.

- El backend también se encarga de interactuar directamente con ZAP, invocando secuencias de APIs que permiten iniciar el Spider Scan, continuar con el Active Scan, y finalmente recuperar los hallazgos mediante Core Alerts.

c. Definición de los aspectos de seguridad en el diseño.

La seguridad fue considerada como un eje transversal en el diseño del sistema de pruebas automatizadas, no solo por su propósito como herramienta de análisis de vulnerabilidades, sino también porque cualquier fallo en su propia arquitectura podría comprometer la confidencialidad, integridad o disponibilidad de los datos procesados. En consecuencia, durante esta fase se establecieron lineamientos y mecanismos clave para mitigar riesgos potenciales desde el diseño.

- 1) Se analizaron los principales vectores de ataque que podrían afectar tanto a los usuarios como a la plataforma en sí. Entre los riesgos identificados destacan:
 - Exposición de endpoints sin autenticación.
 - Inyección de comandos o URLs maliciosas a través de formularios.
 - Fuga de información sensible durante la transmisión de datos.
- 2) Se definieron medidas para asegurar la protección de la información gestionada por el sistema:

- Uso de HTTPS en entornos de despliegue para cifrar las comunicaciones.
- Validación y sanitización de todas las entradas de usuario (por ejemplo, URLs).
- Control de cabeceras HTTP para prevenir ataques como inyección de scripts.

3) Durante esta etapa se establecieron principios de programación segura a ser respetados durante el desarrollo, tales como:

- No exponer información del entorno en mensajes de error.
- Implementar control de excepciones para evitar caídas del sistema.
- Usar librerías mantenidas y actualizadas, evitando dependencias vulnerables.
- Separar responsabilidades entre componentes para minimizar el impacto de posibles fallos.

Sprint 4: Desarrollo

Este sprint se enfoca en la implementación funcional del sistema de pruebas automatizadas, siguiendo la planificación, arquitectura y especificaciones definidas en los sprints anteriores. Materializamos el diseño en componentes reales de software, tanto del frontend como del backend, integrando la lógica del negocio, las interfaces de usuario, los servicios asíncronos y la comunicación con Zed Attack Proxy (ZAP).

Backlog del Sprint

- a. Desarrollo del backend del sistema de pruebas automatizadas.
- b. Especificación de la integración con ZAP.
- c. Desarrollo del frontend del sistema de pruebas automatizadas.

- d. Preparación para la integración.

Resultados

a. Desarrollo del backend del sistema de pruebas automatizadas.

El backend es el responsable de exponer las siguientes funcionalidades concernientes a un conjunto de pruebas: iniciar un escaneo, listar escaneos previos, mostrar los resultados detallados de un escaneo y descargar PDF. De igual forma, orquesta la lógica del escaneo, gestiona tareas asíncronas, interactúa con ZAP, almacena resultados y genera los reportes técnicos. Para su desarrollo, se utilizó el framework Django, acompañado de Django REST Framework, Celery, Redis y PostgreSQL como tecnologías clave.

El backend se organizó siguiendo una arquitectura modular, permitiendo una separación clara de responsabilidades y facilitando el mantenimiento, escalabilidad y extensibilidad futura del sistema.

1) Controlador de conjuntos de pruebas

Es el encargado de recibir la configuración del escaneo desde el frontend, específicamente el ya validado dominio objetivo proporcionada por el usuario, ofrece la posibilidad de listar escaneos previos y los detalles de un escaneo.

- Iniciar escaneo

```
class Run0waspTopScan(APIView):
    """
    API endpoint to trigger vulnerability tests.
    """

    __domain_pattern_regex = re.compile(r"^(?!-)[A-Za-z0-9-]+\.[A-Za-z]{2,}+$")

    def __is_valid_domain(self, domain: str) -> bool:
        return bool(self.__domain_pattern_regex.match(domain))

    def post(self, request, *args, **kwargs):
        domain = request.data.get('domain')
        if not domain or not self.__is_valid_domain(domain):
            raise exceptions.ValidationError("You need to provide a valid domain to scan")
        found_domains = asyncio.run(scanner.main(domain=domain, scan_sub_domains=True))
        return Response(found_domains, status=status.HTTP_200_OK)
```

- Listar conjunto de pruebas realizados

```
Egoday, 7 days ago | 1 author (Egoday)
class TestSuiteListView(ListAPIView):
    serializer_class = TestSuiteSerializer
    pagination_class = ProjectPageNumberPagination
    filter_backends = [SearchFilter]
    search_fields = ['domain']

    def get_queryset(self):
        domain = self.request.query_params.get('domain')
        return fetch_test_suites(domain)

    def list(self, request, *args, **kwargs):
        queryset = self.get_queryset()
        if not queryset.exists():
            return Response(status=status.HTTP_204_NO_CONTENT)
        return super().list(request, *args, **kwargs)
```

- Obtener resultados detallados de un conjunto de pruebas

```
Egoday, 7 days ago | 1 author (Egoday)
class TestSuiteScansWithAlertsView(generics.RetrieveAPIView):
    serializer_class = TestSuiteScansWithAlertsSerializer

    def get(self, *args, **kwargs):
        test_suite_id = kwargs['uuid']
        response = None
        if test_suite_id:
            response = get_test_suite_with_alerts(test_suite_id)

        return Response(data=response, status=status.HTTP_200_OK if response else status.HTTP_204_NO_CONTENT)
```

- Descargar reporte de conjunto de pruebas en PDF

```
@api_view(['GET'])
def generate_test_suite_pdf(request, uuid):
    test_suite = get_test_suite_with_alerts(uuid)
    response = HttpResponse(status=status.HTTP_404_NOT_FOUND)
    logo_path = finders.find("images/logo.png")

    if test_suite:
        html_string = render_to_string('reports/test_suite_report.html', {'test_suite': test_suite, 'report_header_logo_path':
            logo_path})
        pdf_file = HTML(string=html_string).write_pdf()

        response = HttpResponse(pdf_file, content_type='application/pdf')

        response.status_code=status.HTTP_200_OK
        response['Content-Disposition'] = f'attachment; filename="test_suite_{uuid}_report.pdf"'
        return response

    return response
```

2) Componente de gestión de tareas asíncronas

Con el objetivo de no bloquear el servidor durante la ejecución de escaneos, se implementó una solución asíncrona mediante Celery y Redis. Este componente se encarga de gestionar las tareas en segundo plano, permitiendo que los procesos de escaneo se ejecuten sin afectar

la experiencia del usuario en el frontend. La utilización de Redis como message broker asegura una comunicación rápida y confiable entre los servicios.

▪ Celery Worker 1 Docker Service

```
celery-worker:
  build:
    context: ./pentesting_automation_container
    dockerfile: Dockerfile_agent
  container_name: pentester_agent_two
  command: celery --app=pentesting_assessment_agents.agent worker --loglevel=info -Q sequential_tests
  depends_on:
    - pentesting-db
    - redis
    - django
    - celery-root
  environment:
    - DEBUG=1
    - SECRET_KEY=0983098120938019823
    - ALLOWED_HOST=localhost, 127.0.0.1
    - CELERY_WORKER_NAME=pentester_agent_two
    - CELERY_BROKER_URL=redis://redis:6379/0
    - CELERY_RESULT_BACKEND=redis://redis:6379/0
    - CELERY_TASKS_MODULE=[{"tasks_module": "pentesting_assessment_agents.scanner.tasks"}]
  networks:
    - pentesting-network
```

▪ Celery Root Docker Service

```
celery-root:
  build:
    context: ./pentesting_automation_container
    dockerfile: Dockerfile_agent
  container_name: pentester_agent_one
  command: celery --app=pentesting_assessment_agents.agent worker --loglevel=info -Q sequential_tests
  depends_on:
    - pentesting-db
    - redis
    - django
  environment:
    - DEBUG=1
    - SECRET_KEY=0983098120938019823
    - ALLOWED_HOST=localhost, 127.0.0.1
    - CELERY_WORKER_NAME=pentester_agent_one
    - CELERY_BROKER_URL=redis://redis:6379/0
    - CELERY_RESULT_BACKEND=redis://redis:6379/0
    - CELERY_TASKS_MODULE=[{"tasks_module": "pentesting_assessment_agents.scanner.tasks"}]
  networks:
    - pentesting-network
```

- Gestión de tareas asíncronas

```

## Celery related configs
config_from_env = {}
def init_celery_config(celery_worker_name, celery_broker_url, celery_result_backend, celery_tasks_module):
    config_from_env.update({
        "CELERY_WORKER_NAME": celery_worker_name,
        "CELERY_BROKER_URL": celery_broker_url,
        "CELERY_RESULT_BACKEND": celery_result_backend,
        "CELERY_TASKS_MODULES": []
    })
    agent = Celery(celery_worker_name)
    if celery_tasks_module:
        value = celery_tasks_module
        if not value:
            python_logger.info(f'No configuration object was found for CELERY_TASKS_MODULE'. exception_message_config_object_sample")
        else:
            config = []
            try:
                config = json.loads(value)
            except Exception as e:
                raise Exception('CELERY_TASKS_MODULE value is not valid. %s' % (EXCEPTION_MESSAGE_CONFIG_OBJECT_SAMPLE))
            if isinstance(config, list) and len(config) > 0:
                invalid_config_message = 'CELERY_TASKS_MODULE value is not valid. "%s" attribute was not found. %s'
                modules: list = []
                for module in config:
                    if "tasks_module" in module:
                        modules.append(module.get("tasks_module"))
                    if "routing" in module:
                        module_tasks = {}
                        for routing in module.get("routing"):
                            if "task" in routing and "queue" in routing:
                                routing_key = "%s.%s" % (module.get("tasks_module"), routing.get("task"))
                                routing_value = {"queue": routing.get("queue")}
                                module_tasks[routing_key] = routing_value
                            else:
                                raise Exception(invalid_config_message % ("task or queue", EXCEPTION_MESSAGE_CONFIG_OBJECT_SAMPLE))
                        config_from_env.setdefault("CELERY_TASKS_ROUTING", {}).update(module_tasks)
            else:
                python_logger.warning(f'No routing configuration was found for CELERY_TASKS_MODULE, in case you are consider to route tasks, '). {EXCEPTION_MESSAGE_CONFIG_OBJECT_SAMPLE}")

            config_from_env["CELERY_TASKS_MODULES"] = modules
            agent.config_from_object(config_from_env, namespace="CELERY")
            # Adding modules to celery where shared_tasks are defined
            if config_from_env.get("CELERY_TASKS_MODULES") and len(config_from_env.get("CELERY_TASKS_MODULES")) > 0:
                python_logger.info(f'setting tasks modules: {config_from_env.get("CELERY_TASKS_MODULES")}')
                agent.conf.imports = config_from_env.get("CELERY_TASKS_MODULES")

            # Configuring routing for celery tasks tasks
            if config_from_env.get("CELERY_TASKS_ROUTING") and len(config_from_env.get("CELERY_TASKS_ROUTING").keys()) > 0:
                python_logger.info(f'setting tasks routes: {config_from_env.get("CELERY_TASKS_ROUTING")}')
                agent.conf.task_routes = config_from_env.get("CELERY_TASKS_ROUTING")
            else:
                python_logger.warning(invalid_config_message % ("tasks_module", EXCEPTION_MESSAGE_CONFIG_OBJECT_SAMPLE))

        else:
            python_logger.warning(f'No configuration object was found for CELERY_TASKS_MODULE'. {EXCEPTION_MESSAGE_CONFIG_OBJECT_SAMPLE}")

    return agent

# Instantiations
zap = ZAPv2(**zap_options)
channel_layer: InMemoryChannelLayer = get_channel_layer()
celery = None
if all(key in os.environ for key in ['CELERY_WORKER_NAME', 'CELERY_BROKER_URL', 'CELERY_RESULT_BACKEND']):
    logger.info(f"Celery agent configuration detected for {os.environ['CELERY_WORKER_NAME']}")

```

3) Capa de servicio para integración con Zed Attack Proxy

Esta capa encapsula toda la lógica necesaria para interactuar con la herramienta ZAP. A través de la invocación secuencial de sus APIs (SpiderViewScan, AscanActionScan, CoreView Alerts), se ejecutan las pruebas automatizadas y se recuperan los hallazgos.

```

async def __setup_zap(self, zap: ZAPv2, zap_session: str, policy: ZapPolicy):
    logger.info(f"Configuring Policy {policy.name} to zap:")
    zap.ascan.add_scan_policy(scanpolicyname=policy.name, alertthreshold="LOW", attackstrength="LOW")

    for rule_id in policy.zap_rules_ids:
        zap.ascan.enable_scanners(ids=str(rule_id), scanpolicyname=policy.name)

```

```

async def __run_spider_scan(self, zap: ZAPv2, domain: str):
    logger.info("Starting spider discovery")
    zap.spider.scan(domain)
    update_message = {"phase": "Spider Scan", "value": {
        "current_step": 1,
        "total_steps": 2,
        "step_progress": 0,
    }}
    while int(zap.spider.status()) < 100:
        progress = zap.spider.status()
        update_message["value"]["step_progress"] = progress
        await self.__report_scan_update(ZapScanUpdate.PROGRESS, update_message)
        await asyncio.sleep(2)
    logger.info("Spider completed.")
    update_message["value"]["step_progress"] = 100
    await self.__report_scan_update(ZapScanUpdate.PROGRESS, update_message)

```

```

async def start_zap_scan(self):
    logger.info(f"Starting ZAP scan on {self.target}")
    try:
        await self.__setup_zap(self.zap, self.zap_session, self.zap_policy)
        await self.__run_spider_scan(self.zap, self.target)
        self.zap.urlopen(self.target)
        scan_id = self.zap.ascan.scan(url=self.target, scanpolicyname=self.zap_policy.name)
        start_time = time.time()
        logger.info(f"ZAP scan started on {self.target} at {start_time} under scan_id: {scan_id}")
        update_message = {"phase": "active scan", "value": {
            "current_step": 1,
            "total_steps": 2,
            "step_progress": 0,
        }}
    except Exception as e:
        logger.error(f"Error during ZAP scan setup: {e}")
        return

    while True:
        try:
            # Query the scan status from ZAP API
            status = self.zap.ascan.status(scan_id)

            # Check if the status is a valid number
            if status.isdigit():
                progress = int(status)
            else:
                # Handle the case where the scan doesn't exist or is invalid
                if status == "does not exist":
                    logger.warning(f"Scan ID {scan_id} does not exist, scan might have finished or failed.")
                    break
                else:
                    logger.warning(f"Unexpected status response: {status}")
                    break # Break if the status is unexpected

            # If the scan is still in progress (less than 100), report progress
            if progress < 100:
                update_message["value"]["step_progress"] = progress
                await self.__report_scan_update(ZapScanUpdate.PROGRESS, update_message)
                await asyncio.sleep(3) # Wait for 3 seconds before checking again
            else:
                logger.info(f"Scan {scan_id} has completed.")
                break

            # Timeout check to avoid infinite loop
            if time.time() - start_time > self.timeout:
                logger.info(f"ZAP scan timed out after {self.timeout} seconds.")
                break

        except Exception as e:
            logger.error(f"Error during scan status check: {e}")
            break

    logger.info(f"ZAP scan completed")
    update_message["value"]["step_progress"] = 100
    await self.__report_scan_update(ZapScanUpdate.PROGRESS, update_message)

    alerts = self.zap.core.alerts(baseurl=self.target)
    for alert in alerts:
        if isinstance(alert, dict):
            self.results.append({
                "type": "ZAP",
                "alert": alert['alert'],
                "alert_id": alert['id'],
                "risk": str(alert['risk']).lower() if alert['risk'] else "",
                "url": alert['url']
            })
    await self.__report_scan_update(ZapScanUpdate.ALERT, {"phase": self.zap_policy.name, "value": self.results})
except Exception as e:
    logger.error(f"Error during ZAP scan for id '{scan_id}' if not scan_id else scan_id: {e}", exc_info=True)
    await self.__report_scan_update(ZapScanUpdate.FAILED, {"phase": self.zap_policy.name, "value": f"Error during ZAP scan for id '{scan_id}' if not scan_id else scan_id: {e}"})
finally:
    logger.info(f"ZAP scan finished for {self.target} under zap session {self.zap_session.zap} under scanId {scan_id}")
    if scan_id:
        self.zap.ascan.stop(scan_id=scan_id)

```

4) Capa de persistencia y gestión de resultados

Una vez recuperadas las alertas generadas por ZAP, esta capa se encarga de almacenar y organizar la información resultante del proceso de escaneo en una base de datos relacional, en este caso PostgreSQL. El

modelo de datos fue diseñado en el sprint de análisis y ahora se refleja en una estructura de tablas relacionales normalizadas, capturando el dominio objetivo del escaneo y los hallazgos o vulnerabilidades detectadas.

```
__main__.py X
pentesting_automation > pentesting_automation_container > pentesting_assessment_core > test_suite_scan_service > __main__.py > ...

9
10 def create_test_suite(domain):
11     try:
12         test_suite = TestSuite.objects.create(domain=domain, uuid=uuid.uuid4)
13         logger.debug(f"TestSuite #{test_suite.uuid} created for domain: {test_suite.target_url}")
14         return test_suite.uuid
15     except Exception as ex:
16         logger.error(f"There was an issue while creating TestSuite for domain: {domain}. ex => {ex}")
17         raise InternalServerErrorException(
18             detail=f"{ex}"
19         )
20
21 def create_test_suite_with_scans(data: dict):
22     try:
23         test_suite = TestSuite.objects.create(domain=data["domain"], uuid=data["test_suite_id"])
24         logger.debug(f"TestSuite #{test_suite.uuid} created for domain: {test_suite.domain}")
25
26         test_scans = [
27             TestScan(uuid=item["test_scan_id"], test_suite=test_suite, subdomain=item["subdomain"], celery_task_id=item["task"])
28             for item in data["batch"]
29         ]
30
31         TestScan.objects.bulk_create(test_scans)
32
33         return test_suite
34     except Exception as ex:
35         logger.error(f"There was an issue while creating TestSuite for domain: {data['domain']}. ex => {ex}")
36         raise InternalServerErrorException(
37             detail=f"{ex}"
38         )
39
40 def fetch_test_scan(correlation_id: str):
41     if correlation_id:
42         test_scan = TestScan.objects.get(uuid=correlation_id)
43         return test_scan
44
45 def fetch_test_suites(domain=None):
46     queryset = TestSuite.objects.all()
47
48     if domain:
49         queryset = queryset.filter(domain__icontains=domain)
50
51
```

```

__main__.py X
pentesting_automation > pentesting_automation_container > pentesting_assessment_core > test_suite_scan_service > __main__.py > ...

44
45 def fetch_test_suites(domain=None):
46     queryset = TestSuite.objects.all()
47
48     if domain:
49         queryset = queryset.filter(domain__icontains=domain)
50
51
52     queryset = queryset.annotate(
53         total_scans=Count('test_scans', distinct=True),
54         total_alerts=Count('test_scans__alerts', distinct=True)
55     ).order_by('-created_at')
56
57     return queryset
58
59 def fetch_test_suite(correlation_id: str):
60     if correlation_id:
61         try:
62             test_suite = TestSuite.objects.get(uuid=correlation_id)
63             return test_suite
64         except TestSuite.DoesNotExist:
65             return None
66
67 def record_test_scan_failure(correlation_id: str):
68     test_scan = fetch_test_scan(correlation_id=correlation_id)
69     test_scan.status = TestScan.Status.FAILED
70     test_scan.save()
71
72 def create_test_scan_alerts_from_alerts_result(alerts_data: dict):
73     scan_id = alerts_data['message']['scan_id']
74     alert_values = alerts_data['message']['value']
75
76     try:
77         test_scan = TestScan.objects.get(uuid=scan_id)
78
79         alert_objects = [
80             TestScanAlert(
81                 test_scan=test_scan,
82                 scan_type=alert['type'],
83                 alert_name=alert['alert'],
84                 alert_id=alert['alert_id'],
85                 alert_risk=alert['risk'].lower(),
86                 url=alert['url'],
87             )
88         ]
89         for alert in alert_values:
90             pass
91     except Exception as ex:
92         logger.error(f"TestScan with zap_scan_id {scan_id} not found. ex => {ex}")
93         raise InternalServerErrorException(
94             detail=f"{ex}"
95         )
96     alerts = TestScanAlert.objects.bulk_create(alert_objects)
97     test_scan.status = TestScan.Status.COMPLETED
98     test_scan.save()
99     return alerts

```

5) Capa de generación de reportes

El backend incluye una capa dedicada a la generación de informes en formato PDF. Este informe contiene la información general del escaneo, el resumen de hallazgos, su severidad y una lista detallada de cada vulnerabilidad detectada.

- Descargar reporte de conjunto de pruebas en PDF

```

@api_view(['GET'])
def generate_test_suite_pdf(request, uuid):
    test_suite = get_test_suite_with_alerts(uuid)
    response = HttpResponse(status=status.HTTP_404_NOT_FOUND)
    logo_path = finders.find("images/logo.png")

    if test_suite:
        html_string = render_to_string('reports/test_suite_report.html', {'test_suite': test_suite, 'report_header_logo_path':
            logo_path})
        pdf_file = HTML(string=html_string).write_pdf()

        response = HttpResponse(pdf_file, content_type='application/pdf')

        response.status_code=status.HTTP_200_OK
        response['Content-Disposition'] = f'attachment; filename="test_suite_{uuid}_report.pdf"'
        return response

    return response

```

- Muestra del PDF generado con datos simulados

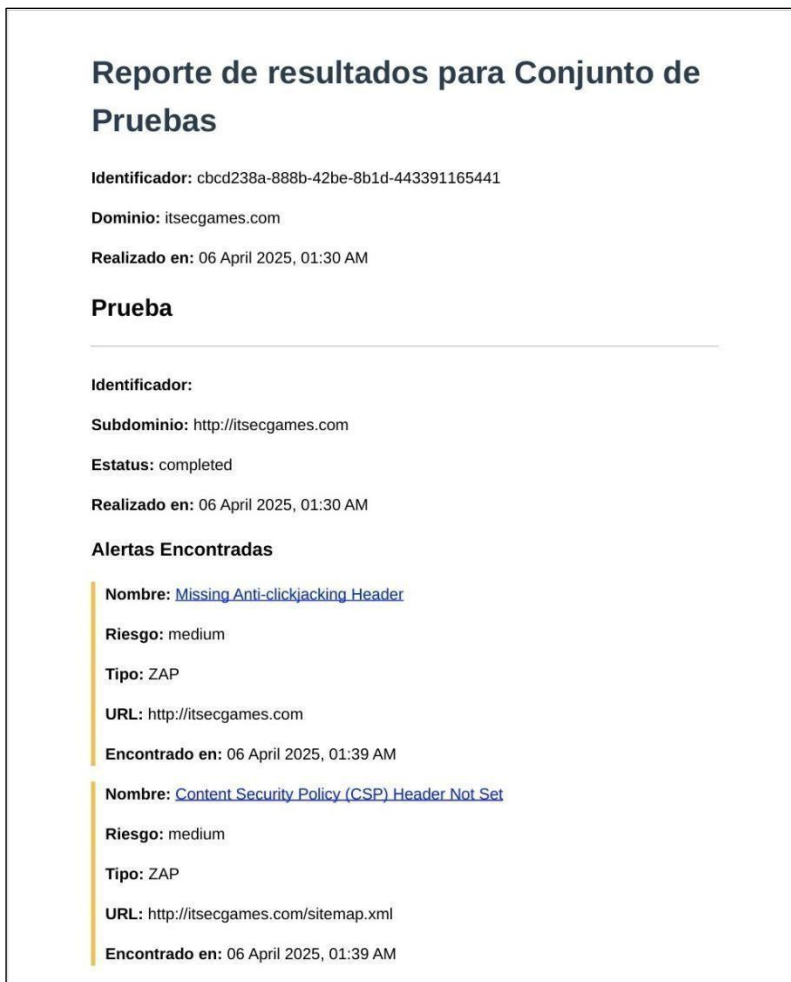


Figura 14: Reporte de resultados para conjunto de pruebas

6) Dependencias y estructura del backend

El desarrollo se apoyó en un conjunto de dependencias cuidadosamente seleccionadas para garantizar la seguridad, estabilidad y compatibilidad del sistema:

- Listado de dependencias

```

pentesting_automation
├── README.md
├── docker-compose.yml
├── pentesting_automation_container
│   ├── Dockerfile
│   ├── Dockerfile_agent
│   ├── django_app.log
│   ├── documentation
│   │   └── diagrams
│   │       ├── components_diagram.md
│   │       └── zenuml_sequence_diagram.md
│   └── entrypoint.sh
├── manage.py
├── pentesting_assessment_agents
│   ├── __init__.py
│   ├── admin.py
│   ├── agent
│   │   ├── __init__.py
│   │   ├── agent_definition.py
│   │   ├── requirements.txt
│   │   └── settings.py
│   ├── apps.py
│   ├── migrations
│   │   ├── 0001_initial.py
│   │   ├── 0002_rename_domain_testscan_subdomain_and_more.py
│   │   └── 0003_alter_testscan_zap_scan_id_alter_testsuite_domain.py
│   ├── models.py
│   ├── scanner
│   │   ├── __init__.py
│   │   ├── main.py
│   │   └── tasks.py
│   ├── serializers.py
│   ├── tests.py
│   └── views.py
├── pentesting_assessment_core
│   ├── __init__.py
│   ├── asgi.py
│   ├── channels.py
│   ├── exceptions.py
│   ├── models.py
│   ├── owasp_top_ten_scanner
│   │   ├── __init__.py
│   │   └── main.py
│   ├── pagination.py
│   ├── settings.py
│   ├── test_suite_scan_service
│   │   ├── __init__.py
│   │   └── main.py
│   ├── urls.py
│   ├── views.py
│   └── wsgi.py
├── requirements.txt
├── static
│   ├── css
│   │   └── tailwind.min.css
│   ├── images
│   │   └── report_header.png
├── templates
│   └── reports
│       └── test_suite_report.html
└── vuln_scan.log

```

▪ Estructura de carpetas del proyecto

```

pentesting_automation_ui
├── README.md
├── app.vue
├── assets
│   ├── css
│   │   └── tailwind.css
│   ├── img
│   │   └── report_header.png
│   └── svg
│       ├── logo-animated.svg
│       └── logo-static.svg
├── components
│   ├── AppConfig.vue
│   ├── AppNotFound.vue
│   ├── AppTestScansDetails.vue
│   └── core
│       ├── AppContainer.vue
│       ├── AppFooter.vue
│       ├── AppLoading.vue
│       ├── AppSideContent.vue
│       ├── AppSideMenu.vue
│       └── AppTopbar.vue
├── composables
│   ├── use-http.ts
│   ├── use-layout.js
│   ├── use-loading.ts
│   ├── use-pentesting-service.ts
│   ├── use-stopwatch.ts
│   └── use-websockets.ts
├── nuxt.config.ts
├── package.json
├── pages
│   ├── dashboard.vue
│   ├── disclaimer.vue
│   ├── index.vue
│   ├── reports.vue
│   ├── test-suite
│   │   └── [id].vue
├── pnpm-lock.yaml
├── public
│   ├── favicon.ico
│   └── robots.txt
├── server
│   └── tsconfig.json
├── tsconfig.json
├── types
│   └── Owasp10.ts
├── utils
│   └── utility.ts

```

b. Especificación de la integración con Zed Attack Proxy.

Uno de los elementos clave en el desarrollo del sistema es la integración con ZAP, una herramienta de análisis de seguridad ampliamente utilizada para pruebas de penetración en aplicaciones web. En este sprint se implementó la automatización completa del flujo de escaneo a través del consumo directo de su API REST, permitiendo lanzar y controlar escaneos desde el backend del sistema.

1) APIs utilizadas de ZAP

Para lograr una integración efectiva, se implementaron llamadas a las siguientes funciones de la API oficial de ZAP:

- **AscanActionAddScan Policy:** Agrega una política de escaneo personalizada.

```
async def __setup_zap(self, zap: ZAPv2, zap_session: str, policy: ZapPolicy):  
    logger.info(f"Configuring Policy {policy.name} to zap:")  
    zap.ascan.add_scan_policy(scanpolicyname=policy.name, alertthreshold="LOW", attackstrength="LOW")
```

Documentación:

<https://www.zaproxy.org/docs/api/#ascanactionaddscanpolicy>

- **AscanActionEnable Scanners:** Activa los escáneres deseados según el tipo de prueba.

```
for rule_id in policy.zap_rules_ids:  
    zap.ascan.enable_scanners(ids=str(rule_id), scanpolicyname=policy.name)
```

Documentación:

<https://www.zaproxy.org/docs/api/#ascanactionenablescanners>

- **SpiderAction Scan:** Inicia el mapeo de la aplicación mediante crawling.

```

async def __run_spider_scan(self, zap: ZAPv2, domain: str):
    logger.info("Starting spider discovery")
    zap.spider.scan(domain)

```

Documentación: <https://www.zaproxy.org/docs/api/#spideractionscan>

- **Spider View Status:** Consulta el estado del Spider Scan.

```

while int(zap.spider.status()) < 100:
    progress = zap.spider.status()
    update_message["value"]["step_progress"] = progress
    await self.__report_scan_update(ZapScanUpdate.PROGRESS, update_message)

```

Documentación: <https://www.zaproxy.org/docs/api/#spiderviewstatus>

- **SakanAction Scan:** Inicia el escaneo activo sobre las rutas descubiertas.

```

scan_id = self.zap.ascan.scan(url=self.target, scanpolicyname=self.zap_policy.name)
start_time = time.time()

```

Documentación: <https://www.zaproxy.org/docs/api/#ascanactionscan>

- **AscanViewStatus:** Verifica el estado del escaneo activo.

```

# Query the scan status from ZAP API
status = self.zap.ascan.status(scan_id)

```

Documentación: <https://www.zaproxy.org/docs/api/#ascanviewstatus>

- **CoreViewAlerts:** Recupera las alertas generadas por los escaneos.

```

alerts = self.zap.core.alerts(baseurl=self.target)

```

Documentación:

<https://www.zaproxy.org/docs/api/?python#coreviewalerts>

- **AscanActionStop:** Detiene el escaneo.

```

finally:
    logger.info(f"Zap Scan finished for {self.target} under zap session {self.zap_session}.zap under scanId {scan_id}")
    if scan_id:
        self.zap.ascan.stop(scanid=scan_id)

```

Documentación:

<https://www.zaproxy.org/docs/api/?python#ascanactionstop>

2) Códigos de vulnerabilidad monitoreados

El sistema fue configurado para capturar e interpretar los siguientes códigos de alerta generados por ZAP:

Tabla 4: Códigos de vulnerabilidad

Código	Vulnerabilidad Detectada
40018	SQL Injection
40012	Cross-Site Scripting (XSS)
40026	Weak Hashing Algorithm
40024	Insecure Direct Object References
40016	Security Misconfiguration
40027	Known Vulnerable Components (Retire.js)
40014	Insufficient Logging and Monitoring
40020	Server-Side Request Forgery (SSRF)
40022	Insecure Deserialization
40017	Authentication Bypass

Estos códigos se interpretan desde la API y se agrupan por tipo de vulnerabilidad y severidad, para su posterior almacenamiento, visualización y generación del informe PDF.

Captura del procesamiento de códigos

```
@shared_task
def exec_zap_scan(domain: str, test_suite_corr_id: str, test_scan_id: str):
    logger.info(f"starting owasp10 scan on {domain}")
    try:
        zap_owasp_10_policy = ZapPolicy(
            name = 'OWASP_TOP_10_POLICY',
            description = 'ZAP Scan Policy for focusing on OWASP Top 10 vulnerabilities',
            zap_rules_ids = [
                # Refer to the ZAP documentation for rule IDs: https://www.zaproxy.org/docs/alerts/
                40018, # SQL Injection
                40012, # Cross-Site Scripting (XSS)
                40026, # Weak Hashing Algorithm
                40024, # Insecure Direct Object References
                40016, # Security Misconfiguration
                40027, # Known Vulnerable Components (Retire.js)
                40014, # Insufficient Logging and Monitoring
                40020, # Server-Side Request Forgery (SSRF)
                40022, # Insecure Deserialization
                40017, # Authentication Bypass
            ]
        )
        scanner = VulnerabilityScanner(test_suite_corr_id, test_scan_id, domain, 600, zap, zap_owasp_10_policy, {'rate_limit_time':
            1, 'rate_limit_max': 10})
        loop = asyncio.get_event_loop()
        loop.run_until_complete(scanner.start_zap_scan())
    except Exception as e:
        logger.error(f"Error: {e}")
        return scanner.results
```

3) Flujo técnico de integración

- El usuario establece un dominio en la vista del dashboard en el cliente.
- Clickea sobre el botón “Iniciar Escaneo”.
- Se realiza una petición HTTP Post al servidor con el dominio en el cuerpo de la petición para iniciar un conjunto de pruebas sobre el dominio.
- Se realiza un descubrimiento de subdominios.
- Se encola una tarea de escaneo por cada subdominio.
- Se ejecuta la tarea en un celery worker.
- Se configura ZAP con una política para escanear las vulnerabilidades deseadas.
- Se realiza el mapeo de la web sobre el subdominio.
- Se consulta el estado del progreso del mapeo hasta que sea completado.
- Se inicia el Active Scan sobre el dominio.
- Se consulta el estado del progreso del Active scan hasta que sea completado.

- Se consultan las alertas encontradas sobre el escaneo una vez el Active Scan ha finalizado.
- Se persisten las alertas encontradas en la base de datos.

4) Beneficios de la integración con ZAP

- El sistema automatiza todo el flujo de escaneo sin requerir intervención del usuario.
- Se garantiza un proceso de evaluación uniforme y repetible.
- Las alertas se traducen en hallazgos comprensibles para los usuarios.
- La API permite escalar el sistema para múltiples escaneos concurrentes.
- Esta integración convierte al sistema en una solución técnica avanzada para análisis de seguridad web, alineada con metodologías de pentesting reales, pero adaptada a entornos automatizados y amigables para el usuario.

c. Desarrollo del frontend del sistema de pruebas automatizadas.

El frontend del sistema fue desarrollado utilizando tecnologías modernas que permiten construir interfaces interactivas, escalables y responsivas. Se utilizaron JavaScript y TypeScript como lenguajes base, junto con los frameworks Vue.js y Nuxt.js, adicionalmente, se combina el uso del framework PrimeVue para la implementación de componentes de interfaz de usuario con Tailwind CSS como sistema de estilos utilitario, lo que permite un desarrollo ágil y coherente con principios de diseño responsivo.

Alineándose con los principios de diseño definidos en el sprint anterior, la interfaz de usuario del sistema se organizó en seis secciones principales, cada una diseñada para ofrecer una experiencia clara y orientada al usuario final.

A continuación, se detallan los componentes desarrollados para cada pantalla y los componentes reutilizables:

1) Pantalla de Inicio

Pantalla de bienvenida al sistema, que presenta sus funcionalidades y las vulnerabilidades detectadas. Integra una presentación visual clara del propósito del sistema y su utilidad para el usuario.

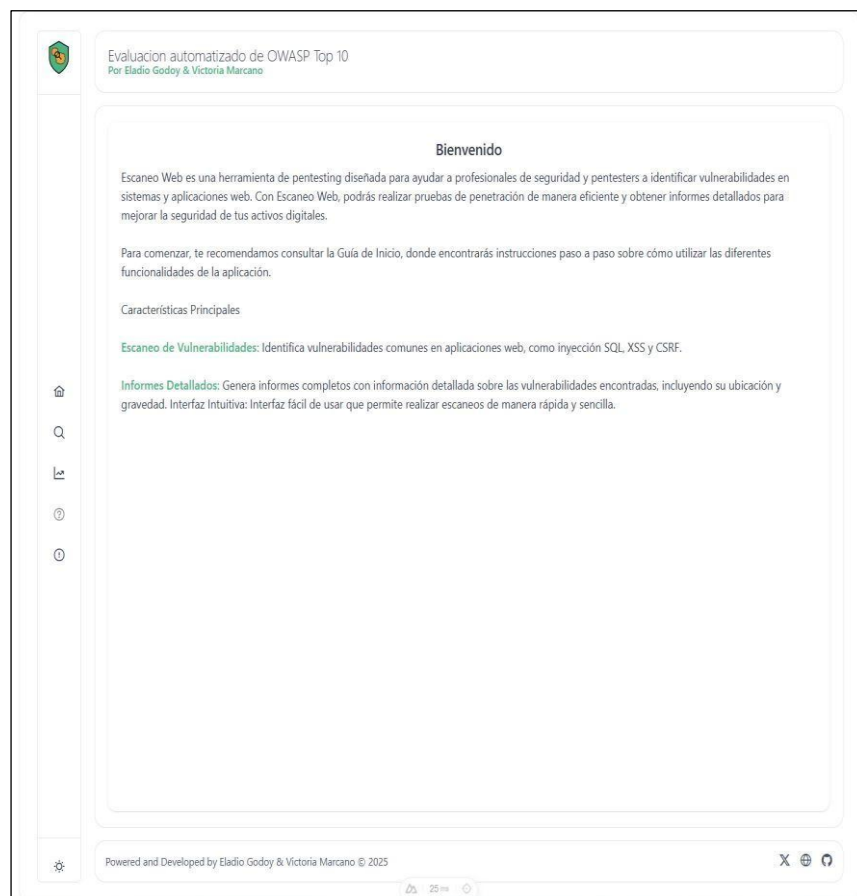


Figura 15: Pantalla de Inicio

2) Pantalla de Dashboard

Desde esta vista, el usuario puede ingresar un dominio objetivo válido, iniciar el escaneo y, al finalizar, visualizar el resumen general de los hallazgos. Se incluye un formulario validado, botón de acción y estado visual del proceso.

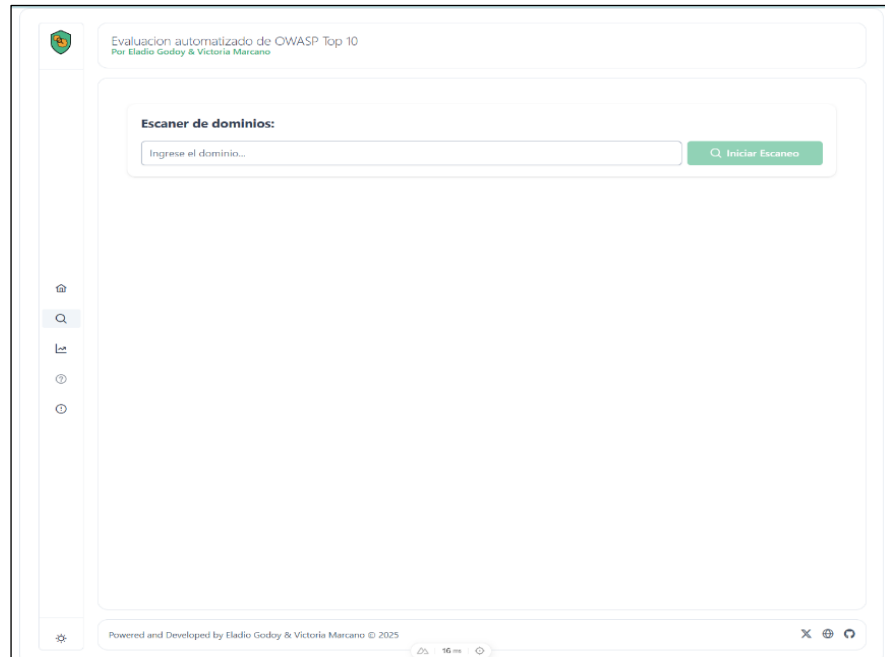


Figura 16: Pantalla de Dashboard

3) Pantalla de Resultados

Una vez finalizado el escaneo, los resultados se muestran organizados por tipo de vulnerabilidad y nivel de severidad. Se utilizaron componentes visuales (etiquetas, tarjetas, badges) para hacer la información más comprensible.

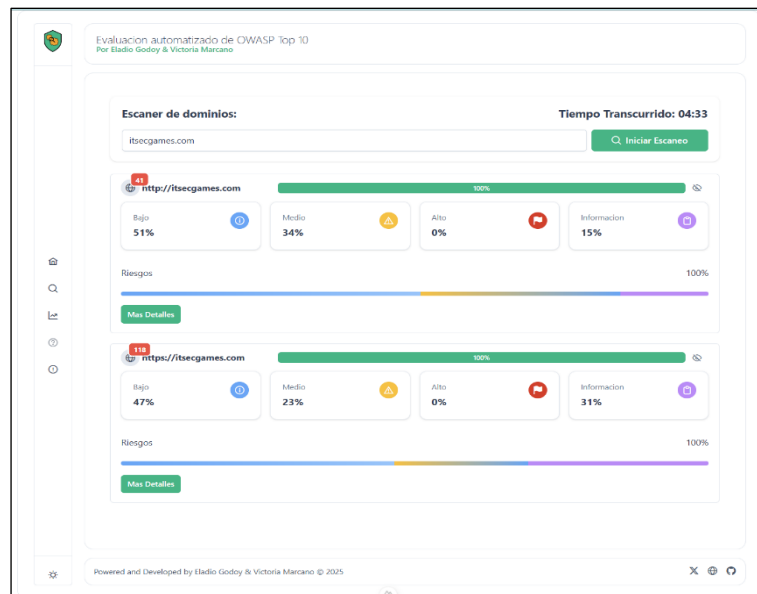


Figura 17: Pantalla de Resultados

4) Pantalla de Reportes

Desde aquí el usuario puede acceder al historial de escaneos previos, visualizarlos individualmente y descargar los reportes generados en formato PDF. Se incluyó funcionalidad para filtrar por dominio.

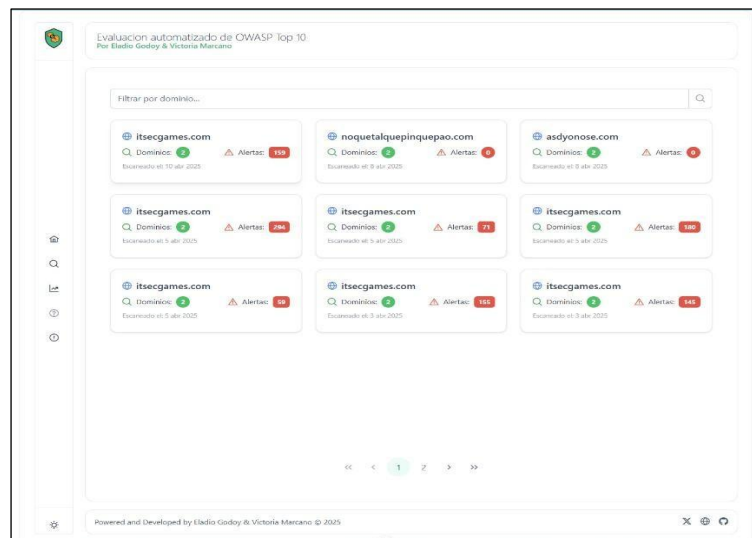


Figura 18: Pantalla de Reportes

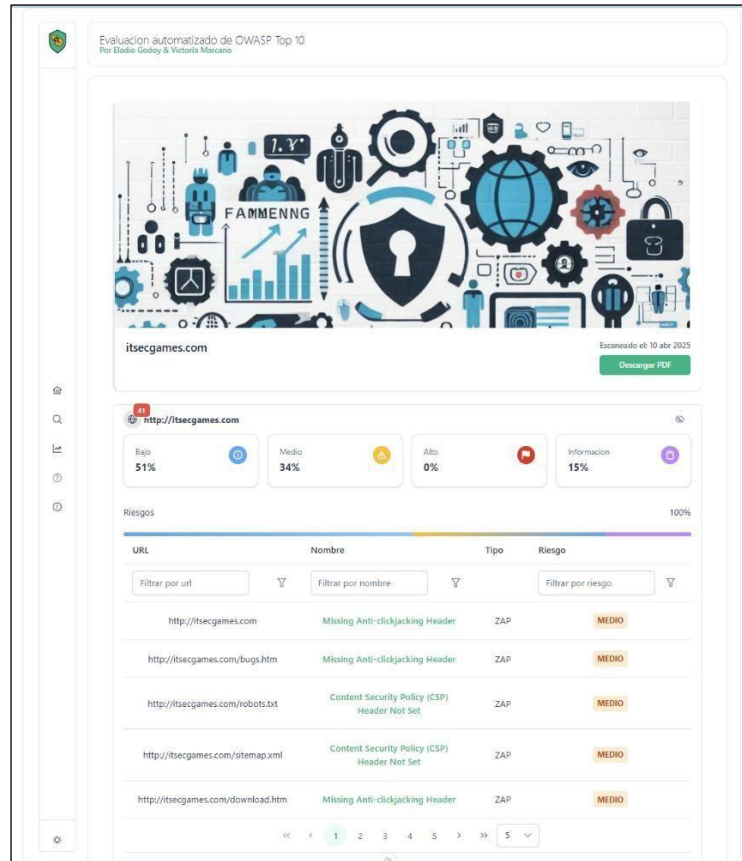


Figura 19: Pantalla de Reportes

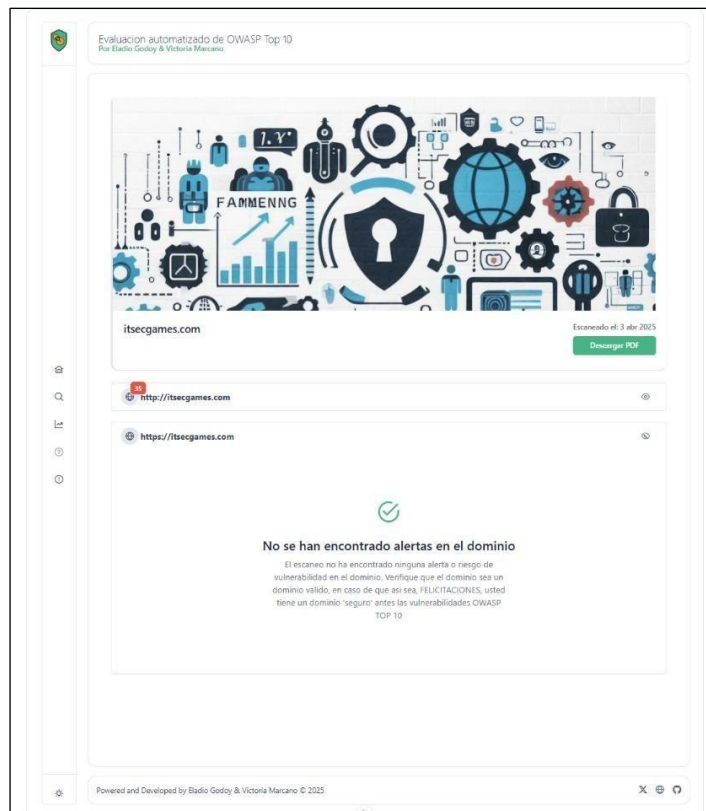


Figura 20: Pantalla de Reportes

5) Pantalla Acerca de

Incluye información técnica sobre el sistema, su propósito académico, tecnologías utilizadas y autores responsables del desarrollo.

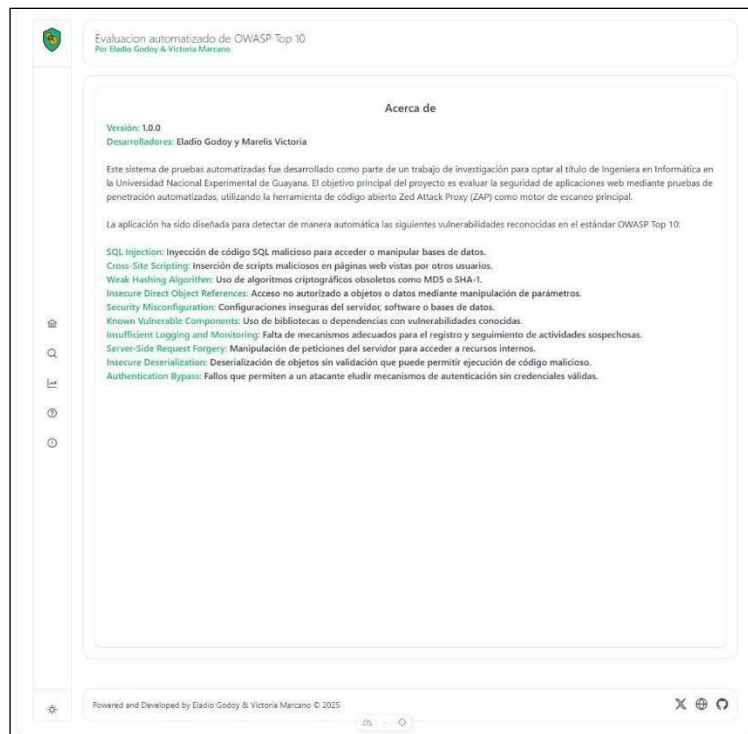


Figura 21: Pantalla Acerca de

6) Pantalla Disclaimer

Esta sección informa al usuario sobre las condiciones legales del sistema, incluyendo un aviso de responsabilidad y la aceptación obligatoria de los términos de uso antes de iniciar cualquier prueba.

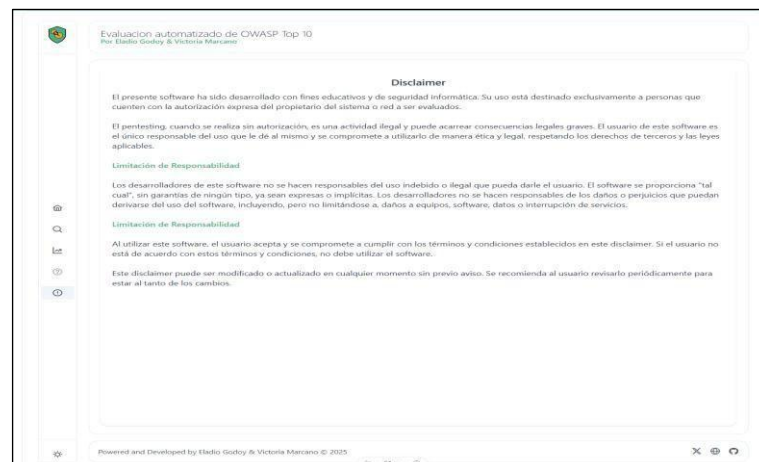


Figura 22: Pantalla Disclaimer

7) Componentes reutilizables con PrimeVue

Durante el desarrollo, se aprovecharon los componentes de PrimeVue para garantizar consistencia visual y acelerar la construcción del frontend. Algunos de los componentes utilizados incluyen: *Input Text*, *Button*, *Card*, *Data Table*, *Dialog*, *Tag*, entre otros.

Sistema de diseño responsive y accesible por defecto.

Todos estos elementos fueron personalizados utilizando Tailwind CSS, respetando la paleta de colores y los estilos definidos en el sprint de diseño.

d. Preparación para la integración.

Una vez completado el desarrollo del backend y frontend, se procedió a la preparación para la integración del sistema. Esta etapa es clave para garantizar que ambos entornos se comuniquen correctamente y que los componentes interactúen como una solución unificada.

La preparación para la integración incluyó la verificación de consistencia entre los endpoints definidos en el backend y las solicitudes realizadas desde el frontend, el control de versiones del proyecto, y la configuración del entorno compartido de desarrollo.

1) Verificación de compatibilidad entre frontend y backend

Se revisaron todos los endpoints implementados en la API REST desarrollada con Django, asegurando que:

- Las rutas esperadas por el frontend estuvieron correctamente definidas y documentadas.
- Las respuestas de la API incluyeran los datos requeridos en los formatos adecuados.
- Se aplicará control de errores y validaciones en ambos extremos para prevenir fallos de integración.

- También se validó que los métodos HTTP (GET, POST) y los códigos de estado (200, 201, 400, 500) estuvieran siendo correctamente manejados en ambos lados.

2) Estandarización de estructura de datos

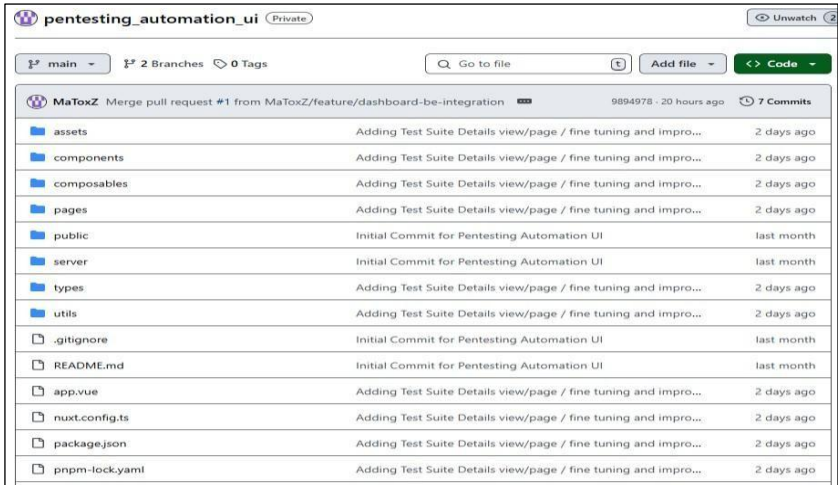
Para facilitar la integración, se definió una estructura de respuesta estándar para las solicitudes del frontend. Esto incluye identificadores únicos para cada escaneo y estructuras anidadas para representar los hallazgos y su severidad.

Esto permitió que la visualización de los resultados fuese consistente, flexible y fácilmente manipulable por los componentes de la interfaz.

3) Gestión del entorno compartido de desarrollo

Se utilizó Git como sistema de control de versiones y herramienta central de colaboración entre los entornos frontend y backend. Se definieron convenciones claras para garantizar la organización y trazabilidad del código, estas fueron mencionadas previamente en el sprint de análisis.

☐ Estructura de los repositorios Git



File/Folder	Commit Message	Time
assets	Adding Test Suite Details view/page / fine tuning and impro...	2 days ago
components	Adding Test Suite Details view/page / fine tuning and impro...	2 days ago
composables	Adding Test Suite Details view/page / fine tuning and impro...	2 days ago
pages	Adding Test Suite Details view/page / fine tuning and impro...	2 days ago
public	Initial Commit for Pentesting Automation UI	last month
server	Initial Commit for Pentesting Automation UI	last month
types	Adding Test Suite Details view/page / fine tuning and impro...	2 days ago
utils	Adding Test Suite Details view/page / fine tuning and impro...	2 days ago
.gitignore	Initial Commit for Pentesting Automation UI	last month
README.md	Initial Commit for Pentesting Automation UI	last month
app.vue	Adding Test Suite Details view/page / fine tuning and impro...	2 days ago
nuxt.config.ts	Adding Test Suite Details view/page / fine tuning and impro...	2 days ago
package.json	Adding Test Suite Details view/page / fine tuning and impro...	2 days ago
pnpm-lock.yaml	Adding Test Suite Details view/page / fine tuning and impro...	2 days ago

Figura 23: Estructura de los repositorios Git

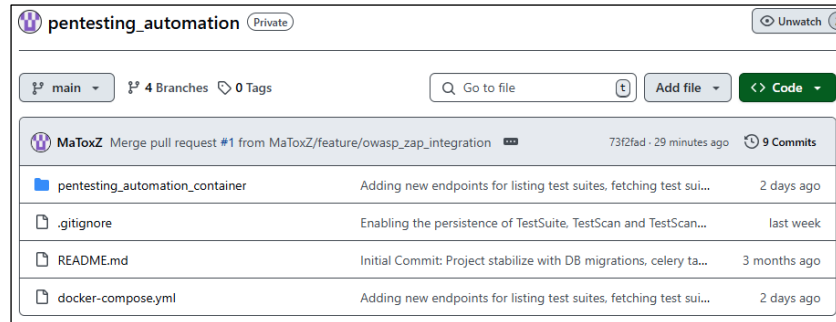


Figura 24: Estructura de los repositorios Git

Esta fase dejó todo listo para el siguiente sprint, donde se llevó a cabo la integración total del sistema y las pruebas funcionales, asegurando una transición fluida y organizada entre el desarrollo aislado de componentes.

Sprint 5: Integración y Pruebas

Una vez implementados todos los componentes del sistema, este sprint se enfocó en la integración total entre el frontend y el backend, unificando los módulos desarrollados para operar como una solución completa y funcional. Además, se realizaron pruebas funcionales y de seguridad utilizando aplicaciones web reales y vulnerables de propósito educativo, asegurando que los resultados obtenidos fueran técnicamente precisos y relevantes.

Backlog del Sprint

- a. Integración del frontend con el backend.
- b. Pruebas funcionales del sistema integrado.
- c. Validación de resultados y detección de errores.

Resultados

a. Integración del frontend con el backend.

Durante esta fase se integraron los endpoints REST desarrollados en Django con la interfaz construida con Vue.js y Nuxt. Se validó que cada acción ejecutada por el usuario en el frontend se comunica correctamente con el backend: iniciar un escaneo, listar escaneos previos, visualizar resultados y descargar reportes PDF.

Se garantizaron respuestas sincronizadas, validación de errores, y retroalimentación visual mediante notificaciones y elementos dinámicos.

b. Pruebas funcionales del sistema integrado.

Se ejecutaron pruebas que validaron todos los casos de uso principales definidos para el sistema, incluyendo:

- Ingreso de un dominio para iniciar un escaneo.
- Inicio y seguimiento del proceso de escaneo.
- Visualización en tiempo real del progreso.
- Revisión detallada de las vulnerabilidades detectadas.
- Listado de escaneos previos con acceso a su información.
- Generación y descarga del reporte en PDF.

Estas pruebas garantizaron que el sistema cubriera los flujos más relevantes desde el punto de vista del usuario, validando tanto entradas esperadas como entradas erróneas.

- Captura de prueba de escaneo



Figura 25: Pruebas funcionales del sistema integrado.

c. Validación de resultados y detección de errores.

Se compararon los resultados obtenidos por el sistema con los esperados en entornos controlados. Para ello, se utilizaron aplicaciones web vulnerables especialmente diseñadas para entrenamiento en seguridad, nuestro caso de prueba utilizó bWAPP (<http://itsecgames.com/>) creada por Malik Messelem.

¿Esta plataforma es una de las recomendadas en el artículo de Recorded Future “What are Vulnerable Websites? Top 12 Sites for Legal Penetration Testing Training” (referencia: <https://www.recordedfuture.com/threat-intelligence-101/vulnerability-management-threat-hunting/vulnerable-websites-for-penetration-testing>), permitiéndonos simular vulnerabilidades reales y comprobar que el sistema identificaba correctamente las fallas esperadas.

Sprint 6: Optimización y Finalización

Luego de integrar y probar completamente el sistema, este sprint se centró en mejorar su rendimiento, usabilidad y presentación visual, además de asegurar la compatibilidad con distintos entornos de ejecución. También se inició la documentación técnica y funcional, con el fin de dejar el sistema listo para su entrega y evaluación académica.

Se eliminan redundancias, se refactoriza código innecesario, y se aplicaron buenas prácticas en la estructura de archivos, nomenclatura y organización del proyecto.

Backlog del Sprint

- a.** Ajustes visuales en el frontend según observaciones de prueba.

- b. Revisión de compatibilidad con navegadores y resolución de pantalla.
- c. Verificación del cumplimiento de los requisitos definidos

Resultados

a. Ajustes visuales en el frontend según observaciones de prueba.

Con base en las pruebas anteriores, se realizaron ajustes visuales para mejorar la presentación de los resultados y la navegación del sistema. Entre los cambios aplicados destacan:

- Rediseño de algunos componentes para facilitar la lectura de las vulnerabilidades.
- Alineación de botones, etiquetas y textos.
- Mejora del contraste visual para mejorar la accesibilidad.
- Optimización del diseño en pantallas pequeñas (responsividad).

b. Revisión de compatibilidad con navegadores y resolución de pantalla.

Se realizaron pruebas cruzadas utilizando navegadores modernos como Chrome, Firefox y Edge, validando que el sistema se renderiza correctamente en cada uno. Además, se probó en distintas resoluciones de pantalla (desde 1366×768 hasta 1920×1080), asegurando que los elementos de la interfaz se adaptarán correctamente.

Evidencia de pruebas en distintos navegadores:

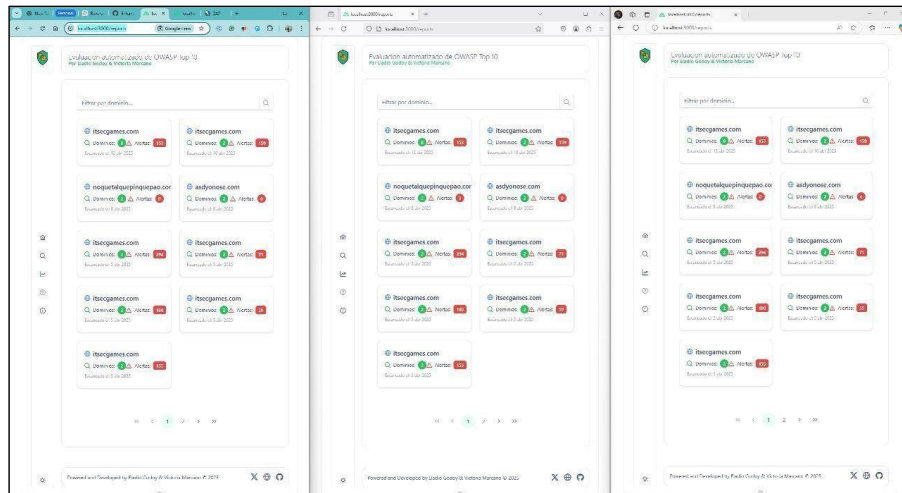


Figura 26: Revisión de compatibilidad con navegadores y resolución de pantalla.

c. Verificación del cumplimiento de los requisitos definidos

Como etapa final del proceso de desarrollo, se realizó una revisión detallada de todos los requisitos definidos en el Sprint 1: Requerimientos Iniciales, asegurando su alineación con el diseño planteado en el Plan de Pruebas Automatizadas y su correcta implementación en el sistema funcional.

Tabla 5: Requisitos Funcionales

Evaluación de Seguridad	Cumplida en su totalidad
Gestión de Resultados	Cumplida en su totalidad
Compatibilidad Técnica	Cumplida en su totalidad
Interfaz de Usuario	Cumplida en su totalidad
Automatización y Flujo de Pruebas	Cumplida en su totalidad

Se verificó que el sistema es capaz de ejecutar pruebas de seguridad automatizadas, detectar vulnerabilidades en sistemas web, visualizar los hallazgos, y generar reportes en formatos adecuados. La interfaz permite configurar escaneos y gestionar los resultados sin requerir intervención técnica avanzada por parte del usuario.

Tabla 6: Requisitos No Funcionales

Rendimiento y Escalabilidad	Cumplida en su totalidad
Compatibilidad Técnica	Cumplida en su totalidad
Usabilidad y Experiencia de Usuario	Cumplida en su totalidad

Se comprobó que el sistema mantiene un rendimiento estable incluso con múltiples escaneos concurrentes. Su arquitectura permite escalar horizontalmente los componentes de backend si fuera necesario. Asimismo, el diseño de la interfaz y su responsividad garantizan una experiencia de usuario coherente y accesible desde distintos dispositivos y navegadores.

La verificación del cumplimiento de los requisitos confirma que el sistema desarrollado no solo responde adecuadamente a los objetivos definidos al inicio del proyecto, sino que también cumple con los estándares técnicos y de seguridad establecidos.

Esta validación garantiza que la solución propuesta puede ser utilizada como una herramienta confiable de análisis de vulnerabilidades web, integrando automatización, eficiencia y facilidad de uso en un solo entorno.

CONCLUSIONES

Con relación al análisis del estado del arte de la seguridad de los sistemas web, nos muestra un panorama desafiante donde la proactividad, automatización, la integración de la seguridad en el desarrollo y una comprensión profunda de las amenazas son esenciales para proteger los sistemas web de manera efectiva. En ese sentido se logró revisar los siguientes aspectos:

- La amenaza es constante y evoluciona rápidamente, con atacantes más sofisticados y técnicas de ataque más complejas, ya no se trata de vulnerabilidades conocidas sino de nuevas formas de explotación y ataques dirigidos.
- Con la creciente adopción de microservicios, APIs, aplicaciones móviles y la integración con sistemas de terceros aumenta potencialmente las debilidades en un sistema web.
- Las aplicaciones modernas son más complejas con múltiples capas, lenguajes de programación, frameworks y dependencias, lo que introduce más posibilidades para errores de seguridad y dificulta la identificación y mitigación de vulnerabilidades.
- La cultura de seguridad y la toma de consciencia son cruciales pero a menudo insuficientes.
- La automatización y herramientas especializadas son esenciales.
- La seguridad por capas sigue siendo un principio fundamental.
- La seguridad debe integrarse en el ciclo de vida del desarrollo de software
- La gestión e identidades de acceso (IAM) es crítica.
- La protección de datos sensible es una prioridad.
- La monitorización y respuesta a incidentes son vitales.

En cuanto a la Caracterización de las vulnerabilidades de los sistemas web, para identificar el nivel de riesgo en las plataformas, ya no es una simple lista de fallos, sino un proceso complejo que requiere una comprensión profunda de las arquitecturas web, las últimas tendencias en ataques y la aplicación de metodologías rigurosas para asegurar la resiliencia de los sistemas web.

Con respecto al diseño del plan de pruebas automatizadas con base a la información recopilada para evaluar la seguridad de un sistema web, es una inversión crucial que impacta positivamente en la eficiencia, la calidad y el costo del proceso, del desarrollo del software. Permite al equipo de pruebas enfocarse en aspectos más críticos y complejos, mientras que las pruebas automatizadas se encargan de la validación rutinaria y exhaustiva de las funcionalidades del sistema.

En base al Desarrollo del sistema de pruebas automatizadas con base a la información recopilada para evaluar la seguridad de un sistema web, utilizando la metodología SCRUM, dota al equipo de pruebas con herramienta poderosa y eficiente para asegurar la calidad del software de manera continua. Es un paso fundamental hacia la optimización del ciclo de vida del software y la reducción de riesgos asociados a la liberación de nuevas versiones.

Por último, en cuanto a la aplicación del sistema de pruebas automatizadas, utilizando metodologías de pentesting para generar un reporte detallado con los hallazgos encontrados, es indispensable para las organizaciones que buscan desarrollar y mantener software de alta calidad de manera eficiente y competitiva en el mercado actual, se traduce en un retorno de la inversión positivo gracias a la optimización de recursos y la mejora de la calidad del producto final.

RECOMENDACIONES

A continuación, se presentan las recomendaciones en base al trabajo de investigación realizado:

Con relación a la UNEG, se recomienda promover la enseñanza de seguridad de sistemas web a través de pruebas automatizadas de penetración, considerarlo en el Pensum de estudios, dado que este tema es de gran importancia para alcanzar un perfil profesional en el área de informática, ajustado a las exigencias en el entorno laboral.

En cuanto al uso del software, considerar lo siguiente:

1. Seleccionar la herramienta propuesta en este trabajo de grado, porque permite en forma eficaz identificar vulnerabilidades como inyecciones SQL, XSS y configuraciones incorrectas.
2. Establecer el alcance de las pruebas, especificando qué sistemas, aplicaciones o redes serán evaluados, esto evita interrupciones no deseadas en otros sistemas.
3. Aunque la herramienta automatizada es rápida y eficaz, se recomienda complementar los resultados con validaciones manuales para evitar falsos positivos o negativos.
4. Configura las pruebas para imitar ataques reales, como intentos de explotación de vulnerabilidades conocidas o ataques de fuerza bruta.
5. Actualizar regularmente la herramienta de prueba, mantener las últimas bases de datos de vulnerabilidades, para detectar amenazas emergentes.
6. Generar informes detallados, documentando los hallazgos con claridad, incluyendo las vulnerabilidades detectadas, su impacto potencial y recomendaciones para mitigarlas.
7. Dar cumplimiento a las normativas y estándares, que las pruebas cumplan con regulaciones como GDPR, ISO 27001 o PCI DSS, según sea aplicable.

REFERENCIAS

- Aguilar, J. (2021). Retos y oportunidades en materia de ciberseguridad de América Latina frente al contexto global de ciber amenazas a la seguridad nacional y política exterior.
- Arias, F. (2012). El proyecto de investigación. 6ta ed. [Documento digital]. Consultado el 15 de octubre del 2024 en <https://abacoenred.org/wp-content/uploads/2019/02/El-proyecto-de-investigaci%C3%B3n-F.G.-Arias-2012-pdf-1.pdf>
- Bernal, C. (2010). Metodología de la investigación. 3ra ed. [Documento digital]. Consultado el 30 de septiembre del 2024 en <https://abacoenred.org/wp-content/uploads/2019/02/El-proyecto-de-investigaci%C3%B3n-F.G.-Arias-2012-pdf.pdf>
- Camacho, H. y Finol M. (2008). El proceso de investigación Científica. Maracaibo, Venezuela: Editorial Ediluz.
- Castro, C. (2018). Pruebas de penetración e intrusión, para optar al Título de especialista en Seguridad Informática en la Universidad Piloto de Colombia.
- Cegarra SJ. (2004) Metodología de la investigación científica y tecnológica. [Documento digital]. Consultado el 20 de Julio en <https://books.google.com.pe/books?id=8SA8KZyurk4C&printsec=frontcover&hl=es#v=onepage&q&f=true>
- Cisco, (2018). Seguridad de la Información.
- Diario.com (2024).
- Diaz, E. (2018). Análisis de metodologías para pruebas de penetración mediante ethical hacking – Bogotá, Colombia, en la Universidad Nacional Abierta y a Distancia “UNAD” Escuela de Ciencias Básicas, Tecnología e Ingeniería proyecto de seguridad informática.
- Dowd, M., McDonald, J., and Schuh, J. (2006). The Art of Software Security As. Ing Software Vulnerabilities
- IBM (2024). Pruebas de penetración.

Instituto Nacional de Ciberseguridad, 2021. Ciberseguridad.

Fortinet, Sophos, Trend Micro, Watchguard y Kaspersky (2024). Computing (2024).

EY Global Cybersecurity Leadership Insights Study (2024).

Normas ISO 27001, HIPAA, SOC 2 y GDPR.

Glosario de términos de ciberseguridad (2021). Una guía de aproximación para el empresario.

Guía de Pruebas OWASP (2008).

Hernández Sampieri, R., Collado, C., Lucio, P. (1991). Metodología de la Investigación. [Documento online]. Consultado el 29 de agosto del 2024 en https://www.uv.mx/personal/cbustamante/files/2011/06/Metodologia-de-la-Investigaci%C3%83%C2%B3n_Sampieri.pdf

Kennedy, D. O’Gorman, J. Kearns, D. and Aharoni, Mati. (2011). Metasploit-The-Penetration-Tester-s-Guide. San Francisco.

Ley Sobre Mensajes De Datos Y Firmas Electrónicas (2000).

Ley Orgánica De Telecomunicaciones (2000).

Ley Especial Contra Los Delitos Informáticos (2001).

Ley Orgánica De Ciencia, Tecnología e Innovación (2005).

López, R & Deslauriers, J. (2011). La entrevista cualitativa como técnica para la investigación en Trabajo Social. [Documento online]. Consultado el 30 de Agosto del 2024 en <http://148.202.167.116:8080/xmlui/bitstream/handle/123456789/2711/La%20entrevista%20cualitativa%20como%20t%C3%A9cnica%20para%20la%20investigaci%C3%B3n.pdf?sequence=1&isAllowed=y>

Nieto, E. (2018, 25 de junio). Tipo de investigación. [Documento digital]. Consultado en 20 de agosto del 2024 en <http://repositorio.usdg.edu.pe/bitstream/USDG/34/1/Tipos-de-Investigacion.pdf>

Norma ISO/IEC 27032, (2012). Protección de Datos.

- Ortiz, V. (2019, 05 de febrero). Investigación de campo y técnicas e instrumentos para la recolección de datos. [Documento online]. Consultado el 20 de septiembre del 2024 en https://www.academia.edu/38307187/Investigaci%C3%B3n_de_campo
- Palella, S y Martins, F. (2006). Metodología de la investigación cuantitativa. [Documento digital]. Consultado el 20 de agosto del 2024 en <https://gc.scalahed.com/recursos/files/r161r/w23578w/w23578w.pdf>
- Pressman, R. (2010). Ingeniería del software: Un enfoque práctico. Disponible en: <http://cotana.informatica.edu.bo/downloads/Id-Ingenieria.de.software.enfoque.practico.7ed.Pressman.PDF> [Consultado: 2023 junio 04].
- Sabino, C. (1992). El proceso de investigación. [Documento digital]. Consultado el 15 de octubre del 2024 en https://www.perio.unlp.edu.ar/tif/wp-content/uploads/2021/04/CarlosSabino-ElProcesoDeInvestigacion_0.pdf
- Valencia, D., (2021). Gestión de la Seguridad de la Información.
- Schwaber, K. & Sutherland, J. (2020). La Guía Scrum.
- Taylor, S & Bogdan, R. (1994). Introducción a los métodos cualitativos de investigación. [Documento online]. Consultado el 18 de octubre del 2024 en <https://pics.unison.mx/maestria/wp-content/uploads/2020/05/Introduccion-a-Los-Metodos-Cualitativos-de-Investigacion-Taylor-S-J-Bogdan-R.pdf>
- VueJS Organization (2023). What is vue?. Documentación en línea. Disponible en: <https://vuejs.org/guide/introduction.html>. [Consultado: 2023 octubre 26].
- Weidman, G. (2014). Penetration testing. A Hands-On Introduction to Hacking. San Francisco.
- Wiley, J. & Sons. (2017). Advanced Penetration Testing. Canadá. Arias, F. (2012). El proyecto de investigación: introducción a la metodología científica. Sexta Edición, Editorial Episteme.